RAMAIAH
Institute of Technology

Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of AI and ML/AI and DS

Course Name: Introduction to Artificial Intelligence

Course Code: AI35/AD35

Credits: 3:0:0

2023 Batch

Introduction

- **Why** consider artificial intelligence to be a subject most worthy of study?
- **What** exactly it is?
- Our intelligence is very important to us.
- For thousands of years, we have tried to u**nderstand how we think and act**?- perceive, understand, predict, and manipulate a world far larger and more complicated than itself

RAMAIAH
Institute of Technology

Introduction: What is AI?

- Artificial intelligence (AI) is the study of ideas that enable **computers to be intelligent**.

- The goals of the field of AI are
  - To make **computers** more useful
  - To understand the **principles** that make intelligence possible.
  - **Building intelligent entities**—machines that can compute how to act effectively and safely in a wide variety of novel situations.

Introduction: What is AI?

- Some have defined intelligence as:
  - Fidelity to **human performance**
  - An abstract, formal definition of intelligence called **rationality**—loosely speaking, doing the **"right thing."**
  - Property of **internal thought processes and reasoning**
  - Focus on **intelligent behavior** and external characterization.

Goals of AI

Artificial Intelligence emphasizes three cognitive skills of learning, reasoning, and self-correction, skills that the human brain possess to one degree or another. **Define these in the context of AI as:**

1. **Learning:** The acquisition of information and the rules needed to use that information.
2. **Reasoning:** Using the information rules to reach definite conclusions.
3. **Self-Correction:** The process of continually fine-tuning AI algorithms and ensure that they offer the most accurate results they can.

Goals of AI

Researchers and programmers have extended and elaborated the goals of AI to the following:

Intelligent Agents : **It** is an entity that perceives its environment and takes actions to achieve its goals. It can be software-based (like a chatbot) or hardware-based (like a robot).

Logical Agents : **P**erceives the environment, represents knowledge in a logical form, and uses reasoning to derive conclusions and determine actions using formal languages such as propositional logic or first-order logic (FOL).

knowledge-based agent uses knowledge to make decisions. The KB is a collection of facts and rules about the world, and the agent uses logical reasoning to deduce new information and decide on actions.

Classical planning involves generating a sequence of actions to achieve a specific goal, assuming a fully observable and deterministic environment.

Goals of AI

Example Scenario: Imagine a Intelligent Agent Robot acting as an intelligent agent tasked with navigating a building.

Perception: The robot perceives that the door ahead is blocked (sensors provide this information).

Knowledge Base: The robot updates its knowledge base to include the fact: “Door 1 is blocked.”

Logical Reasoning: The robot uses logic to reason, "If Door 1 is blocked, I cannot go through it." It infers that it needs an alternative route.

Classical Planning: Using classical planning, the robot generates a new plan by evaluating other routes to reach its goal (e.g., going through Door 2 instead).

Unit I

Introduction: (Chapter 1 of Text Book 1)

- What is AI?
- History of AI
- Stages of AI
- Domains of AI

Intelligent Agents: (Chapter 2 of Text Book 1)

- Agents and Environments
- Rationality
- The Nature of Environments
- The Structure of Agents

Solving Problems by search: (Chapter 3 of Text Book 1)

- Problem-Solving Agents
- Example Problems
- Search Algorithms
- Uniformed Search Strategies- uniform-cost search
- Informed (Heuristic) Search Strategies- Greedy best-first search
- A* search.

Text Books:

1. Stuart J Russel and Peter Norvig: “Artificial Intelligence - A Modern Approach”, 4th Edition, Pearson Education, 2021.
2. Tom M Mitchell, “Machine Learning”, McGraw-Hill Education (Indian Edition), 2013.
3. Elaine Rich, Kevin Knight, Shivashankar B Nair: Artificial Intelligence, 3rd Edition, Tata McGraw Hill, 2011.

/A.I. TIMELINE

1950
TURING TEST

Computer scientist Alan Turing proposes a test for machine intelligence. If a machine can trick humans into thinking it is human, then it has intelligence

1955
A.I. BORN

Term 'artificial intelligence' is coined by computer scientist, John McCarthy to describe "the science and engineering of making intelligent machines"

1961
UNIMATE

First industrial robot, Unimate, goes to work at GM replacing humans on the assembly line

1964
ELIZA

Pioneering chatbot developed by Joseph Weizenbaum at MIT holds conversations with humans

1966
SHAKEY

The 'first electronic person' from Stanford, Shakey is a general-purpose mobile robot that reasons about its own actions

A.I. WINTER

Many false starts and dead-ends leave A.I. out in the cold

1997
DEEP BLUE

Deep Blue, a chess-playing computer from IBM defeats world chess champion Garry Kasparov

1998
KISMET

Cynthia Breazeal at MIT introduces KISmet, an emotionally intelligent robot insofar as it detects and responds to people's feelings

1999
AIBO

Sony launches first consumer robot pet dog AiBO (AI robot) with skills and personality that develop over time

2002
ROOMBA

First mass produced autonomous robotic vacuum cleaner from iRobot learns to navigate and clean homes

2011
SIRI

Apple integrates Siri, an intelligent virtual assistant with a voice interface, into the iPhone 4S

2011
WATSON

IBM's question answering computer Watson wins first place on popular $1M prize television quiz show *Jeopardy*

2014
EUGENE

Eugene Goostman, a chatbot passes the Turing Test with a third of judges believing Eugene is human

2014
ALEXA

Amazon launches Alexa, an intelligent virtual assistant with a voice interface that completes shopping tasks

2016
TAY

Microsoft's chatbot Tay goes rogue on social media making inflammatory and offensive racist comments

2017
ALPHAGO

Google's A.I. AlphaGo beats world champion Ke Jie in the complex board game of Go, notable for its vast number (2^{170}) of possible positions

Stages of AI?

- **In modern science, Artificial Intelligence has three stages. They are as follows:**

1). Artificial Narrow Intelligence (ANI)

2). Artificial General Intelligence (AGI)

3). Artificial Super Intelligence (ASI)

Stages of AI: Artificial Narrow Intelligence(ANI): Weak AI

- ANI **performs a set of instructions only for the specified data**.
- It is made for a specific set of tasks and **trained for a well-defined and limited range of activities**
- These systems are **not capable of generalizing their knowledge** to perform tasks beyond their predefined scope.
- Often called **Weak AI**, ANI is the AI already **embedded in our daily lives.**
- Example: Virtual Assistant such as Apple Siri, Amazon Alexa, and Google Assistant.

Stages of AI: Artificial General Intelligence (AGI): Strong AI

- AGI is also referred to as **Strong AI.**
- It aims to **replicate human-level intelligence and cognitive abilities in machines**.
- AGI systems possess the capability to **understand, learn, and adapt to a wide range of tasks, similar to a human being.**
- General AI is a **long-term aspiration** in the field of AI, we have **not yet achieved it**.
- Most of the AI systems in use today, including the most advanced ones, are still considered **Narrow AI.**
- Achieving General AI remains a significant challenge due to the **complexity of human cognition** and the need for **machines to possess common sense, intuition, and a deeper understanding of the world.**
- Example: ChatGPT, Self-Driving Car, Expert Systems

Stages of AI: Artificial Super Intelligence(ASI)

- Artificial Intelligence system that has **decision-making ability and the ability to mimic human intelligence is way better than human**, is termed as ASI.
- It is the system which **surpasses human abilities**.
- The idea of **ASI raises important ethical and existential questions**, as the emergence of super intelligent AI could have **implications for humanity**.
- ASI has huge **potential but** it also has **huge risk**.
- In the present industry, this type of system is available only in the form of **fictions**.
- Example, in Avengers: Endgame (Movie),

Domains of AI

- The term "Domains of AI" refers to the various **specialized areas within artificial intelligence.**
- These Artificial Intelligence domains deal with **specific problems, techniques, and applications,** making it easier to **categorize and understand** the vast field of AI.
  1. Machine Learning
  2. Deep Learning
  3. Natural Language Processing
  4. Computer Vision
  5. Data Science

Domains of AI: Machine Learning

Data:

Machine learning algorithms rely on large sets of data (examples, observations) to learn patterns and relationships. Data can be structured (like spreadsheets) or unstructured (like text or images).

Training:

During training, the machine learning model is exposed to a dataset and adjusts its internal parameters to improve its performance on a specific task.

Model:

A machine learning model is a mathematical representation or algorithm that tries to map input data to desired output. This model is "trained" using data and is then used to make predictions or decisions.

Domains of AI

Learning Algorithms

Machine learning uses various algorithms to train models, such as:

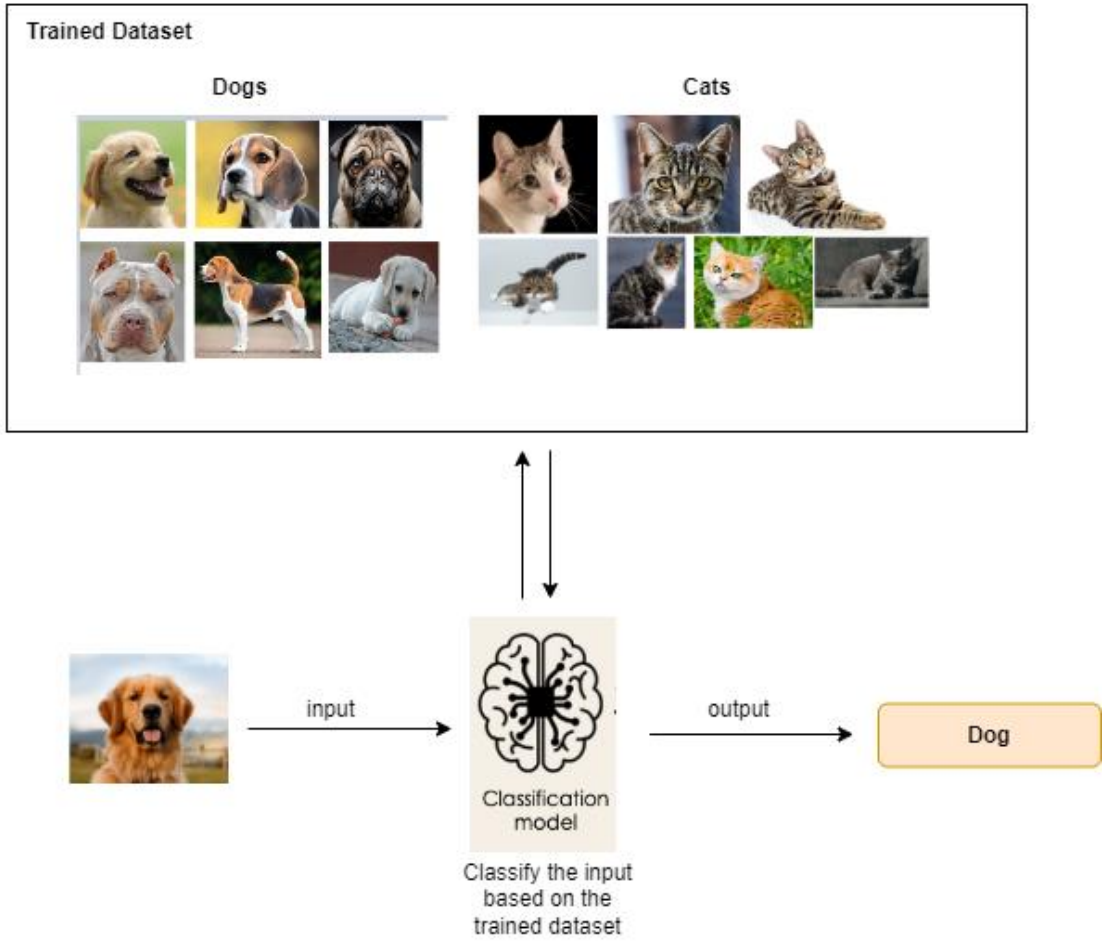
- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning
- Semi-Supervised Learning

Domains of AI

Supervised Learning

In Supervised learning a model is trained on a labeled dataset. The dataset used for training contains both input data and the corresponding label.

- Image Classification
- Speech Recognition
- Fraud Detection
- Medical Diagnosis
- Recommender Systems



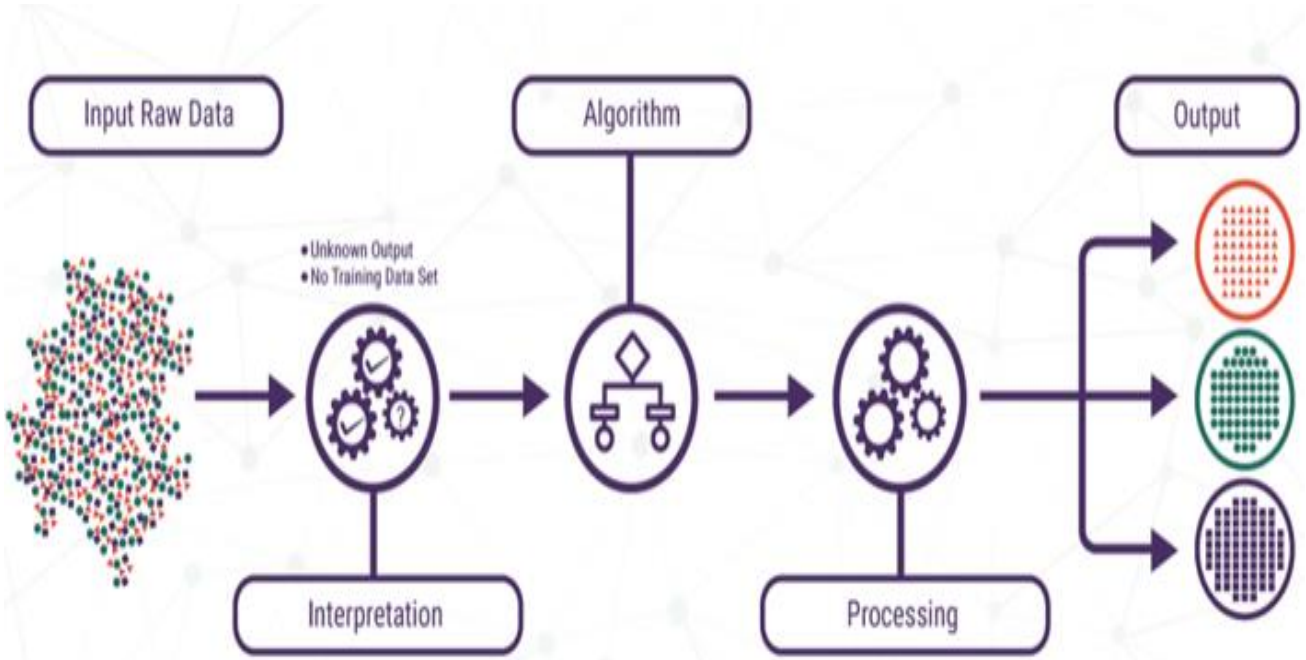
Domains of AI

Unsupervised learning

- Algorithm the model is not provided with labeled data.
- The model then learns patterns and structures in the data without any specific guidance.

Example: Customer segmentation help companies better understand their customers and improve their marketing strategies.

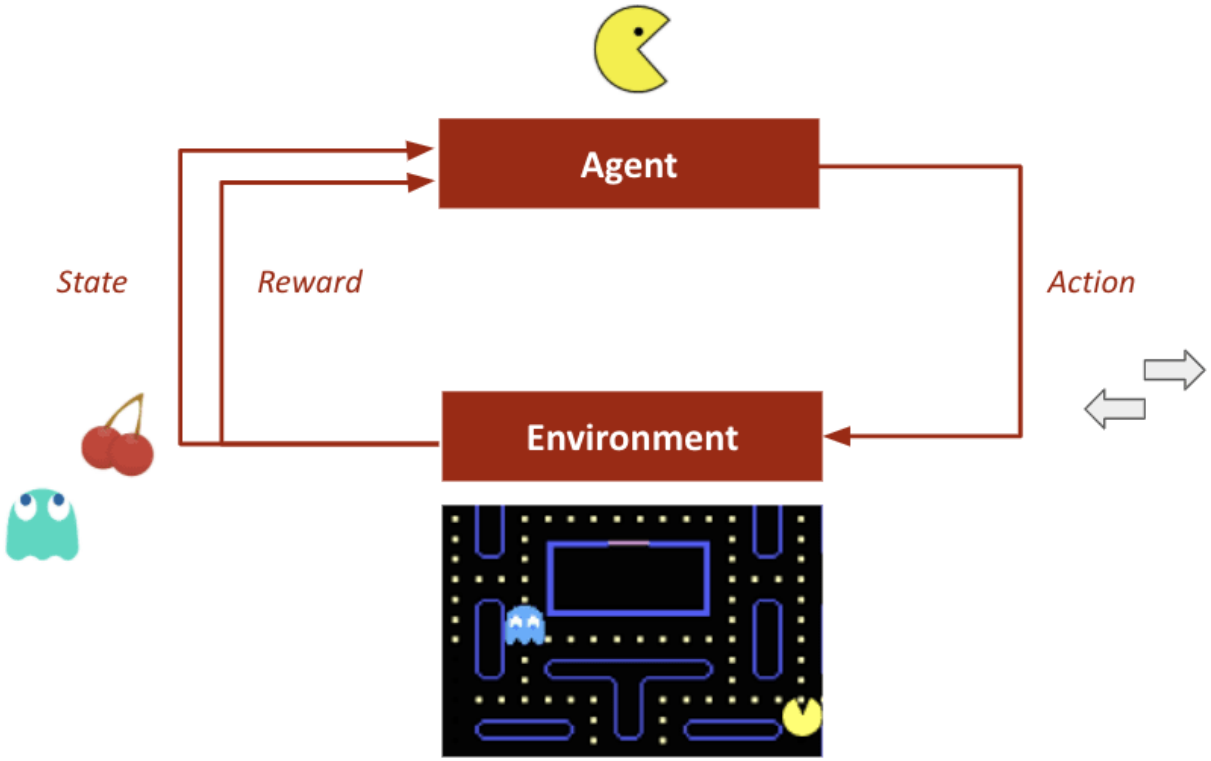
Categorizing articles in news sections



Domains of AI

Reinforcement learning

- It is an approach to teaching machines how to **learn through trial and error**.
- The technical aspect of reinforcement learning involves building an agent that can perceive and interpret its environment, take actions based on that interpretation, and receive feedback in the form of rewards or punishments. **Examples**: Robotics for industrial automation, Autonomous self-driving cars, Game playing



Domains of AI

2. Deep Learning

Deep Learning is a subset of Machine Learning that uses artificial neural networks to model and solve complex problems.

It can handle unstructured image and speech.

Applications of Deep Learning:

Image Classification

Autonomous Vehicles

Healthcare Diagnostics

Gaming and Robotics

Domains of AI

3. Natural Language Processing : It is a domain that focuses on enabling machines to understand, , and generate human language.

- NLP's significance lies in its ability to bridge the communication gap between humans and machines.
- **NLP is used in various applications, including:**
  - **Chatbots and Virtual Assistants**
  - **Sentiment Analysis**
  - **Language Translation**
  - **Information Extraction**
  - **Text Summarization**

Domains of AI

4. Computer Vision enables machines to process and interpret visual data. It involves tasks like object detection, image classification, and video analysis.

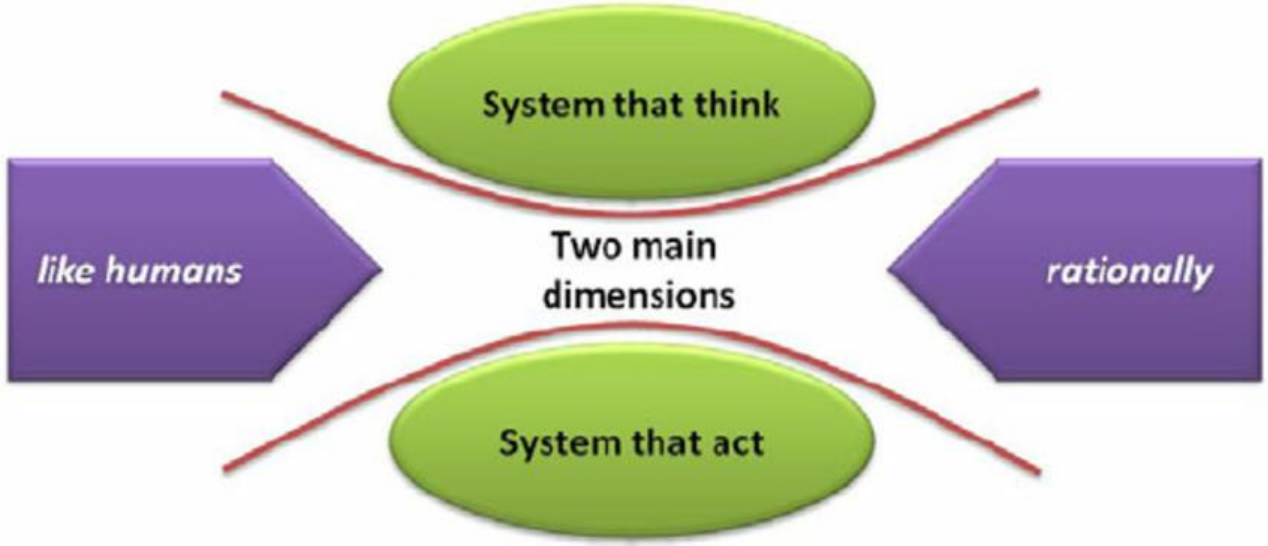
5. Data Science involves the collection, analysis, and interpretation of data to make informed decisions.

Data Science is crucial for AI because it provides the data required for training and testing AI models. It helps organizations make data-driven decisions and predictions

What is Artificial Intelligence?

- AI definitions can be viewed into two dimensions.
- **In the first dimension,** AI can be regarded as a system that think like humans or that thinks rationally.
- **In second dimension,** AI is viewed as a system that acts like humans or that act rationally.

	Humanly	Rationally
Thinking	Think Humanly (e.g., cognitive modeling)	Think Rationally (e.g., logic)
Acting	Act Humanly (e.g., Turing test)	Act Rationally (e.g., engineering)

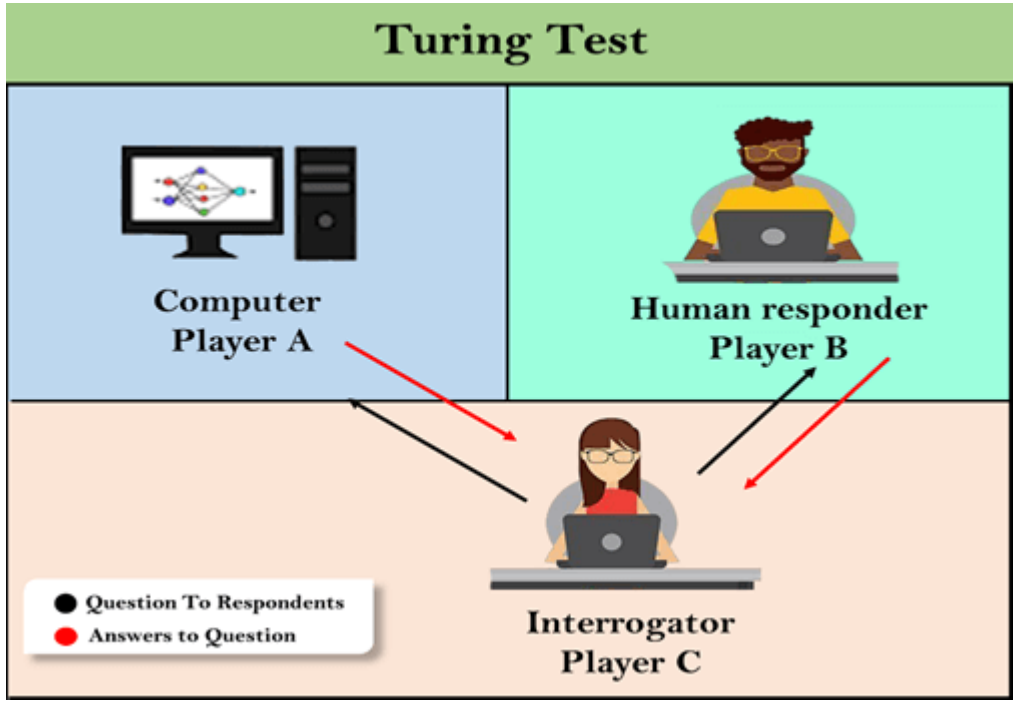


What is Artificial Intelligence?

- From the two dimensions—
  - **Human vs. Rational and Thought vs. Behavior**
  - There are four possible combinations.
  - **The activity of human-like intelligence** : related to **psychology, involving observations and hypotheses** about actual human behavior and thought processes;
  - **The rationalist approach** : statistics, control theory, and economics, a combination of mathematics and engineering

What is Artificial Intelligence? (contd..)

- Acting humanly: The Turing Test approach
- A human interrogates the computer and another human via a terminal simultaneously.
- After a reasonable period, a computer passes the test if a human interrogator, after posing some written questions, and cannot tell whether the written responses come from a person or from a computer.

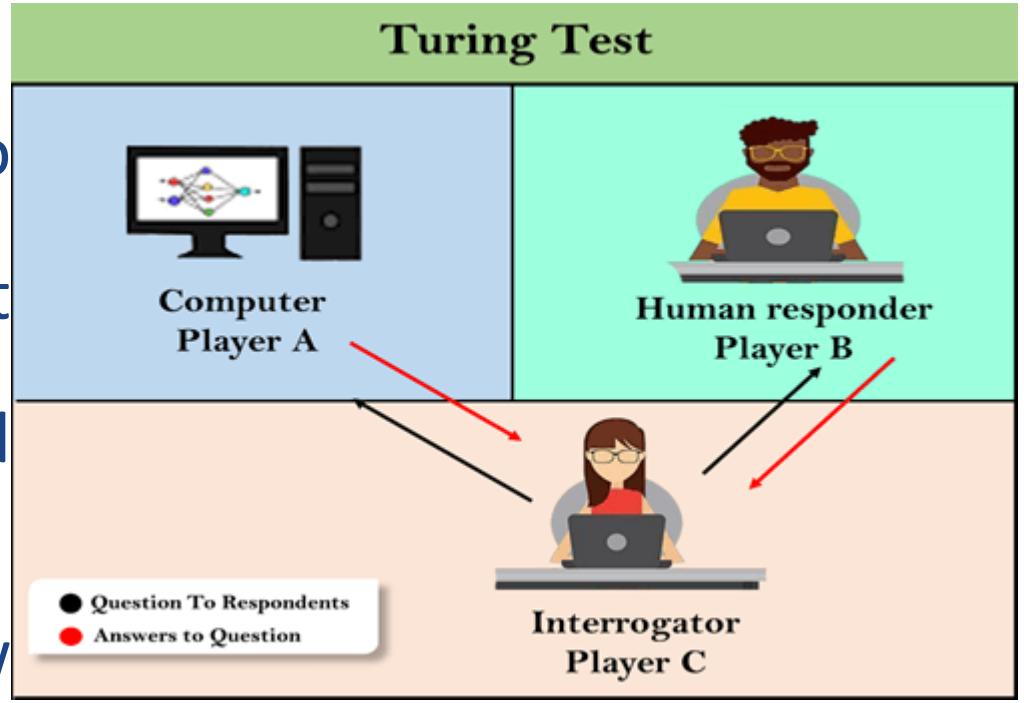


What is Artificial Intelligence? (contd..)

- Acting humanly: The Turing Test approach

The computer would need to possess the following capabilities:

- **Natural Language Processing** to enable it to communicate successfully in English;
- **Knowledge Representation** to store what it knows or hears;
- **Automated Reasoning** to use the stored information to answer questions and to draw new conclusions;
- **Machine Learning** to adapt to new circumstances and to detect and extrapolate patterns



What is Artificial Intelligence? (contd..)

- Acting humanly: The Turing Test approach

- Turing viewed the physical simulation of a person as **unnecessary** to demonstrate intelligence.
- Other researchers have proposed a **total Turing test**, which requires **interaction** with objects and people in the real world.
- To pass the Total Turing test, a **robot** will need:
  - Computer vision to perceive objects,
  - Robotics to manipulate objects and move about.

What is Artificial Intelligence? (contd..)

- **Thinking humanly: The cognitive modeling approach**
  - Program thinks like a human
  - Determining how humans think.
  - Get inside the actual workings of human minds.
  - There are three ways to do this:
    - Introspection—trying to catch our own thoughts as they go by;(Through meditation, self monitoring)
    - Psychological experiments—observing a person in action; and
    - Brain imaging—observing the brain in action.

What is Artificial Intelligence? (contd..)

- Thinking humanly: The cognitive modeling approach
  - **Cognitive science** is the study of the human mind and brain, focusing on how the mind represents and manipulates knowledge and how mental representations and processes are realized in the brain.

  - The **interdisciplinary field** of cognitive science brings together **computer models from AI and experimental techniques from psychology** to construct precise and testable theories of the human mind.

What is Artificial Intelligence? (contd..)

- Thinking rationally: The “laws of thought” approach

  - Rational thinking is a **process**.
  - It refers to the ability **to think with reason**.
  - It encompasses the ability to draw sensible **conclusions from facts, logic and data**.
  - If your thoughts are based on **facts and not emotions**, it is called rational thinking.

    Rational thinking focuses on resolving problems and achieving goals.

What is Artificial Intelligence? (contd..)

- Thinking rationally: The "laws of thought" approach
- **Syllogisms** provided patterns for argument structures that always yielded correct conclusions when given correct premises. A syllogism is a type of logical reasoning where the conclusion is yielded from two linked premises.

- Example, "Socrates is a man; all men are mortal; therefore, Socrates is mortal."

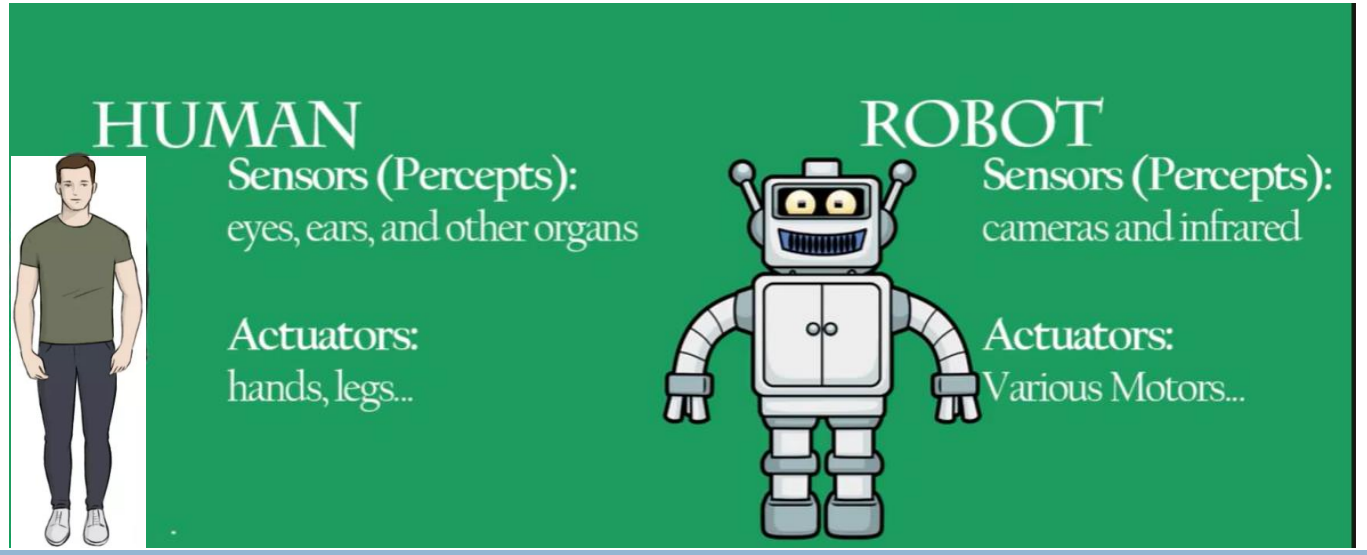
What is Artificial Intelligence? (contd..)

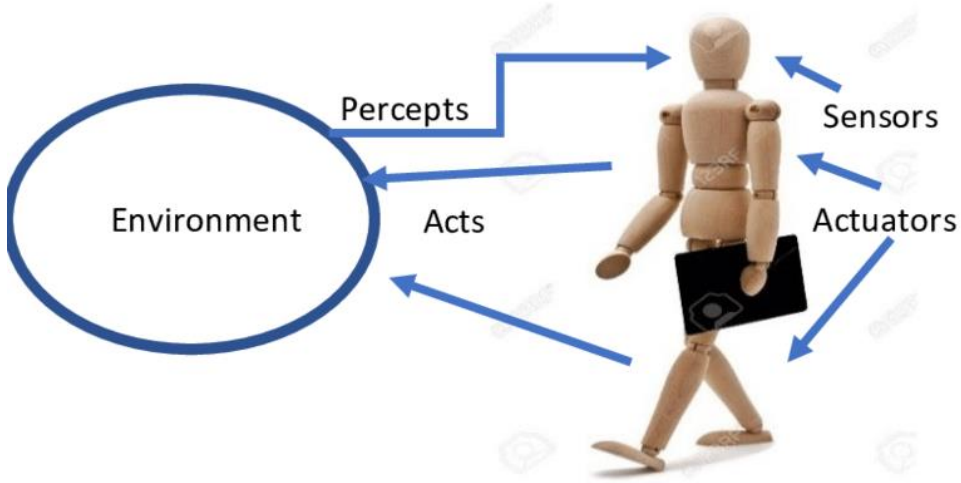
Thinking rationally: The "laws of thought" approach

- The theory of **probability** fills the gap, allowing rigorous reasoning with uncertain information.
- In principle, it allows the construction of a comprehensive model of rational thought, leading from raw perceptual information to an understanding of how the world works to **predictions about the future**.

What is Artificial Intelligence? (contd..)

- Acting rationally: The rational agent approach
- An agent is just something that **acts** (Latin agree- to do).
- An agent can be viewed as perceiving its environment through sensors and acting upon that environment through actuators





What is Artificial Intelligence? (contd..)

- Acting rationally: The rational agent approach

- All computer programs do something, but **computer agents** are expected to do more: operate autonomously, perceive their environment, persist over a prolonged time period, adapt to change, and create and pursue goals.
  - A rational agent is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome.

What is Artificial Intelligence? (contd..)

- Acting rationally: The rational agent approach
  - All the skills needed for the **Turing test also** allow an agent to act rationally.
  - Knowledge representation and **reasoning enable** agents to reach good decisions.
  - Making correct **inferences** is sometimes **part** of being a rational agent, because one way to act rationally is to deduce that a given action is best and then to act on that conclusion.

What is Artificial Intelligence? (contd..)

- Acting rationally: The rational agent approach
  - *In Conclusion:* ***"AI has focused on the study and construction of agents that do the right thing. "***
  - What counts as the right thing is defined by the **objective** that we provide to the agent.
  - This general paradigm we call it the **standard model.**

What is Artificial Intelligence? (contd..)

- Acting rationally: The rational agent approach
- **Need one important refinement to standard model:**
  - **Perfect rationality**—always taking the exactly optimal action—is not feasible in complex environments. The computational demands are just too high.
  - **Limited rationality**—acting appropriately when there is not enough time to do all the computations.

RAMAIAH
Institute of Technology

What is Artificial Intelligence? (contd..)

Thinking Humanly	**Thinking Rationally**
"The exciting new effort to make computers think . . . *machines with minds*, in the full and literal sense." (Haugeland, 1985) "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . ." (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly	**Acting Rationally**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things at which, at the moment, people are better." (Rich and Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole *et al.*, 1998) "AI . . . is concerned with intelligent behavior in artifacts." (Nilsson, 1998)

Some definitions of artificial intelligence, organized into four categories.

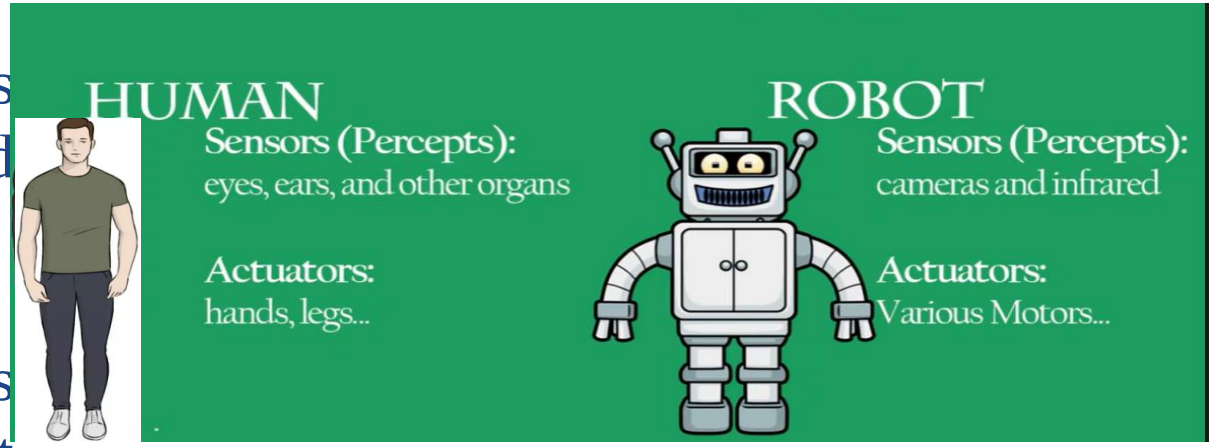
Eight definitions of AI, laid out along two dimensions.

Top - thought processes and reasoning,
Bottom - address behavior.

Left measure success - Human performance
Right ideal performance measure, called rationality

Agents and Environments

- Rational Agents are considered as **approach to artificial intelligence**
- An **agent** is anything that can be viewed as perceiving its **environment** through **sensors** and **acting** upon that environment through **actuators.**
- **Example: Human, Robot**
- **Software Agents :** A software agent receives file contents, network packets, and human input (keyboard/mouse/touchscreen/voice) as sensory inputs and acts on the environment by writing files, sending network packets, and displaying information or generating sounds.



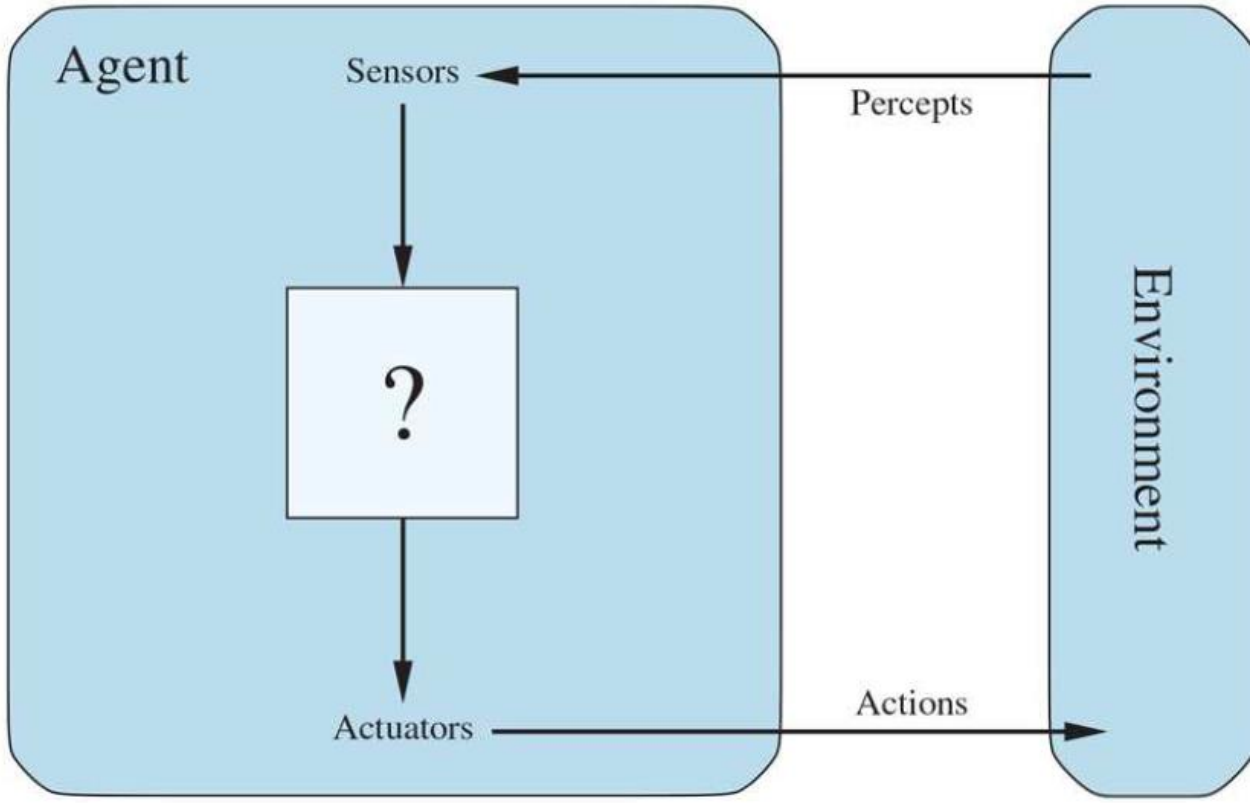
Agents and Environments

- The **environment** could be everything—the entire universe!
- In practice it is just that part of the universe whose state we care about when designing this agent
- The part that affects what the agent perceives and that is affected by the agent's actions.

Agents and Environments

- Perceive the environment through **sensors** (→Percepts)
- Act upon the environment through **actuators** (→Actions)

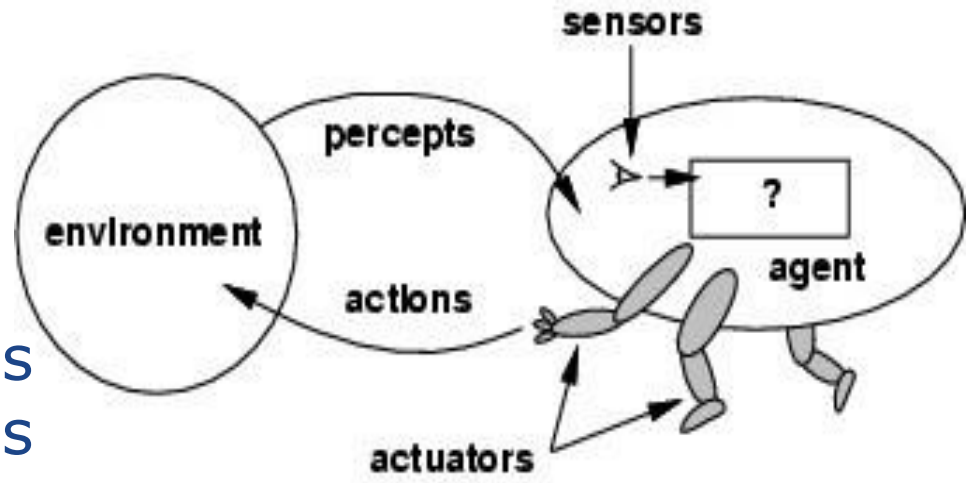
Figure 2.1



Agents interact with environments through sensors and actuators.

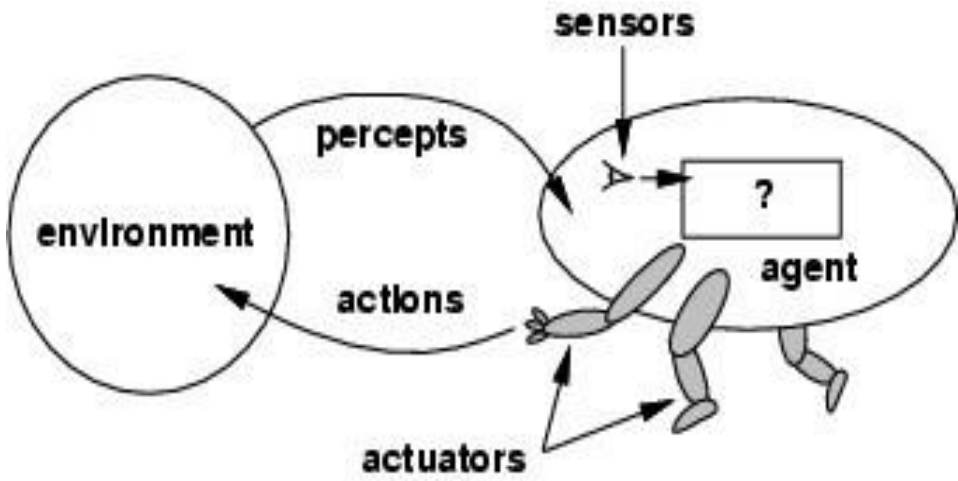
Agents

- Percept: The agent's perceptual inputs at any given instant
- Percept sequence: The complete history of everything the agent has perceived
- An **agent's choice of action depends on**
- Built-in knowledge
- Entire percept sequence observed to date
- Mathematically, an agent's behavior is described by the **agent function** that maps any given percept sequence to an action.



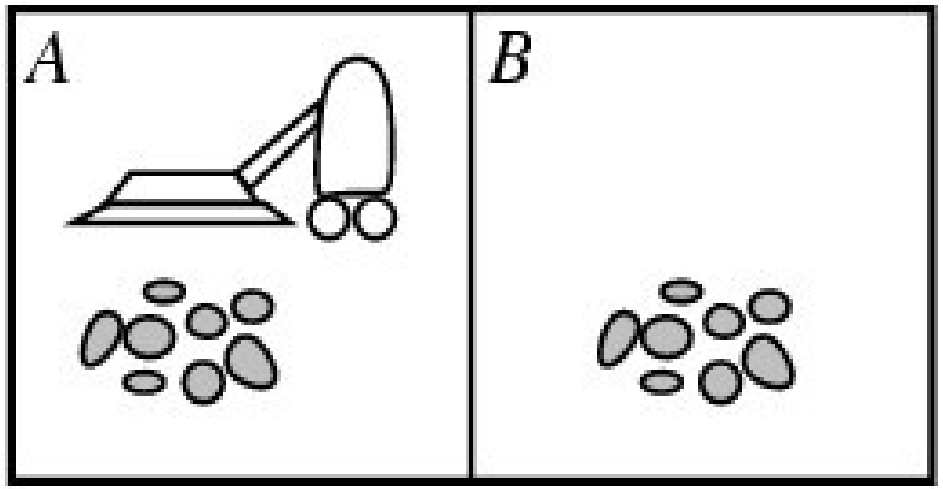
Agents

- **Tabulating** the agent function describes any given agent.
- Internally, the agent function for an artificial agent will be implemented by an **agent program**.
- The **agent function** is an **abstract mathematical** description; the agent program is a **concrete implementation**, running within some physical system.



Agents -> Example : Vacuum-Cleaner World

- Agent : Vaccum Cleaner
- Environment:Area of squares either clean or dirty
- Percepts: Perceive Location(Square), Clean or Dirty e.g., [A, dirty]
- Actions: Move Left, Right, Suck, NoOp



Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Partial tabulation of a simple agent function for the vacuum-cleaner world

Rationality- Rational Agent

What is rational at any given time depends on four things:

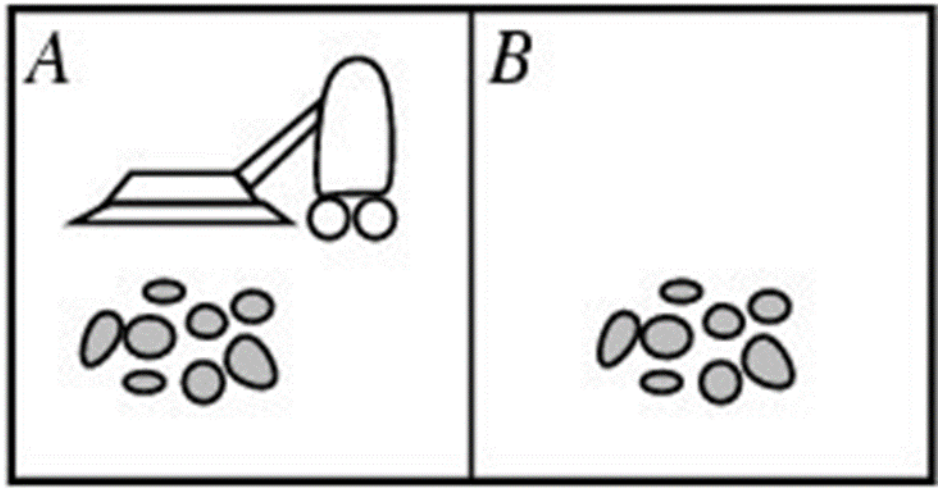
- The performance measure that defines the criterion of success.
- The agent's prior knowledge of the environment.
- The actions that the agent can perform.
- The agent's percept sequence to date.
- **Definition of Rational Agent:**
  - For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rationality

- Performance measure: Use notion called **consequentialism**:
  - Evaluate an agent's behavior by its **consequences**.
  - An agent is in an environment, it g**enerates a sequence** of actions according to the **percepts it receives**.
  - This sequence of actions causes the environment to go through a **sequence of states.**
  - If the sequence is **desirable**, then the agent has **performed well**.
  - This notion of desirability is captured by a **performance measure** that **evaluates** any given sequence of **environment states.**

Rationality –Vacuum Cleaner Example

- A simple agent that cleans a square if it is dirty and moves to the other square if not.
- Is it rational?
- Assumption:
  - **Performance measure**: Awards one point for each clean square at each time step, over a "lifetime" of 1000 time steps.
  - The "**geography**" of the environment is known a priori but the dirt distribution and the initial location of the agent are not.
  - **Actions** = {left, right, suck, no-op}
  - Agent is able to **perceive** the location and dirt in that location



The Nature of Environments

- Specifying the task environment is always the first step in designing agent
- PEAS: **P**erformance, **E**nvironment, **A**ctuators, **S**ensors

P	**Performance** – how we measure the system's achievements
E	**Environment** – what the agent is interacting with
A	**Actuators** – what produces the outputs of the system
S	**Sensors** – what provides the inputs to the system

In **designing** an agent, the first step must always be to specify the task environment as fully as possible.

Taxi Driver Example

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

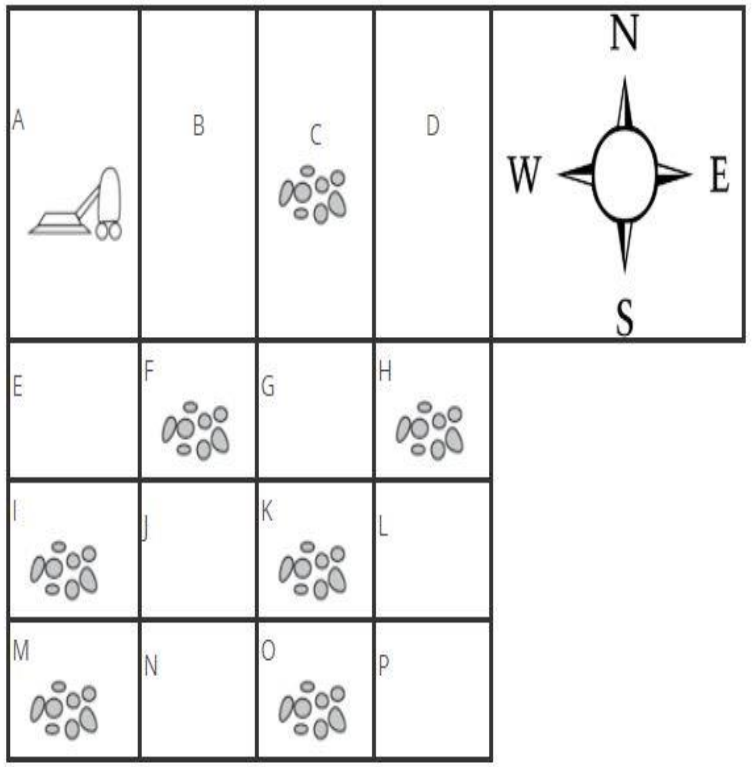
Mushroom-Picking Robot

Performance Measure	Environment	Actuators	Sensors
Percentage of good mushrooms in correct bins	Conveyor belt with mushrooms, bins	Jointed arm and hand	camera, joint angle sensors

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Properties of Task Environments

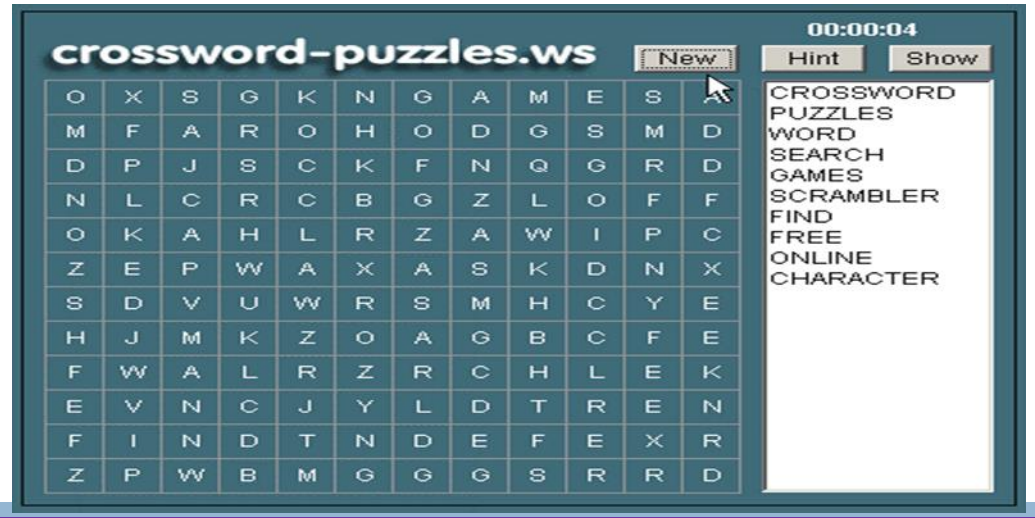
- **Partially Observable :** Environment partially observable because of noisy, missing and inaccurate sensors.
- Example, Vacuum agent , Automated Taxi
- If the agent has no sensors at all then the environment is **unobservable**.



Single agent v/s multi-agent

Single-agent

- If only one agent is involved in an environment, and operating by itself then such an environment is called single agent environment.

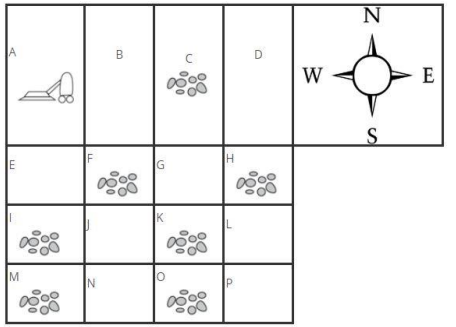


Example: maze

Single agent v/s multi-agent

Chess: The opponent entity is trying to maximize its performance measure, which, by the rules of chess, minimizes agent's performance measure. Thus, chess is a **competitive multiagent environment.**

Taxi-driving environment: Avoiding collisions maximizes the performance measure of all agents, so it is a partially **cooperative multiagent environment**



Example: football

Deterministic agent v/s Nondeterministic agent

- Deterministic : If the next state of the environment is completely determined by the current state and the action executed by the agent(s), then the environment is deterministic;
- There is no uncertainty in a **fully observable**, deterministic environment
- **Example**: Vacuum Cleaning Agent
- Nondeterministic: If the environment is **partially observable**, then it could appear to be nondeterministic.
- Example: Taxi Driving
- Stochastic is used by some as a synonym for “nondeterministic,”
- A model of the environment is stochastic if it explicitly deals with probabilities (e.g., “there’s a 25% chance of rain tomorrow”) .
- (e.g., “there’s a chance of rain tomorrow”).

RAMAIAH
Institute of Technology

Episodic v/s sequential

Episodic

- Only the current percept is required for the action
- Every episode is independent of each other
- **Example**: part picking robot

Sequential

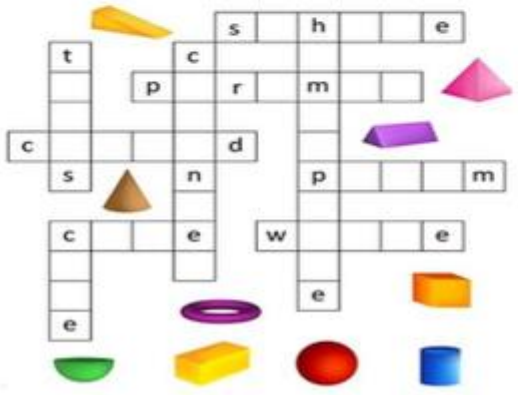
- An agent requires memory of past actions to determine the next best actions.
- The current decision could affect all future decisions.
- **Example**: Chess

RAMAIAH
Institute of Technology

Static v/s dynamic

Static

- The environment does not change while an agent is acting
- **Example**: Crossword puzzles



Dynamic

- The environment may change over time
- **Example**: roller coaster ride

  Taxi driving

RAMAIAH
Institute of Technology

Discrete v/s continuous

Discrete

- The environment consists of a finite number of actions that can be deliberated in the environment to obtain the output.
- **Example**: chess

Continuous

- The environment in which the actions performed cannot be numbered ie.,is not discrete, is said to be continuous.
- **Example**: Self-driving cars

RAMAIAH
Institute of Technology

Known v/s Unknown

This distinction refers not to the environment itself but to the **agent's (or designer's) state of knowledge** about the environment.

In a known environment, the outcomes for all actions are known.

In a unknown environment the agent will have to learn how it works in order to make good decisions.

RAMAIAH
Institute of Technology

Known v/s Unknown

Known environment can be partially observable—for example, in solitaire card games.

Unknown

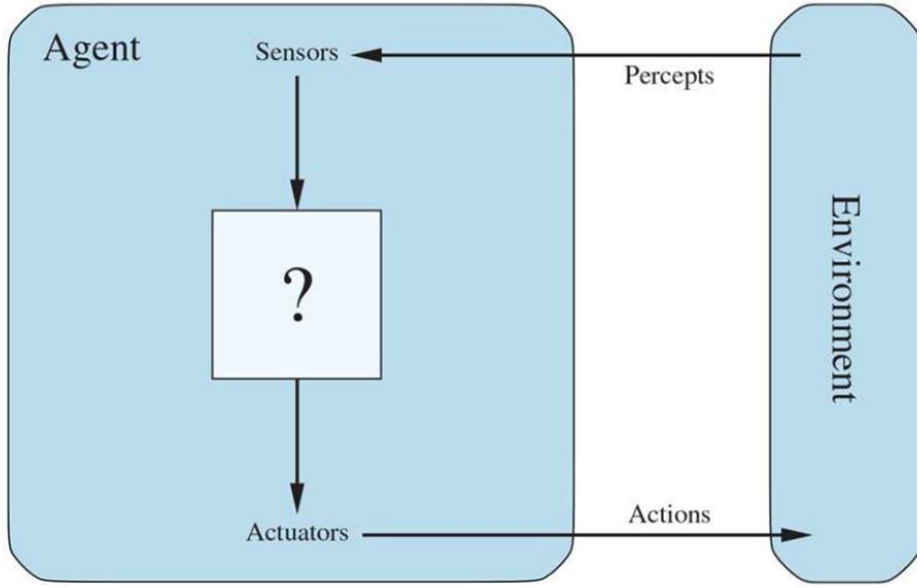
- In unknown environment, agent needs to learn how it works in order to perform an action.
- **Example**: A new video game

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

The Structure of Agents

- Job of AI is to design an **agent program** that implements the **agent function**—the mapping from percepts to actions.
- **Assumption**: This program runs on some sort of computing device with physical sensors and actuators— called as the **agent architecture.**
- An intelligent agent is a combination of Agent Program and Architecture.
  - Intelligent Agent = Agent Program + Architecture

Figure 2.1

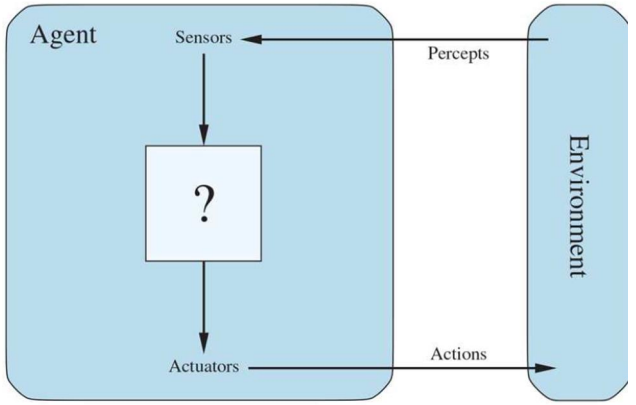


Agents interact with environments through sensors and actuators.

The Structure of Agents

- Program chosen has to be appropriate for the architecture.
- *The architecture makes the precepts from the sensors available to the program, runs the program, and feeds the program's action choices to the actuators as they are generated.*
- Architecture is a computing device used to run the agent program.

Figure 2.1



Agents interact with environments through sensors and actuators.

RAMAIAH
Institute of Technology

Agent Programs

Agent Function: Table-Driven Agent

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Agent Programs

Table-Driven Agent

Agent program that keeps track of the percept sequence and then uses it to index into a table of actions to decide what to do.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Agent Programs

Table-Driven Agent

- Designer needs to construct a table that contains the appropriate action for every possible percept sequence

- Drawbacks?
  - Huge table
  - Take a long time to construct such a table
  - Even with learning, need a long time to learn the table entries

Agent programs

The **key challenge for AI** is to find out how to write programs that, produce rational behavior from a small program rather than from a vast table.

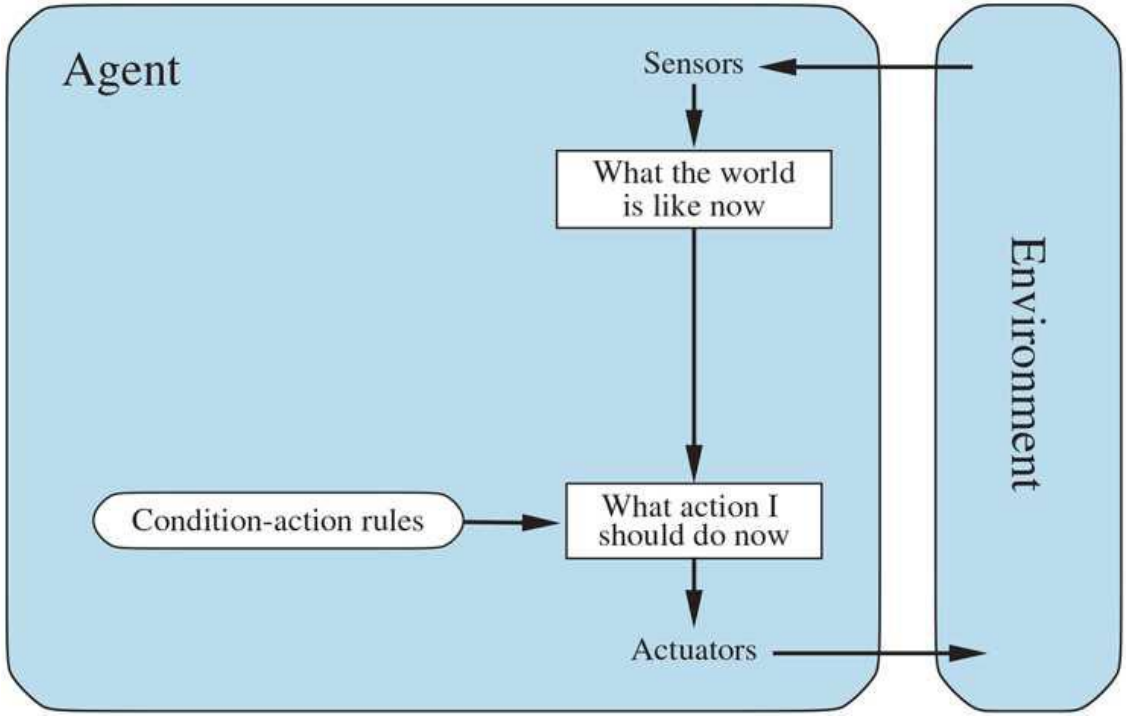
- Arranged in order of increasing generality:
  - Simple reflex agents
  - Model-based reflex agents
  - Goal-based agents
  - Utility-based agents; and
  - Learning agents

Simple Reflex Agent

- The schematic form, showing how the condition–action rules allow the agent to make the connection from percept to action

Schematic diagram of a simple reflex agent.

- Rectangles denote the current internal state of the agent's decision process,
- Ovals to represent the background information used in the process.



Simple Reflex Agent

- A more general and flexible approach is first to build a general-purpose interpreter for condition–action rules and then to create rule sets for specific task environments.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
    persistent: rules, a set of condition–action rules

    state ← INTERPRET-INPUT(percept)
    rule ← RULE-MATCH(state, rules)
    action ← rule.ACTION
    return action
```

INTERRUPT-INPUT – function generates description of the current state from the percept.
RULE-MATCH – function returns the first rule in the set of rules that matches the given state description.
RULE-ACTION – the selected rule is executed as action of the given percept.

Simple Reflex Agent

1. **Thermostat (Heating System):**

- **Percept**: The current room temperature, sensed by the thermostat.
- **Condition-Action Rule:**
  - If the temperature is below the set threshold (e.g., 18°C), then turn on the heater.
  - If the temperature is above the set threshold (e.g., 22°C), then turn off the heater.
- **Action**: Turning the heater on or off based on the current temperature.

2. **Automatic Door:**

- **Percept**: Detecting the presence of a person using motion sensors.
- **Condition-Action Rule**:
  - If a person is detected near the door, open the door.
  - If no person is detected, keep the door closed.
- **Action**: Opening or closing the door based on the sensor's input.

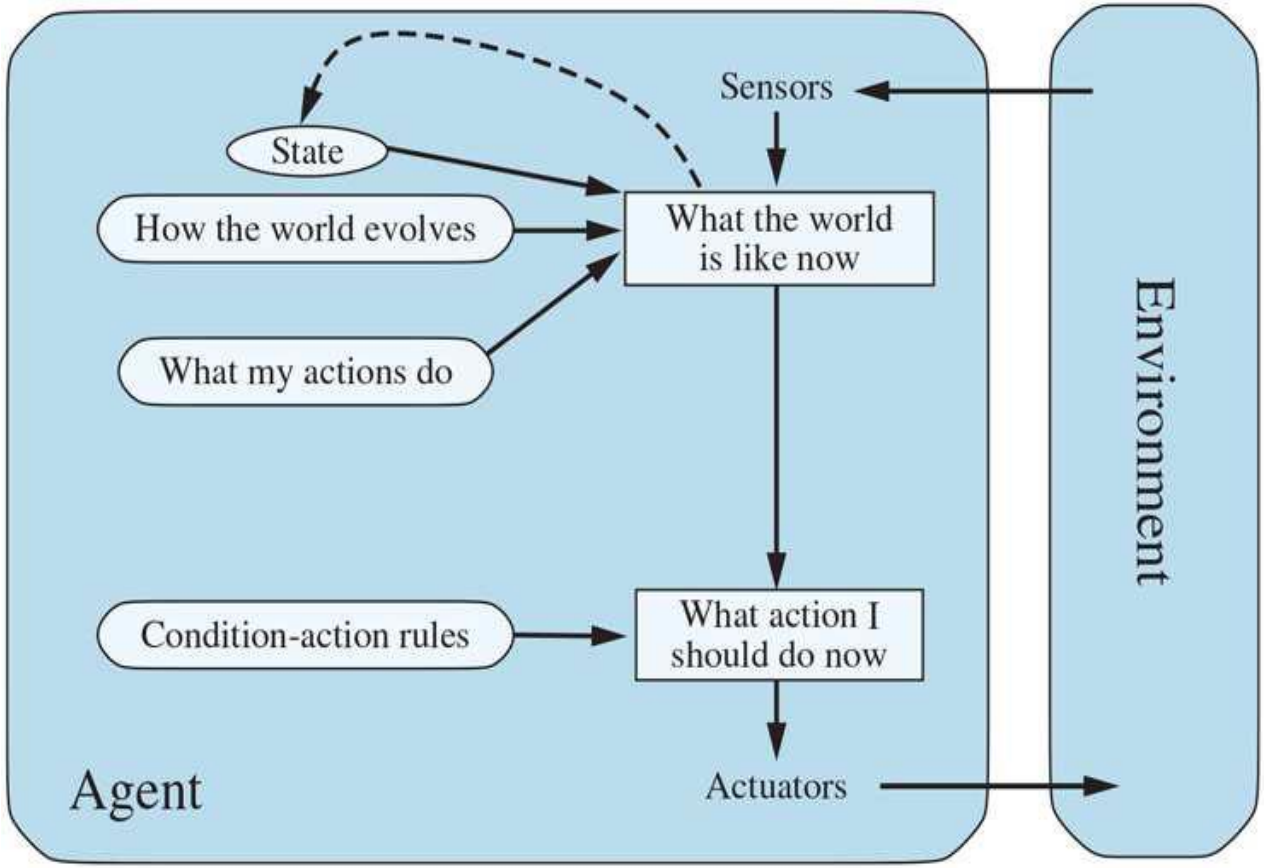
Simple Reflex Agent

- Simple reflex agents are **simple** and of **limited intelligence**.
- Will work only if environment is **fully observable.**
- Even a little bit of unobservability can cause serious trouble.
- **For example**, the braking rule assumes that the condition car-in-front-is-breaking can be determined from the
  - Current percept—a single frame of video.
  - Car in front has a centrally mounted brake light.
  - **Older models** have different configurations of taillights, brake lights, and turn-signal lights.
- A simple reflex agent driving behind such a car would either brake continuously and unnecessarily, or, worse, never brake at all.

RAMAIAH
Institute of Technology

Model-based reflex agents

- **Model-based reflex agents is** and intelligent agent that uses percept history and internal memory to make decisions about the model of the world around it.
- This reflects at least some of the **unobserved aspects** of the current state.
- Agent can handle **partial observability** to keep track of the part of the world it can't see now through Model and Internal State.



RAMAIAH
Institute of Technology

Model-based reflex agents

Updating this internal state information requires two kinds of **knowledge**

1. How the world changes/evolves over time,
   i. The effects of the agent's actions
   ii. How the world evolves independently of the agent.

Ex: Agent turns the steering wheel clockwise, the car turns to the right. When it's raining the car's cameras can get wet. This knowledge about "how the world works"—is called a **transition model** of the world.

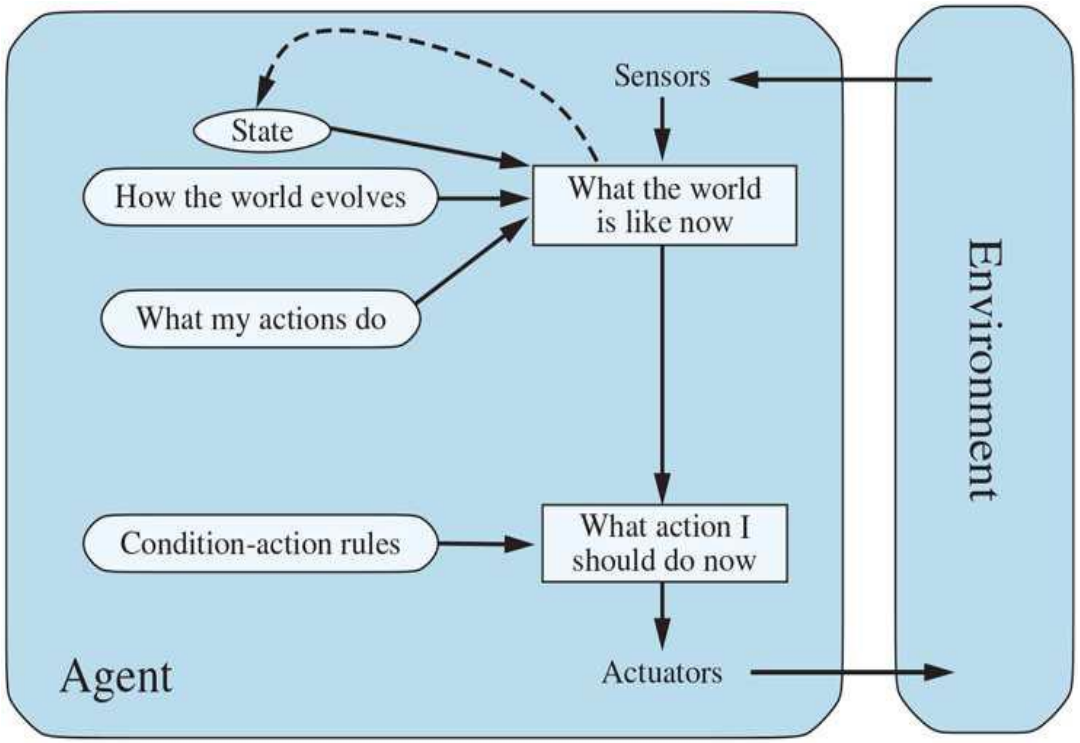
Model-based reflex agents

2. How the state of the world is reflected in the agent's percepts.

Ex: when the car in front initiates braking, one or more illuminated red regions appear in the forward-facing camera image, and, when the camera gets wet, droplet-shaped objects appear in the image partially obscuring the road.

This kind of knowledge is called a **sensor model.**

The **transition model and sensor model** allow an agent to keep track of the state of the world. An agent that uses such models is called a **model-based agent.**



Pseudo-Code

function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
  persistent: *state*, the agent's current conception of the world state
    transition_model, a description of how the next state depends on
      the current state and action
    sensor_model, a description of how the current world state is reflected
      in the agent's percepts
    rules, a set of condition–action rules
    action, the most recent action, initially none

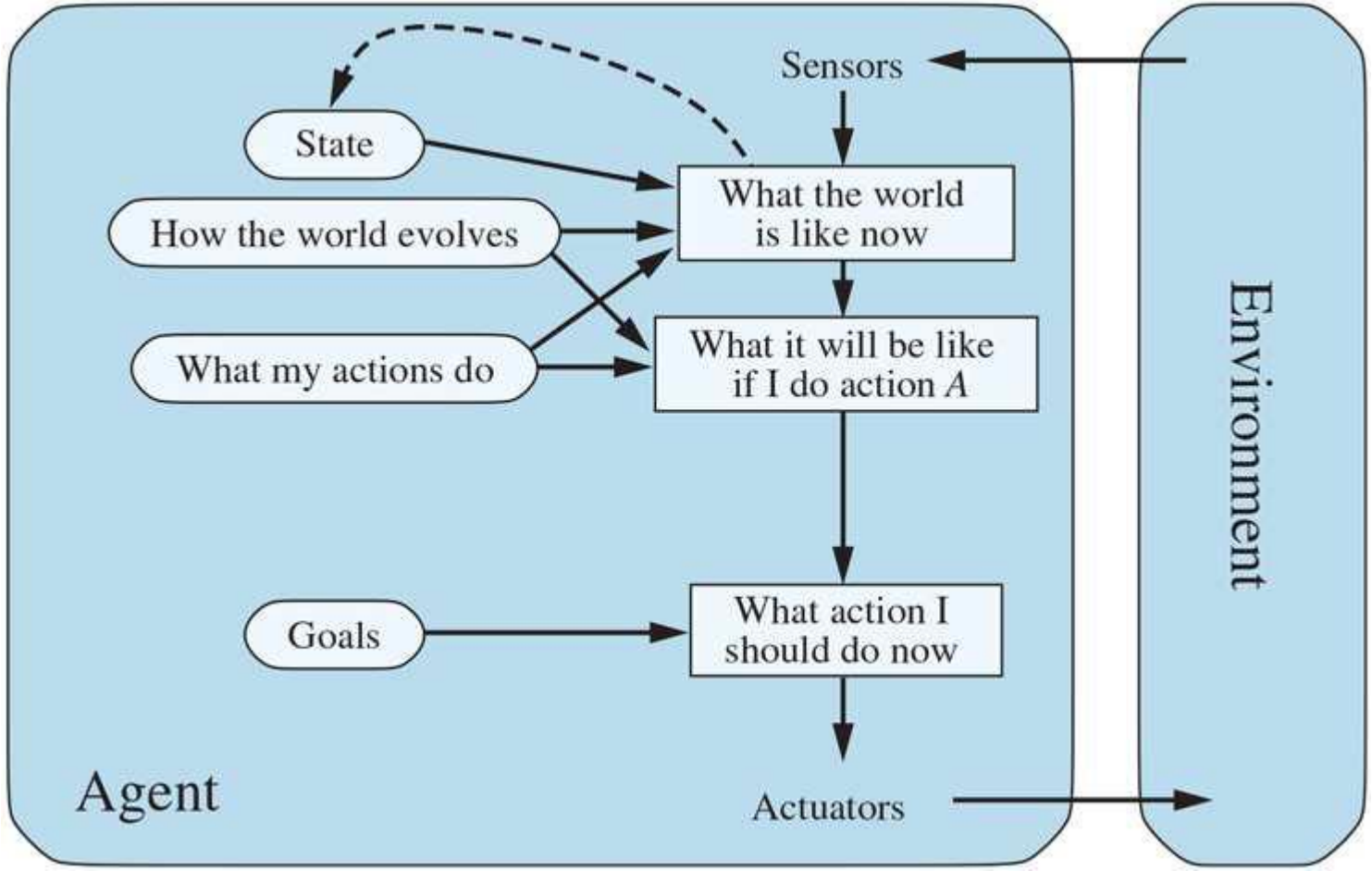
  state ← UPDATE-STATE(*state*, *action*, *percept*, *transition_model*, *sensor_model*)
  rule ← RULE-MATCH(*state*, *rules*)
  action ← *rule*.ACTION
  return *action*

Goal-based agents

- Knowing something about the **current state** of the environment is not always enough to decide what to do in a situation
- The agent needs some sort of **goal information** that describes situations that are desirable.
- The agent program can **combine this with the model** to choose actions that achieve the goal.

Goal-based agents

A model-based, goal-based agent. It keeps track of the world state as well as a **set of goals it is trying to achieve**, and chooses an action that will (eventually) lead to the achievement of its goals.



Goal-based agents

Autonomous Delivery Robot

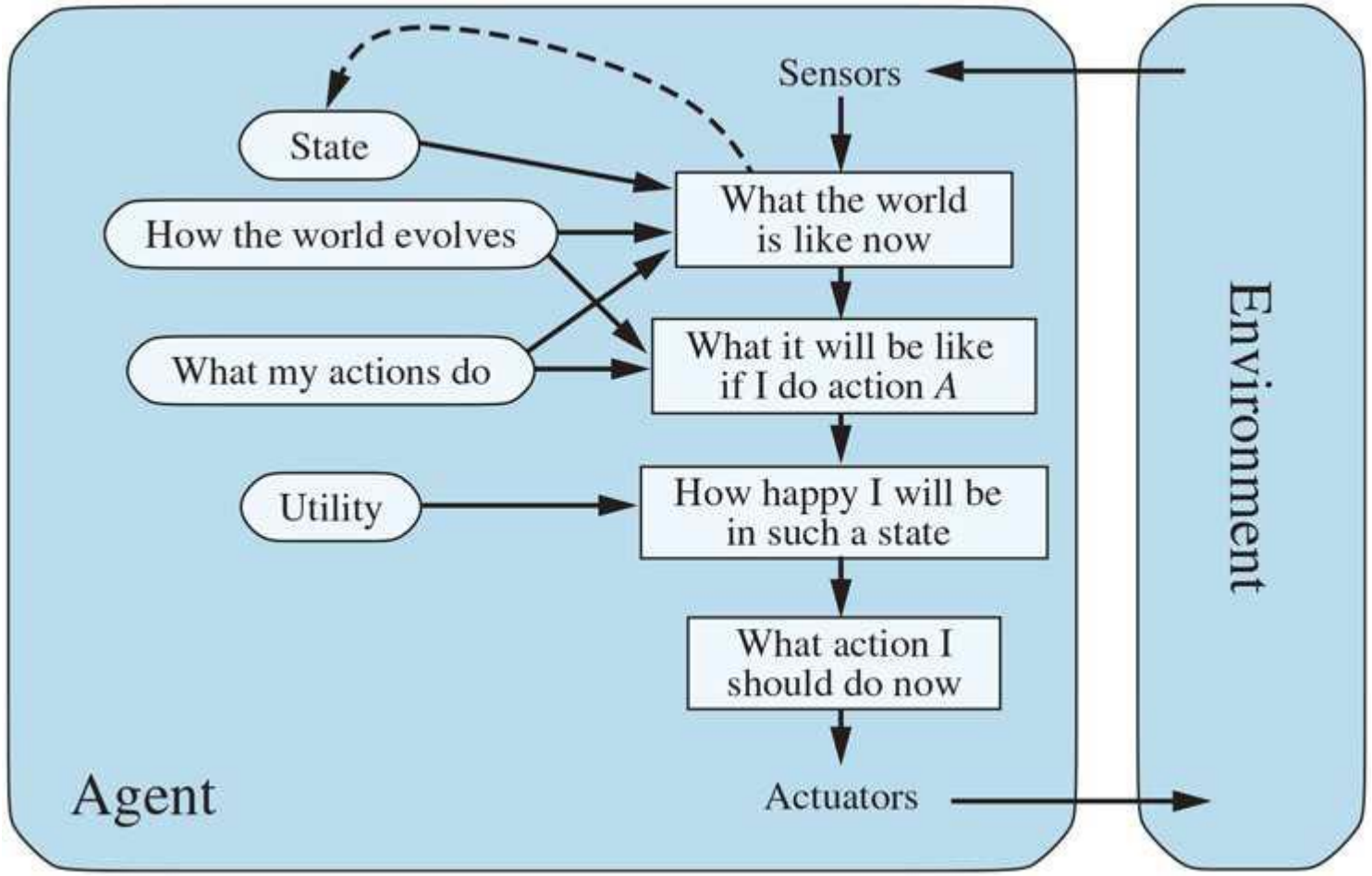
- **Percept**: The robot perceives the environment using its sensors, detecting obstacles, the layout of the surroundings, and the target delivery location.
- **Goal**: Deliver the package to the specified destination in the shortest time possible, without colliding with obstacles.
- **State**: The robot keeps track of its current location, its orientation, the map of the area, and the position of obstacles.
- **Action Selection**: The robot evaluates different potential paths, choosing the one that best moves it toward the goal (delivery point) while avoiding obstacles.
- **Model of the Environment**: The robot can have a model of how it moves through the environment and updates this model as it navigates. If an obstacle appears, the robot will re-plan its path based on its goal.
- **Outcome**: The robot successfully delivers the package by considering its goal.

Utility-based agents

- Goals alone are not enough to generate **high-quality behavior** in most environments.
- Utility based agents is an agent that acts for the **best way to reach that goal**.
- **Ex: Taxi** to its destination (but some are quicker, safer, more reliable, or cheaper than others.)
- Goals just provide a crude binary distinction between “happy” and “unhappy” states.
- Because “happy” does not sound very scientific, economists and computer scientists use the term **utility** instead. Word “utility” here refers to **“the quality of being useful,”**

Utility-based agents

- A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world.
- Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.



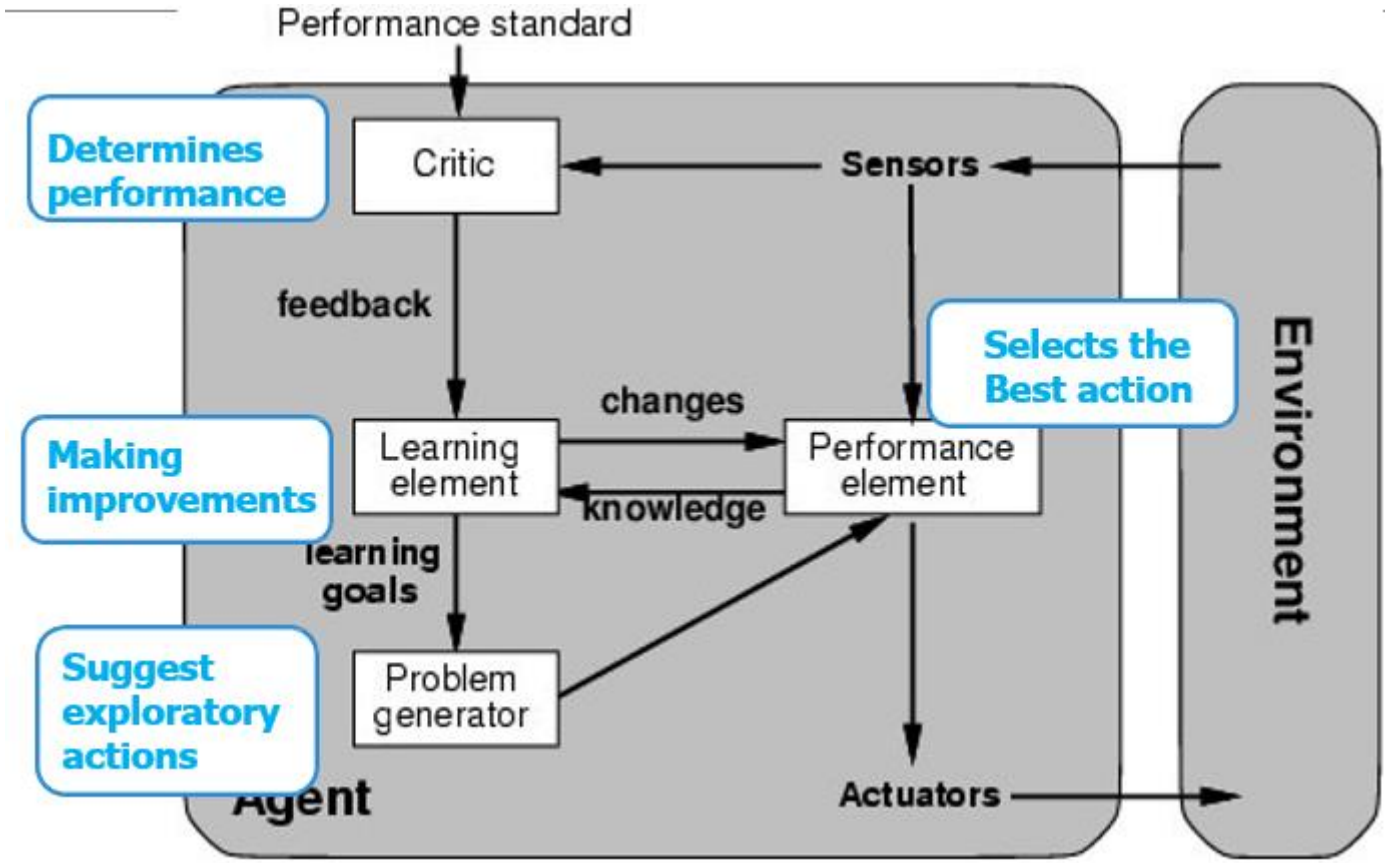
Utility-based agents

- **Performance measure** assigns a score to any given sequence of environment states.
- Provided that the **internal utility function** and the **external performance measure** are in agreement, an agent that **chooses actions to maximize** its utility will be rational according to the external performance measure.

- # Stock Trading Agent Example
- **Percept**: The agent monitors market data, including stock prices, economic indicators, and news updates.
- **Goal**: Maximize profit from stock trading.
- **Utility Function**: The utility function could factor in expected returns, risk tolerance, and time horizon for investments.
- **Action Selection**: The agent selects trades that maximize the expected utility, taking into account both the likelihood of making a profit and the potential risks involved.
- **Model of the Environment**: The agent predicts how stock prices will evolve based on current trends and news, incorporating this prediction into its decision-making process.
- **Outcome**: The agent executes trades that maximize profit while balancing risk, based on its utility function.

RAMAIAH
Institute of Technology

Learning Agent



A learning agent is a type of AI system that learns from its interactions with the environment and improves its performance over time

RAMAIAH
Institute of Technology

Learning Agent Process Flow:

- The **sensors** collect information from the environment and feed it to the performance element.
- The **performance element** selects an action based on the knowledge the agent has acquired so far. It focuses on solving the problem using the current information. Uses the actuators to interact with the environment.
- The **critic** evaluates how well the action worked by comparing the results to a performance standard. Provides feedback to the learning element to make adjustments.

RAMAIAH
Institute of Technology

Learning Agent Process Flow:

- The **learning element** uses this feedback to update the agent’s knowledge base, enabling better decision-making in future actions. It adapts the agent's behavior to perform better over time.
- The **problem generator** introduces variability by suggesting new exploratory actions, which contributes to learning and improvement.

RAMAIAH
Institute of Technology

Learning Agent

- **Autonomous Vehicles (Self-Driving Cars) Example**
- **Performance Element**: The car uses its sensors (cameras, radar, LIDAR) to understand its environment (road, obstacles, traffic signs) and makes decisions about steering, braking, or accelerating.
- **Critic**: Evaluates how well the car is performing in terms of safety and efficiency (e.g., avoiding collisions, staying within lanes, obeying traffic rules).
- **Learning Element**: The car learns from mistakes or feedback. For instance, if the car took too sharp a turn, the system adjusts the driving behavior for smoother future turns.
- **Problem Generator**: Suggests exploratory driving scenarios, such as trying to navigate through different traffic patterns or road conditions, to improve the system's adaptability.

Problem Solving by Search

- Problem solving by search is a **fundamental approach** in AI where an agent explores a set of possible states to find a path from an initial state to a goal state.
- The agent taking a **sequence of actions** that form a path to a goal state is called a **problem-solving agent**, and the computational process it undertakes is called **search**.
- Study several **search algorithms.**

Problem Solving by Search

Some applications of problem-solving by search across various fields are:

1. **Path finding and Navigation** : GPS and robotics.
2. **Game Playing Application**: Chess, video games.
3. **Optimization Problems** : Resource allocation, scheduling, and logistics.
4. **Medical Diagnosis**: Decision support systems. Search Type: Knowledge-based search (e.g., rule-based reasoning,

Problem Solving by Search

- Consider only the **simplest environments:** episodic, single agent, fully observable, deterministic, static, discrete, and known.
- **Informed algorithms** in which the agent can estimate how far it is from the goal
- **Uninformed algorithms** where no such estimate is available.

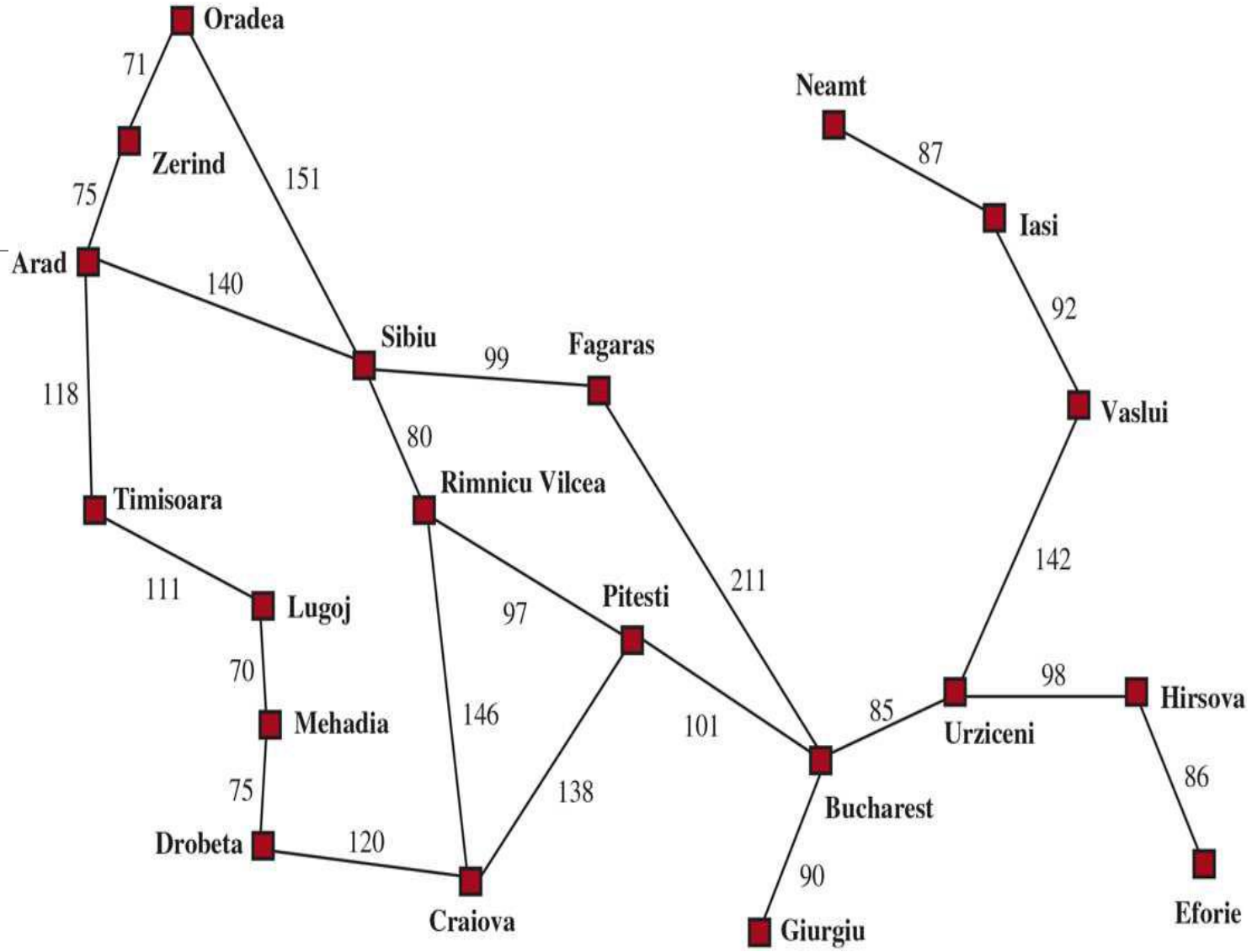
Problem-Solving Agents

- **Example**: Agent enjoying

a touring vacation in Romania.

Current Location : Arad

- If the environment is unknown—then the agent can do no better than to execute one of the actions at random.
- Assume our agents always have access to information about the world, such as the map.



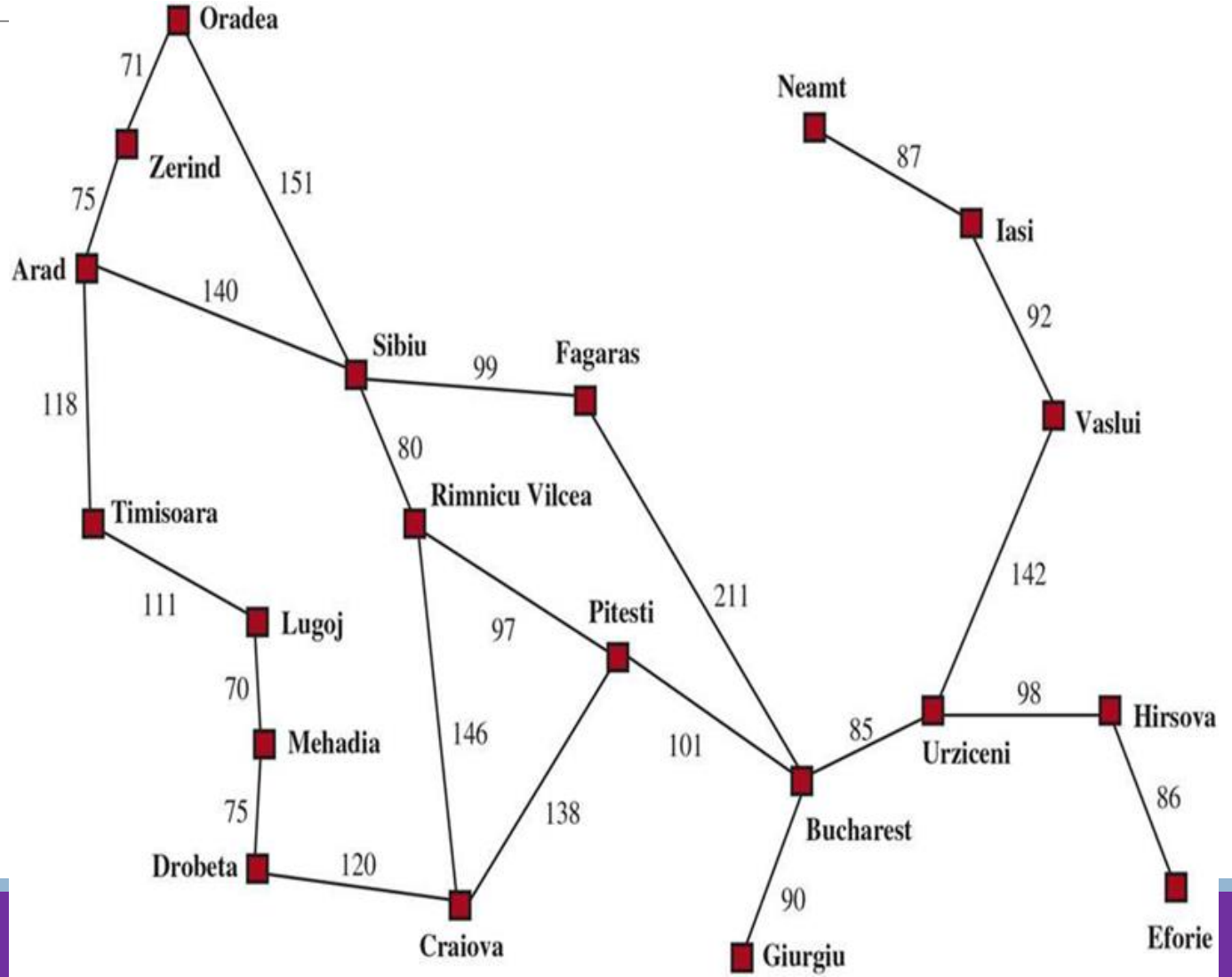
Problem-Solving Agents

The Agent can follow Four-phase problem-solving process:

1. **GOAL FORMULATION:** The agent adopts the goal of reaching Bucharest. Goals **organize behavior** by limiting the **objectives** and hence the **actions** to be considered.
2. **PROBLEM FORMULATION:** The **agent devises a description of the states and actions necessary to reach the goal**—an abstract model of the relevant part of the world.

   Ex: **Actions**: Traveling from one city to an adjacent city

   State of the world : Change in current city.

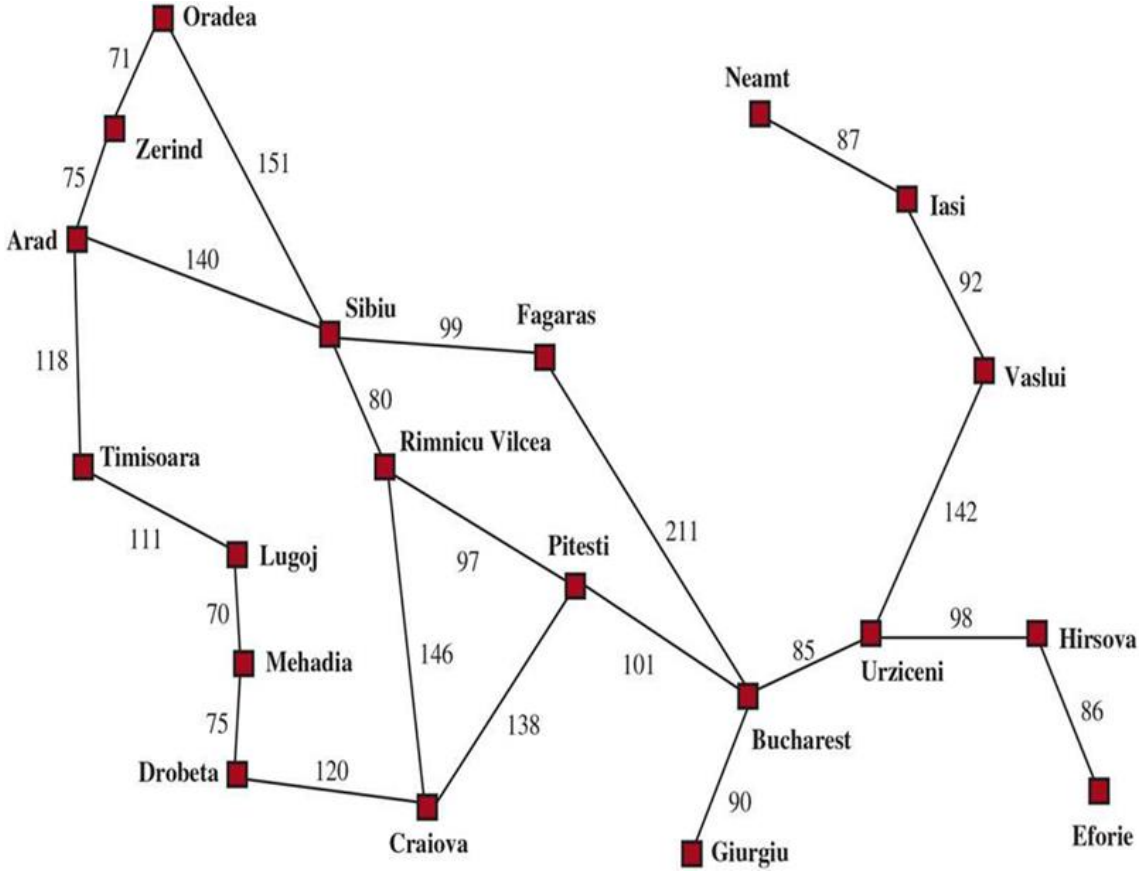


Problem-Solving Agents

3. SEARCH: Before taking any action in the real world, the agent **simulates sequences of actions in its model**, searching until it finds a sequence of actions that reaches the goal. Such a sequence is called a **solution**.

The agent might have to simulate **multiple sequences** that do not reach the goal, or it will find that no solution is possible(Ex: Arad to Sibiu to Fagaras to Bucharest)

4. **EXECUTION:** The agent can now **execute the actions** in the solution, one at a time.



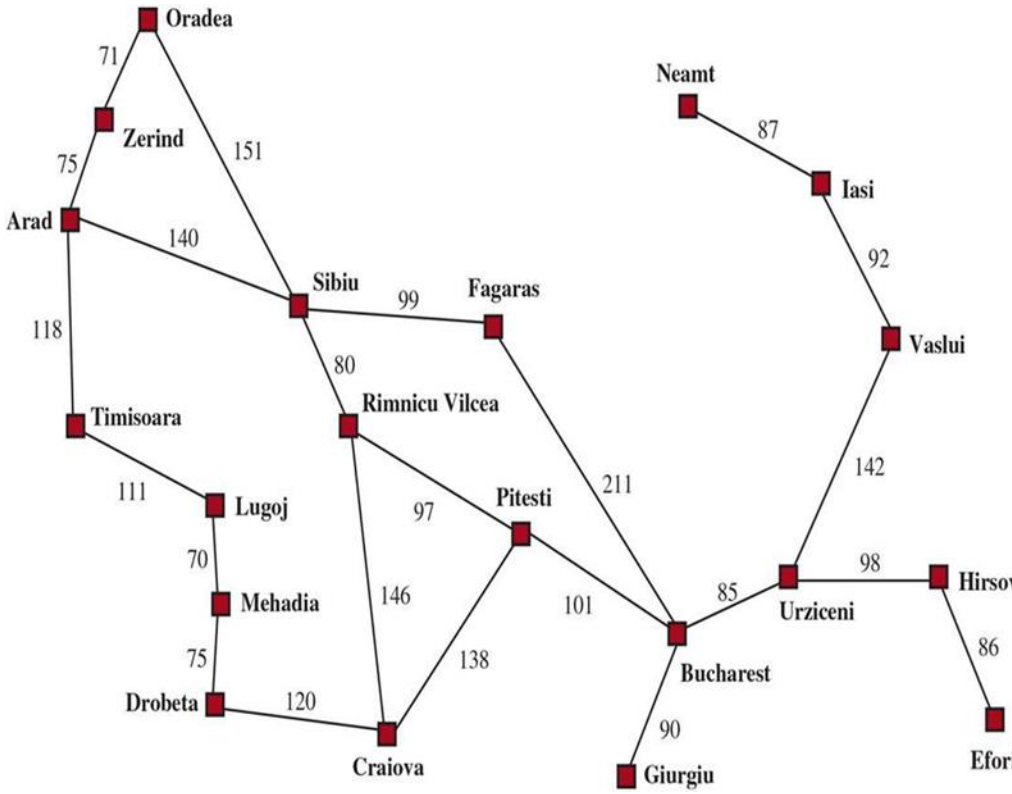
Problem-Solving Agents

Environment : Is fully observable, deterministic, known environment, the solution to any problem is a fixed sequence of actions.

Ex: Drive to Sibiu, then Fagaras, then Bucharest.

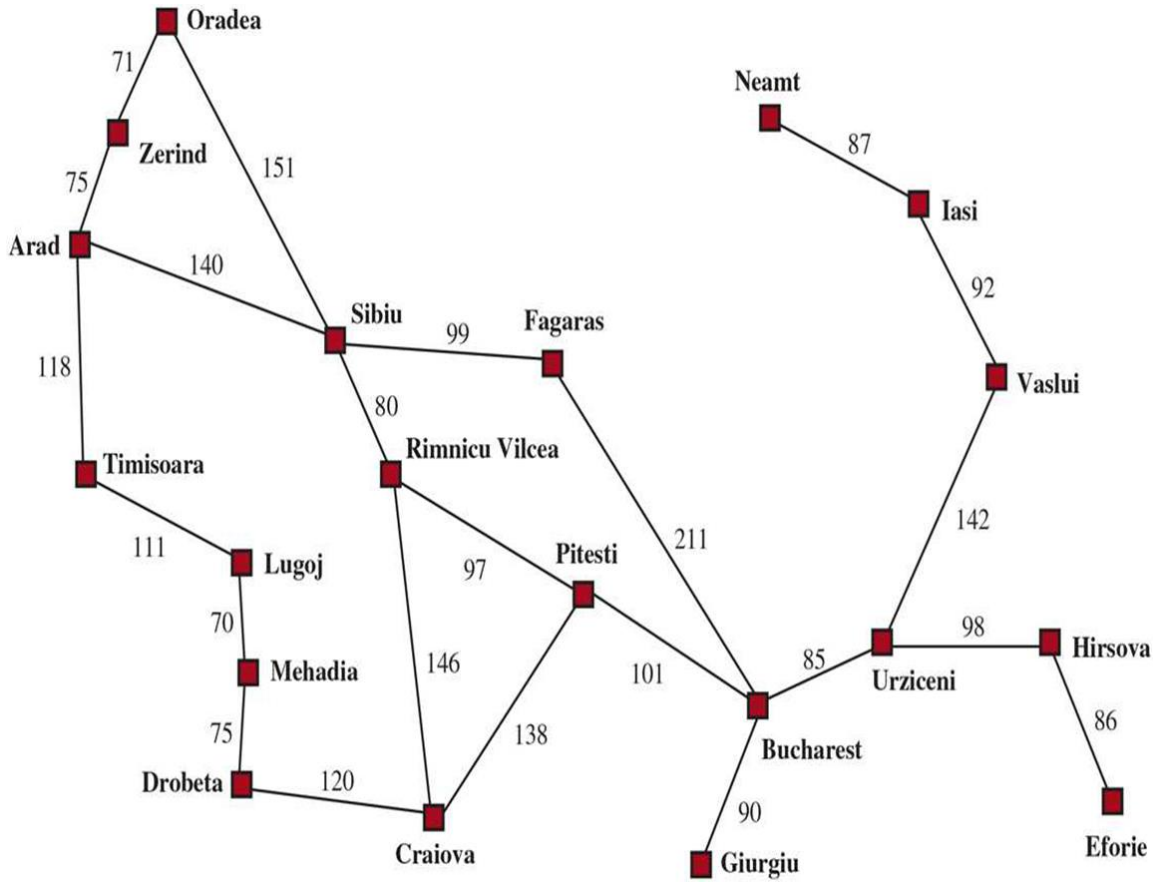
We use Op**en-loop system:** ignoring the percepts breaks the loop between agent and environment. If the environment is partially observable and nondeterministic, then the agent would be safer using a closed-loop approach that monitors the percepts

Ex: The agent might plan to drive from Arad to Sibiu but might need a contingency plan in case it arrives to a sign saying Road Closed.



Search problems and solutions

- A **search problem** can be **defined formally** as follows:
- A set of possible states that the environment can be in called as **state space**.
- The **state space** can be represented as a **graph** in which the **vertices are states** and the **directed edges between them are actions**.
- The **initial state** that the agent starts in. **Ex:** Arad.
- A set of one or more **goal states.**
  - One goal state (e.g., Bucharest),
  - Small set alternative goal states



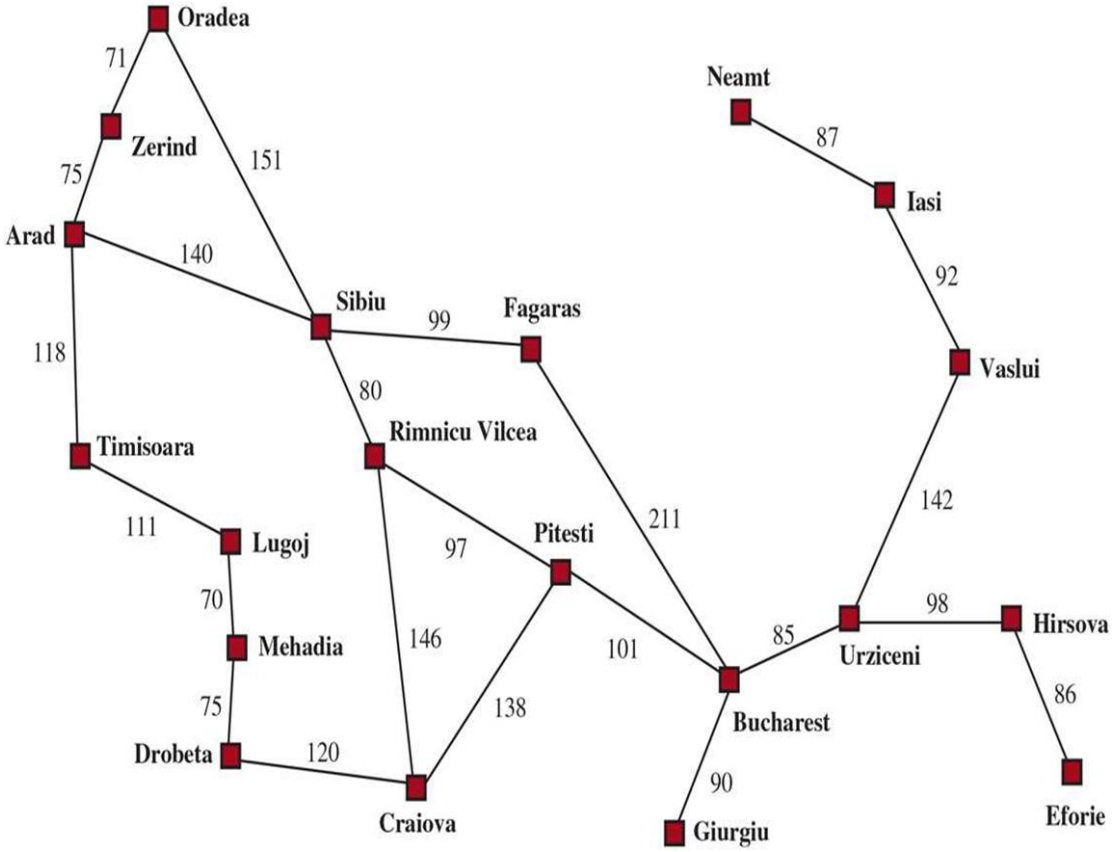
Search problems and solutions

- The **actions** available to the agent. Given a state ACTIONS returns a finite set of actions that can be executed in **s**. Each of these actions is applicable in **s**

  Ex: ACTIONS(Arad) = {ToSibiu, ToTimisoara, ToZerind}.

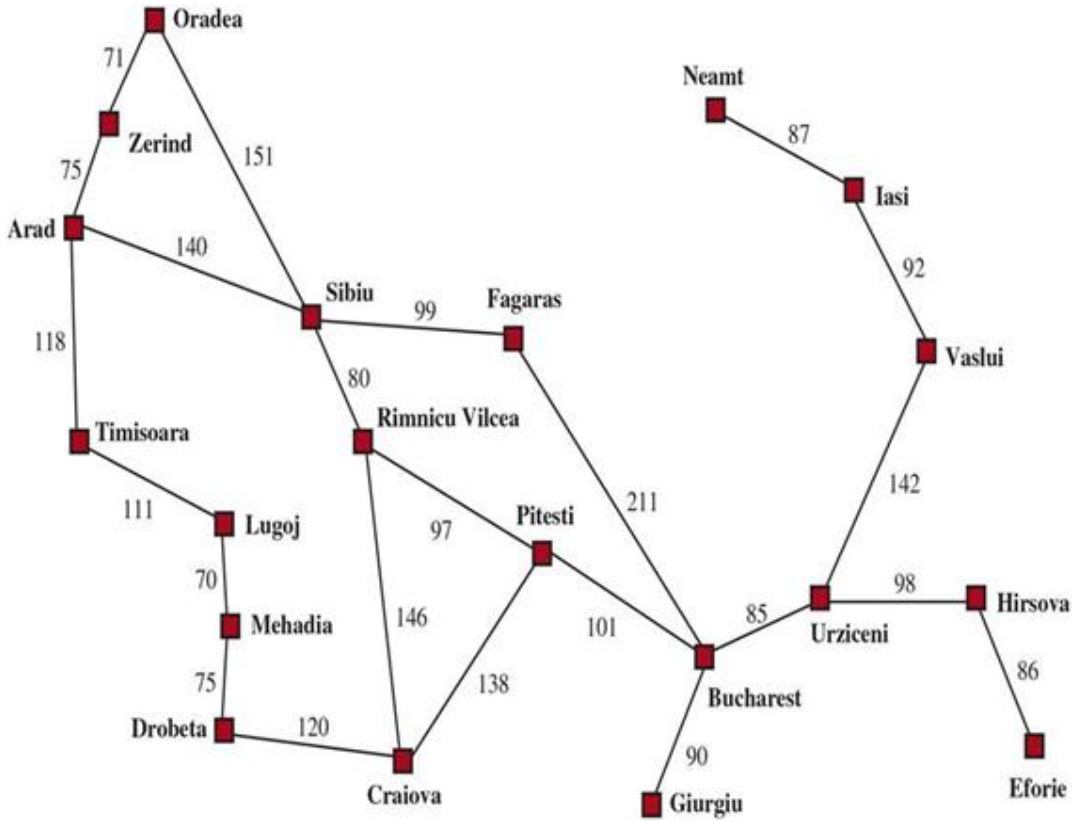
- A **transition model** describes what each action does. **RESULT (s,a)** returns the state that results from doing action in state

  Ex: RESULT(Arad, ToZerind) = Zerind.



Search problems and solutions

- An action cost function, denoted by **ACTION-COST(s,a,s')** gives the numeric cost of applying action **a** in state **s** to reach state **s'**.
- A problem-solving agent should use a cost function that reflects its own **performance measure.**
- **Ex:** For route-finding agents: The cost of an action might be the length in miles or it might be the time it takes to complete the action.

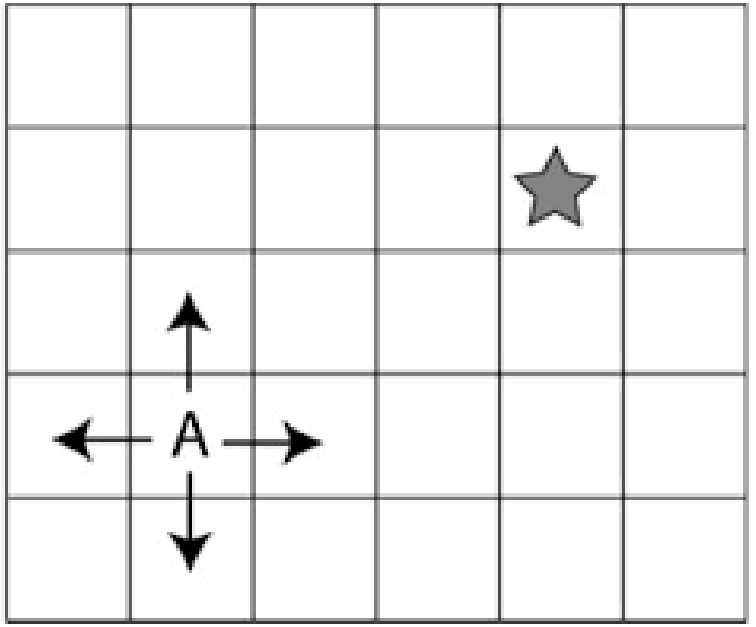


Example Problems

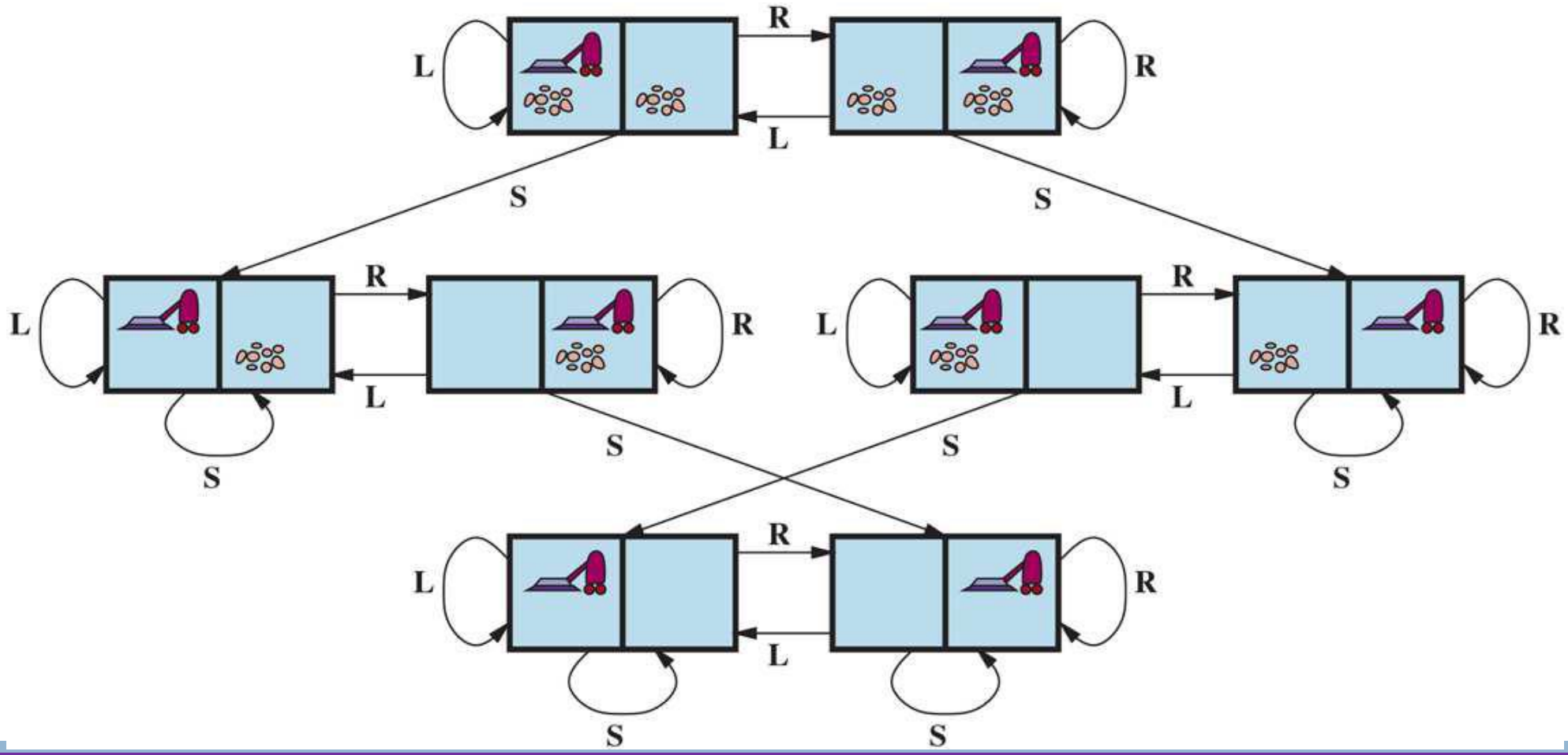
- A **standardized problem** is intended to illustrate or exercise various problem solving methods **suitable as a benchmark** for researchers to compare the performance of algorithms.
- A **real-world problem**, such as **robot navigation**, is one whose solutions **people actually use**.
    - Ex: Each robot has different sensors that produce different data.

Standardized problems: Grid World

- A **grid world** problem is a two-dimensional rectangular array of square cells in which agents can move from cell to cell.
- The agent can move to any obstacle-free adjacent cell, horizontally or vertically and in some problems diagonally.
- Cells can contain objects, that can be pick up, push, or otherwise act upon; a wall or other impassible obstacle in a cell prevents an agent from moving into that cell.
- Ex: The vacuum world formulated as a grid world problem as follows:

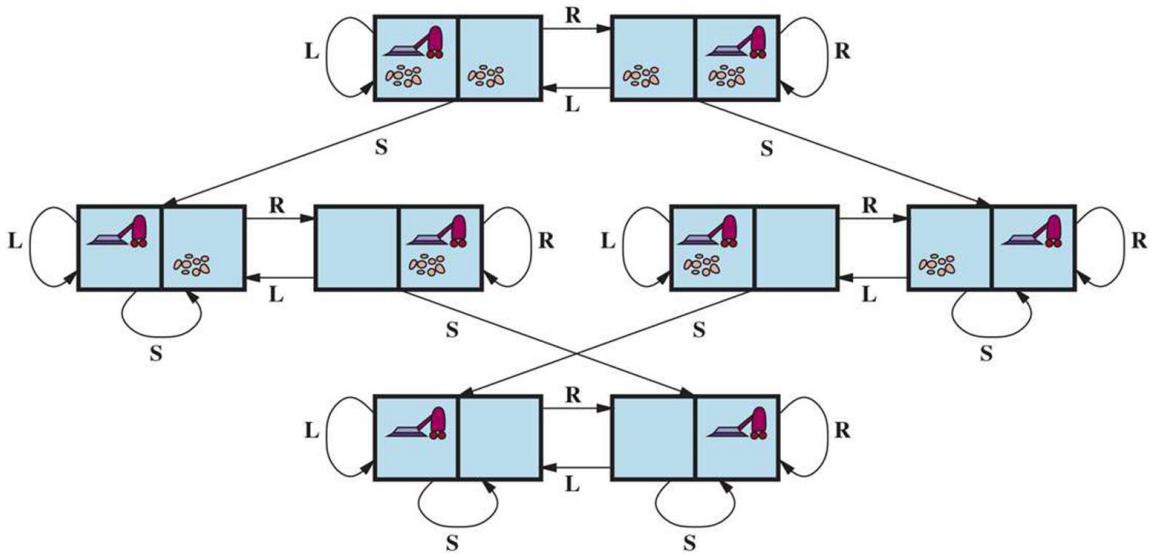


Standardized problems: Grid World: Vacuum World



RAMAIAH
Institute of Technology

Standardized problems: Grid World : Vacuum World

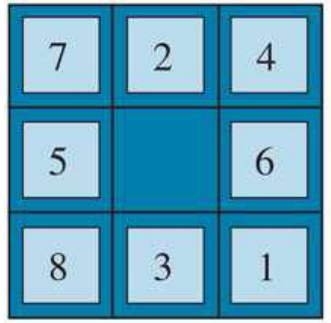


- **STATES:** A state of the world says which objects are in which cells.
  Number of states: $8 = n.2^n$ (n is no. of cells)
- **INITIAL STATE**: Any
- **ACTIONS:** 3 : left, right, suck
- **TRANSITION MODEL**:
  - Suck removes any dirt
  - Forward moves till it hits a wall: action has no effect.
  - Backward moves in the opposite
  - TurnRight and TurnLeft change the direction
- **GOAL**: clean up all dirt :
- **Path Cost**: Each step costs 1

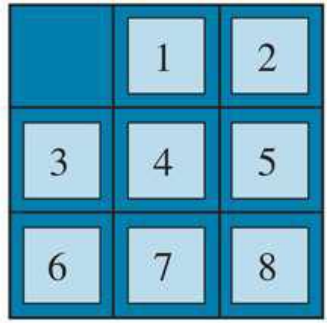
RAMAIAH
Institute of Technology

Standardized problems: Sliding Tile Puzzle

- **The standard formulation of the 8 puzzle is as follows:**
- **STATES**: A state description specifies the location of each of the tiles.
- **INITIAL STATE**: Any state can be designated as the initial state.
- **ACTIONS**: Physical world: Tile slides.
  Describing an action is to think of the blank space moving Left, Right, Up, or Down.
- **TRANSITION MODEL:** Maps a state and action to a resulting state; **Ex**: Apply Left to the start state, the resulting state has the 5 and the blank switched.
- **GOAL STATE:** Numbers in order .
- **ACTION COST:** Each action costs 1.



Start State



Goal State

RAMAIAH
Institute of Technology

Example problems

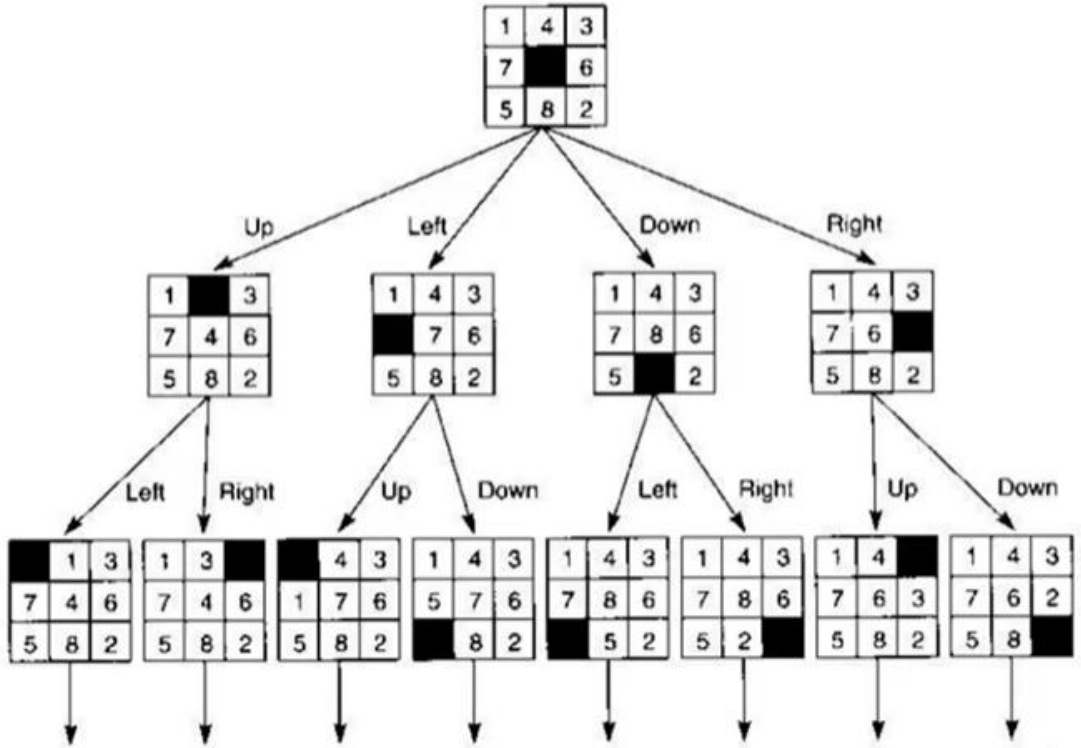
The 8-puzzle has 9!/2 = 181, 440 reachable states and is easily solved.

2	1	6
4		8
7	5	3

Initial State

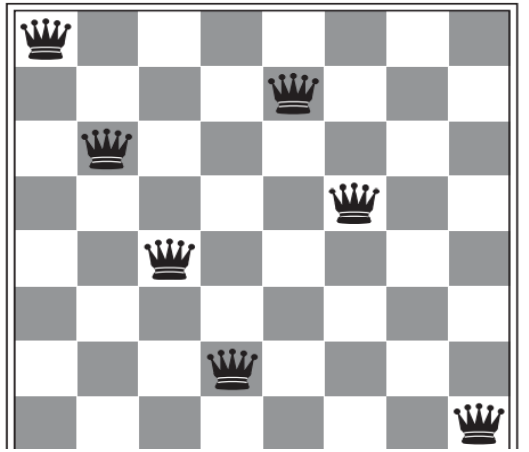
1	2	3
8		4
7	6	5

Goal State



RAMAIAH
Institute of Technology

Example problems

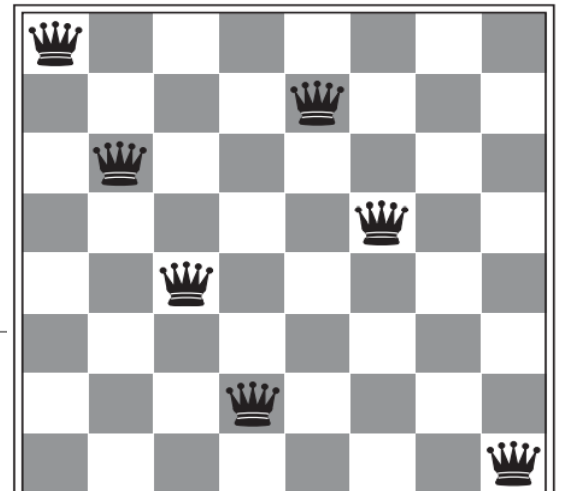


Toy problems - 8-queens

- n queens puzzle of placing 8 queens on an n×n chessboard
- 8 chess queens on an 8×8 chessboard such that none of them is able to capture any other using the standard chess queen's moves.
- Any queen is assumed to be able to attack any other.
- Thus, a solution requires that no two queens share the same row, column, or diagonal.

RAMAIAH
Institute of Technology

Example problems



Toy problems - 8-queens

There are two main kinds of formulation.

An incremental formulation involves operators that augment the state description, starting with an empty state; for the 8-queens problem, this means that each action adds a queen to the state.

A complete-state formulation starts with all 8 queens on the board and moves them around.

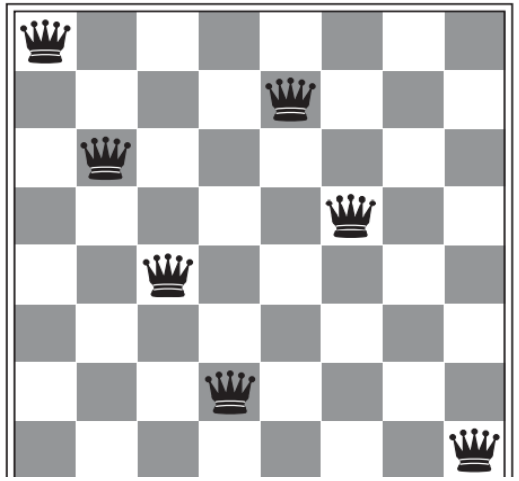
RAMAIAH
Institute of Technology

Example problems

Toy problems - 8-queens

As per the incremental formulation :

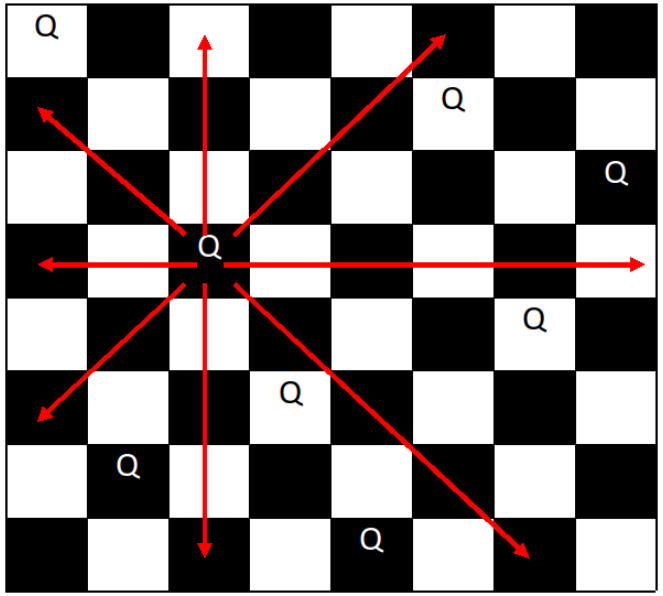
- **States**: Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state**: No queens on the board.
- **Actions**: Add a queen to any empty square.
- **Transition model**: Returns the board with a queen added to the specified square.
- **Goal test**: 8 queens are on the board, none attacked.

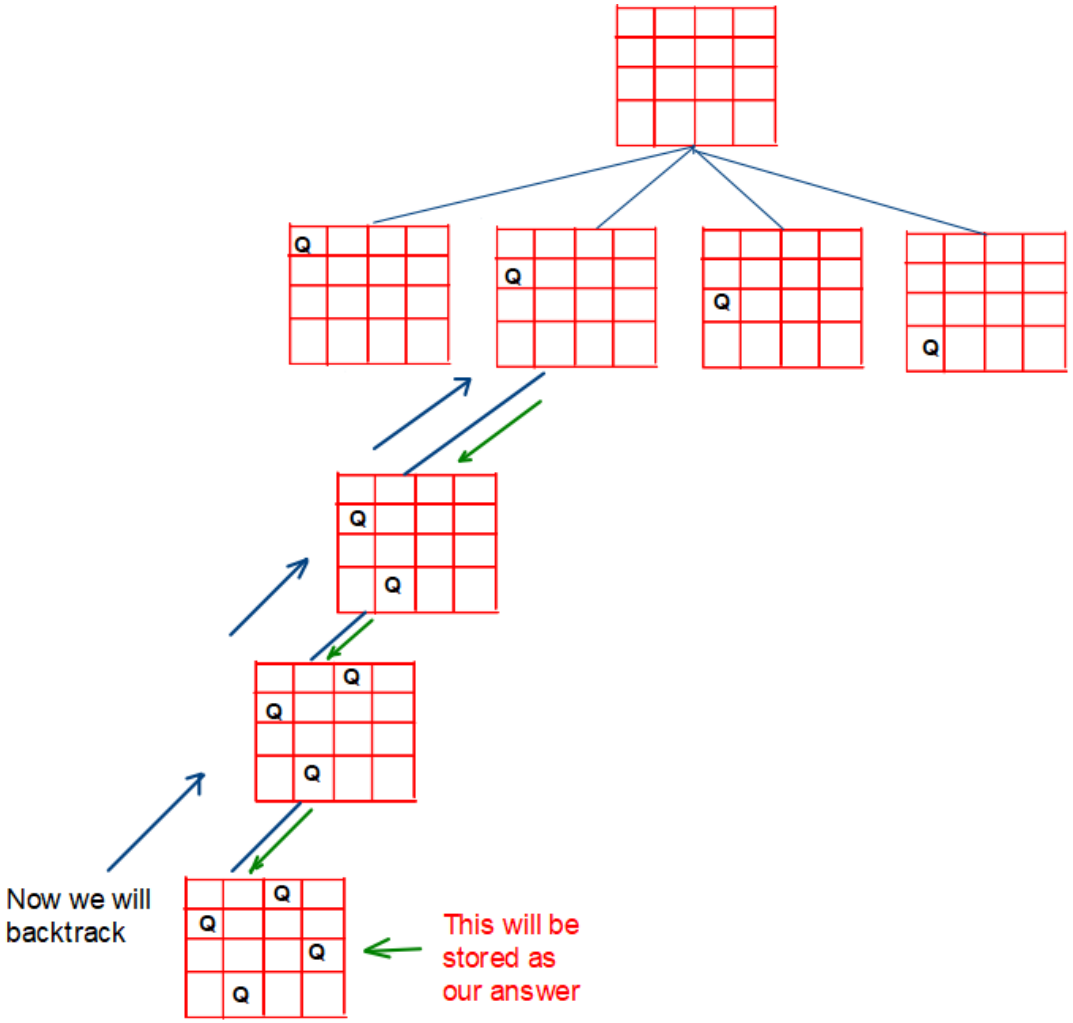


RAMAIAH
Institute of Technology

Example problems

For 8*8 chess board with 8 queens there are total of 92 solutions for the puzzle.





Real-world problems

Airline travel problems

STATES: Each state obviously includes a location (e.g., an airport) and the current time.

INITIAL STATE: The user's home airport.

ACTIONS: Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.

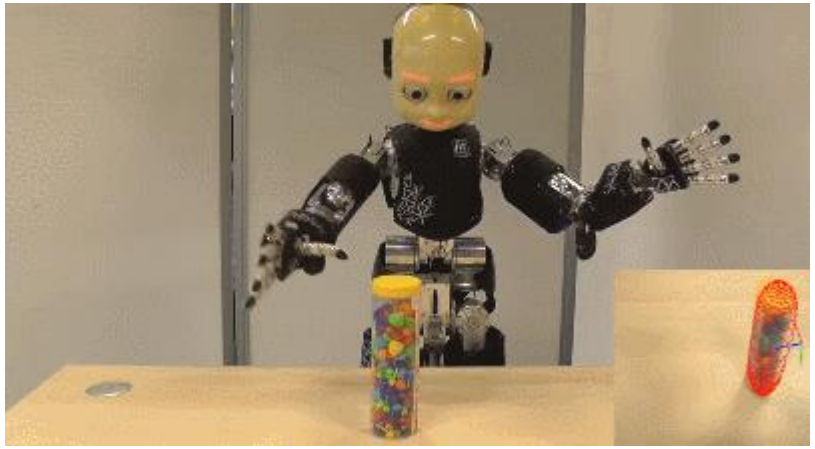
TRANSITION MODEL: The state resulting from taking a flight will have the flight's destination as the new location and the flight's arrival time as the new time.

GOAL STATE: A destination city.

ACTION COST: A combination of monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer reward points, and so on.

Real-world problems

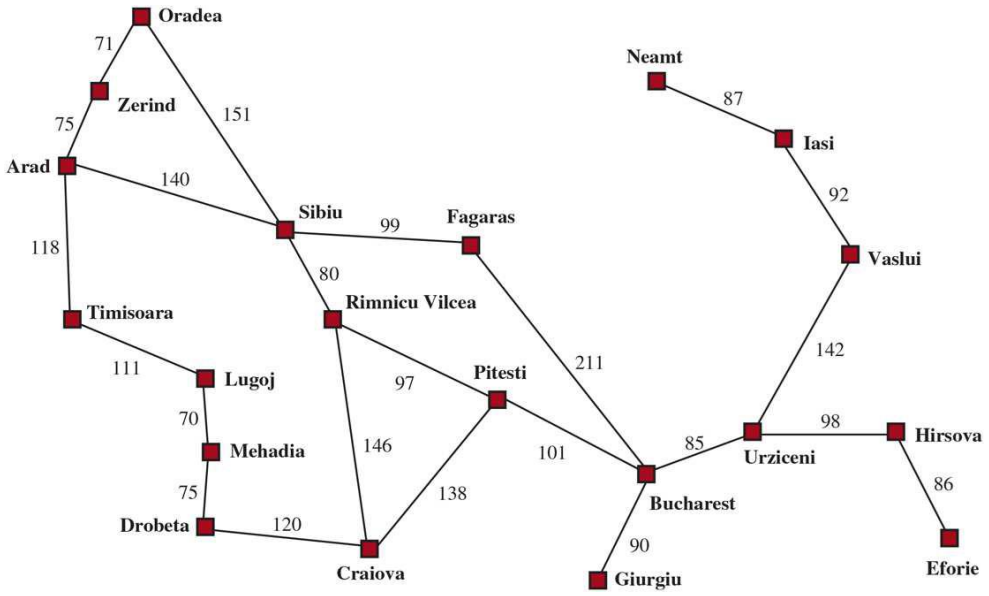
- **Robot Navigation**
- A robot can roam around, in effect making its own paths.
- **Circular robot** moving on a flat surface, the space is essentially two-dimensional.
- **Robot with arms and legs also be controlled:**
- The search space becomes many-dimensional. One dimension for each joint angle.
- In addition to the complexity of the problem, real robots must also deal:
  - Errors in their sensor readings and motor controls,
  - Partial observability, and
  - Other agents that might alter the environment.



RAMAIAH
Institute of Technology

Search Algorithms

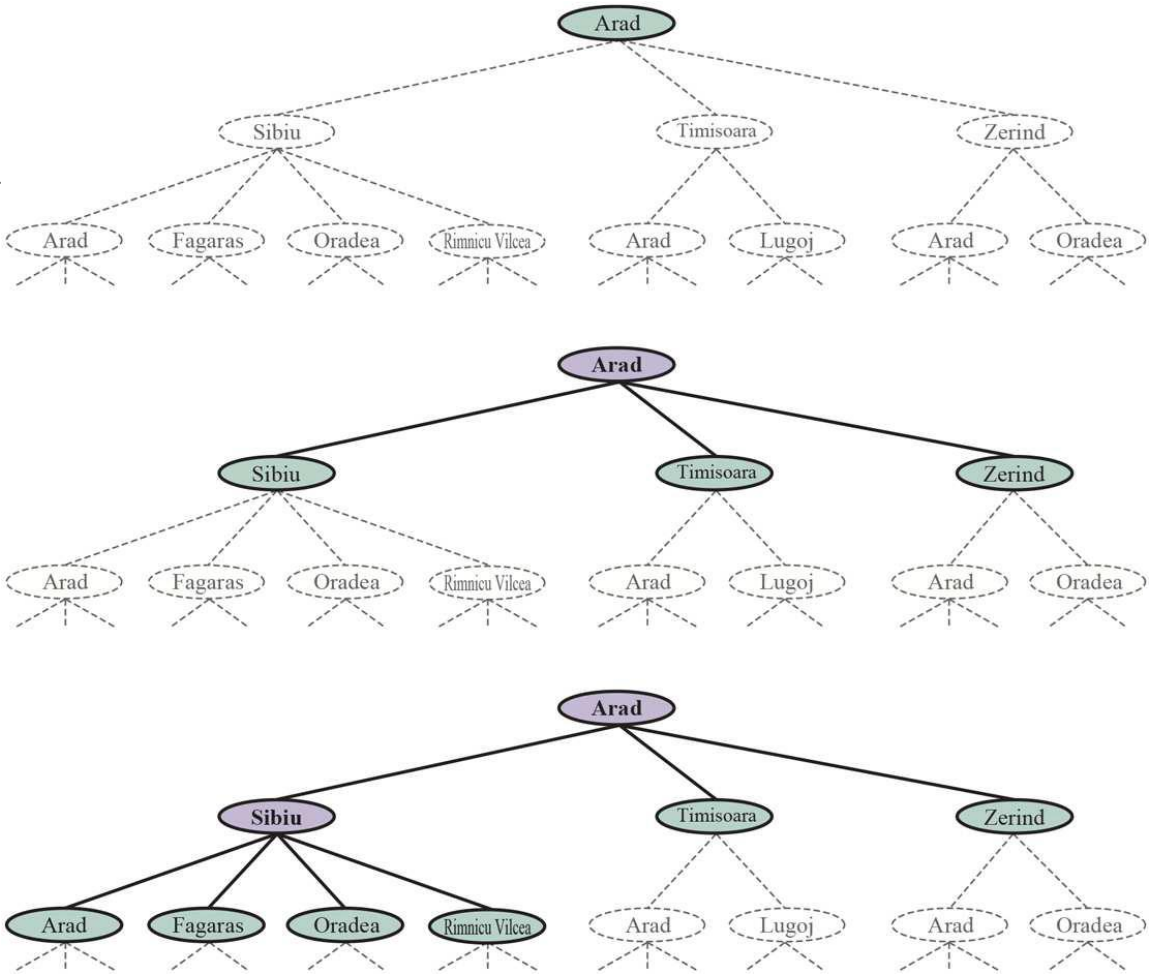
- A search algorithm
  - takes a search problem as input
  - returns a solution, or an indication of failure.
- In a search tree
  - Node => State in the state space
  - Edges =>actions.
  - root => initial state of the problem.
- The **state space** describes the set of states in the world, and the actions that allow transitions from one state to another.
- The search tree may have **multiple paths** to any given state, but each node in the tree has a unique path back to the root.



RAMAIAH
Institute of Technology

Search Algorithms

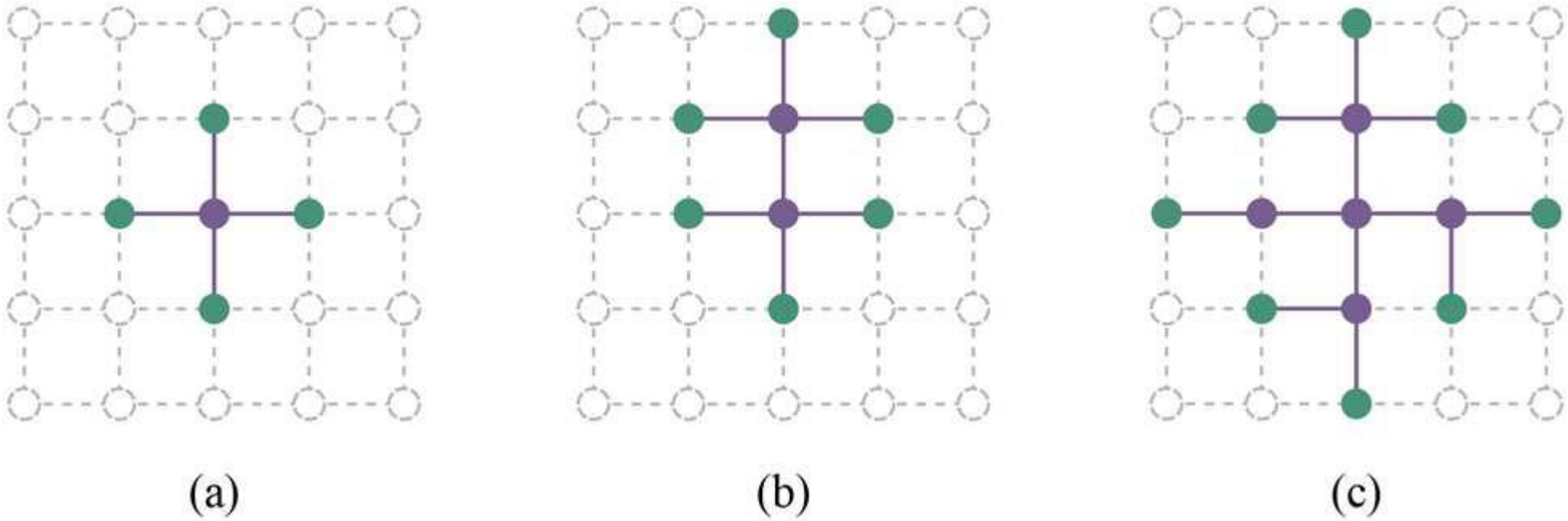
- **Three partial search** trees for finding a route from Arad to Bucharest.
- **Expanded nodes : Lavender** with bold letters; **Frontier nodes:** Generated not yet expanded are in **green**; the set of states corresponding to these two types of nodes are said to have been **reached**.
- Nodes that could be **generated next** are shown in faint dashed lines.
- **Note**: Bottom tree there is a **cycle** from Arad to Sibiu to Arad; that can't be an optimal path.



RAMAIAH
Institute of Technology

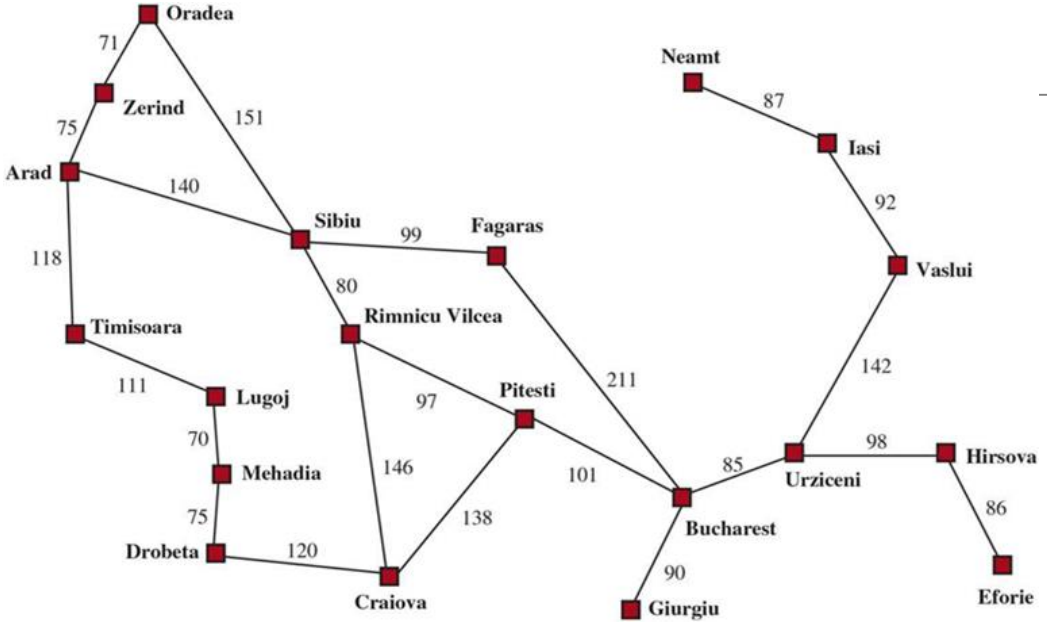
Search Algorithms

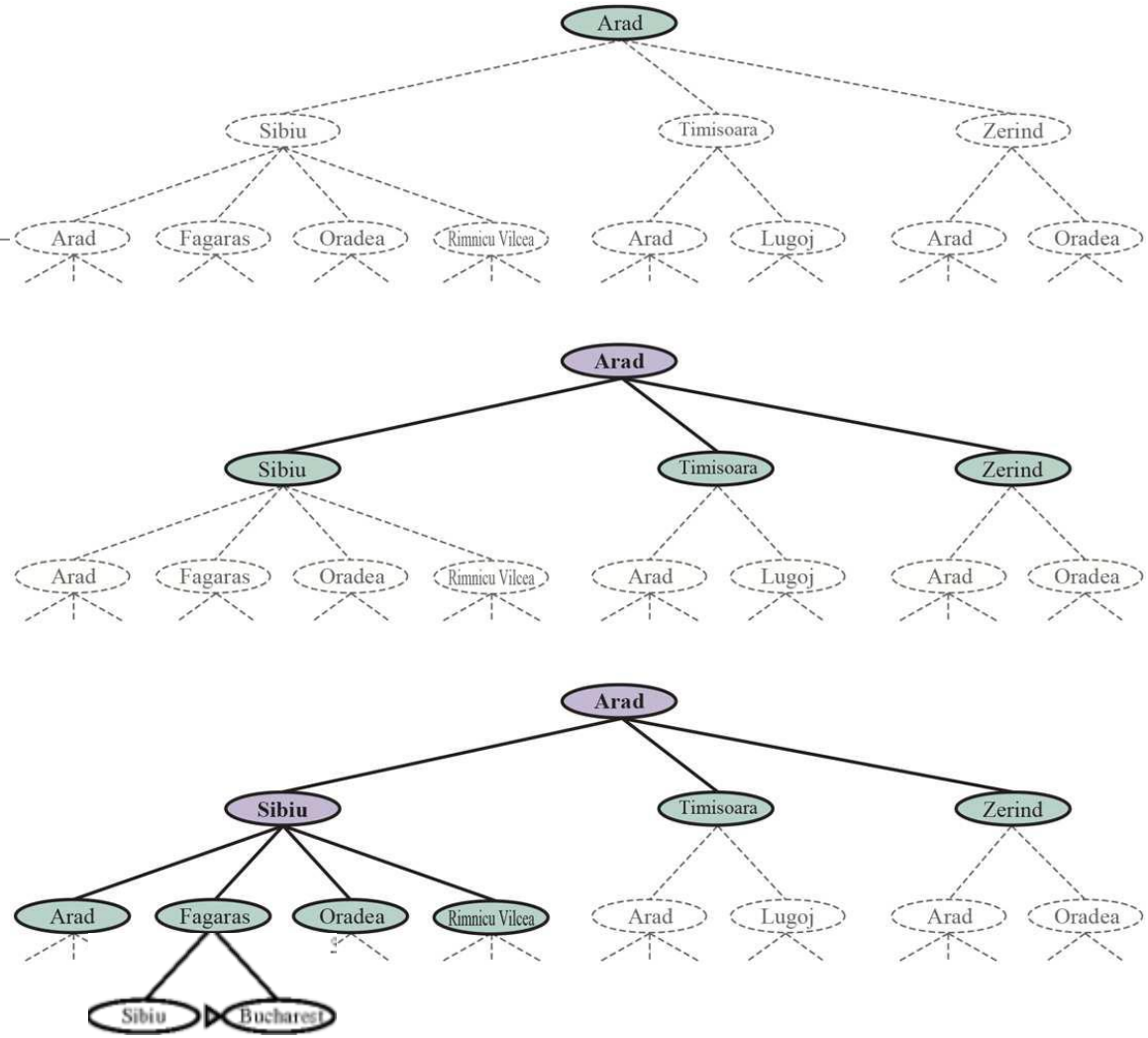
- **Sequence** separates two regions of the state-space graph:
  - an **interior region** where every state has been **expanded(Lavender).**
  - an **exterior region** of states that have not yet been **reached**(Green).



RAMAIAH
Institute of Technology

Best-first search





RAMAIAH
Institute of Technology

Best-first search Text Book Pseudocode

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

RAMAIAH
Institute of Technology

Best-first search Pseudocode

```
1. Best-First-Search(Graph, Start_Node, Goal_Node)
2.   Create an empty Priority Queue called Open (priority based on heuristic)
3.   Create an empty Set called Close
4.   Insert Start_Node into Open with priority equal to its heuristic value
5.
6.   While Open is not empty:
7.       Remove the node N with the lowest heuristic value from Open
8.
9.       If N is the Goal_Node:
10.          Return the path or success
11.
12.      Add N to Close
13.
14.      For each neighbor of N:
15.          If the neighbor is not in Close and not in Open:
16.              Calculate the heuristic value of the neighbor
17.              Insert the neighbor into Open with priority equal to its heuristic value
18.          Else if the neighbor is already in Open:
19.              If the current path to the neighbor is better (lower heuristic), update its priority
20.
21.  If Goal_Node was not found:
22.      Return failure
```

RAMAIAH
Institute of Technology

Best-first search

Search data structures:

Search algorithms require a data structure to keep track of the search tree.

A node in the tree is represented by a data structure with four components:

1. node.STATE: The state to which the node corresponds;
2. node.PARENT: The node in the tree that generated this node;
3. node.ACTION: The action that was applied to the parent's state to generate this node;
4. node.PATH-COST: The total cost of the path from the initial state to this node.

RAMAIAH
Institute of Technology

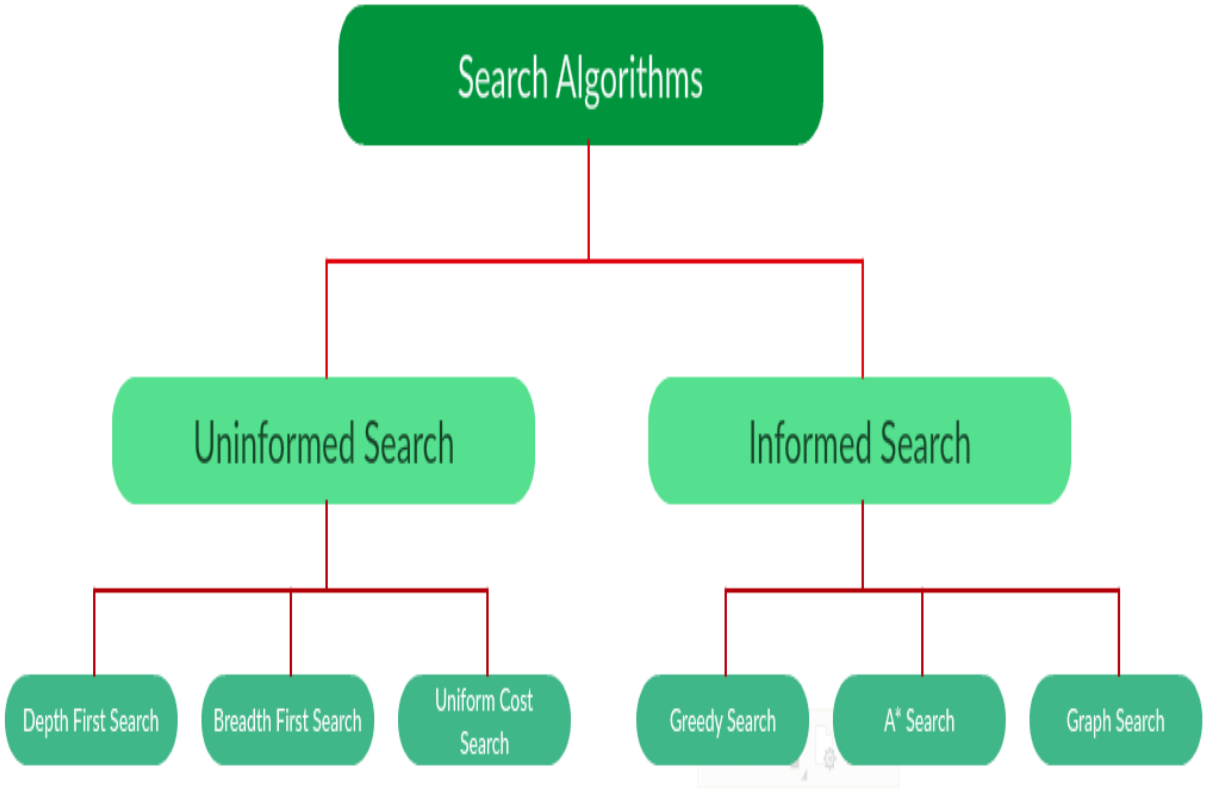
Search Strategies

Uninformed search (Blind search)

- No information about the number of steps or the path cost from the current state to the goal
- Search the state space **blindly**

Informed search, or heuristic search

- A cleverer strategy that searches toward the goal, based on the information from the current state so far



RAMAIAH
Institute of Technology

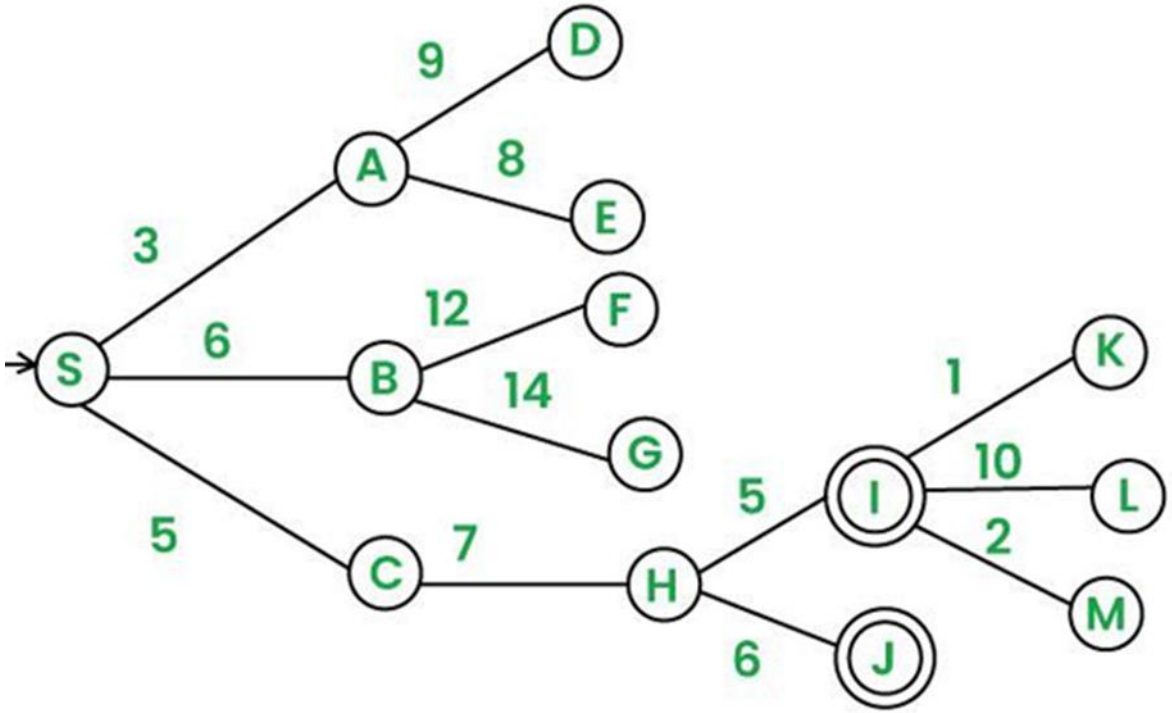
Measuring problem-solving performance

In AI, complexity is expressed with 3 quantities

- **b**, branching factor, maximum number of successors of any node
- **d**, the depth of the shallowest goal node.
- **m**, the maximum length of any path in the state space

Time and Space is measured in

- **Time** - number of nodes generated during the search
- **Space** -maximum number of nodes stored in memory



Measuring problem-solving performance

- **There are 4 criteria's used to choose among search algorithms.**
- **Algorithm's performance can be evaluated in four ways:**
- The evaluation of a search strategy in 4 ways:
  - **Completeness:**
    - is the strategy guaranteed to find a solution when there is one?
  - **Optimality**:
    - does the strategy find the highest-quality solution when there are several different solutions?
  - **Time complexity**:
    - how long does it take to find a solution?
  - **Space complexity**:
    - how much memory is needed to perform the search?

RAMAIAH Institute of Technology

For the following graph find the traversal path and path cost from S as initial node to I as Destination node using Best First Search Technique.

1. Start with:
   Open List: {S}
   Closed List: {}
2. Explore node S:
   Open List: {A (3), B (6), C (5)}
   Closed List: {S}
3. Explore node A:
   Open List: {B (6), C (5), D (9), E (8)}
   Closed List: {S, A}
4. Explore node C:
   Open List: {B (6), D (9), E (8), H (7)}
   Closed List: {S, A, C}
5. Explore node B:
   Open List: {D (9), E (8), H (7), F (12), G (14)}
   Closed List: {S, A, C, B}
6. Explore node H:
   Open List: {D (9), E (8), F (12), G (14), I (5), J (6)}
   Closed List: {S, A, C, B, H}

Goal node I reached! Reconstruct the Final Path:

Reconstruct the path using the Parent mapping:

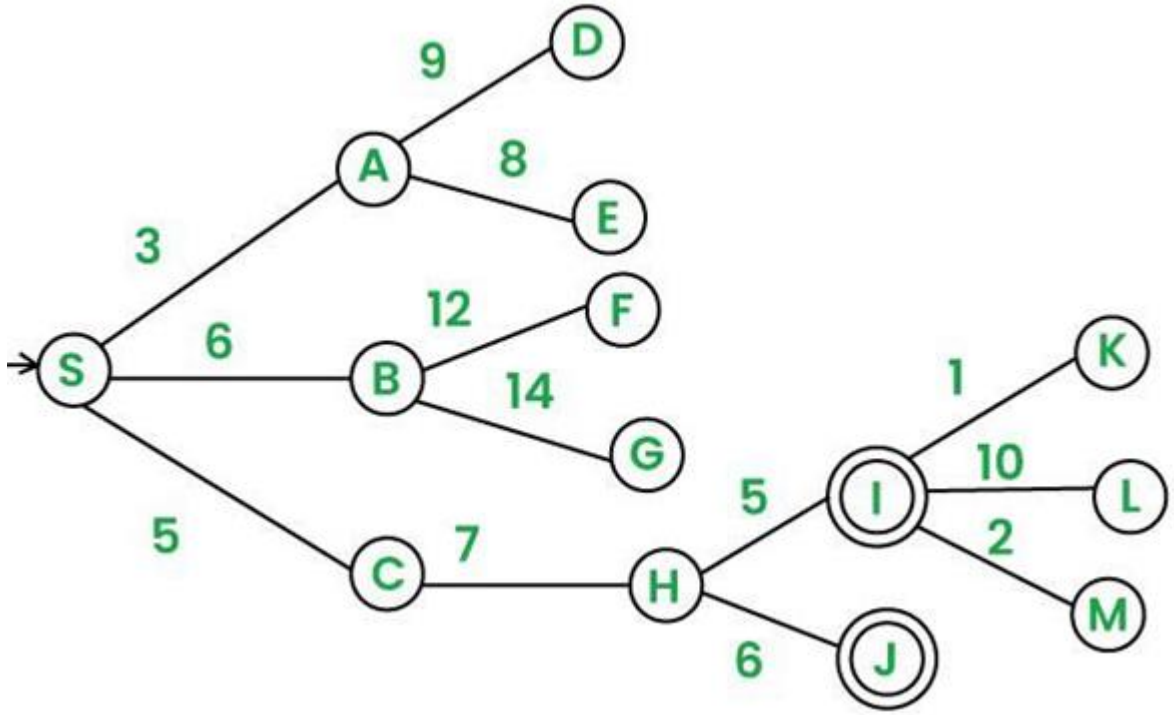
Start from node I, and look up its parent:

Parent of I = H, Parent of H = C , Parent of C = S

Reverse the sequence to get the final path:

Path: S → C → H → I

Path Cost = 17



RAMAIAH Institute of Technology

For the following graph find the traversal path and path cost from S as initial node to G as Destination node using Best First Search Technique.

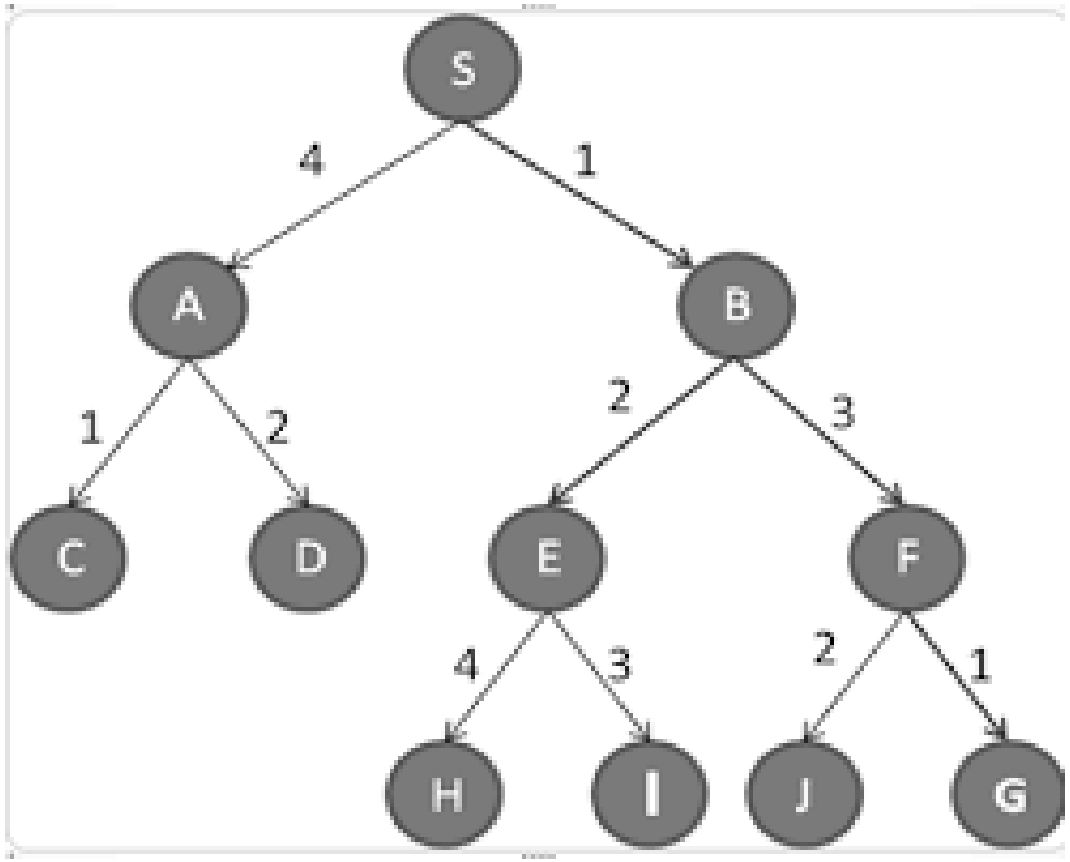
1. Start with:
   Open List: {S}
   Closed List: {}
2. Explore node S:
   Open List: {A (4), B (1)}
   Closed List: {S}
3. Explore node B:
   Open List: {A (4), F (3), E (2)}
   Closed List: {S, B}
4. Explore node E:
   Open List: {A (4), F (3), H (4), I (3)}
   Closed List: {S, B, E}
5. Explore node I:
   Open List: {A (4), I(3), H (4)}
   Closed List: {S, B, E,F}
6. Explore node F:
   Open List: {A (4), H (4),J(2),G(1)}
   Closed List: {S, B, E,I,F}
7. Explore node G:
   Open List: {A (4), I(3),H (4),J(2)}
   Closed List: {S, B, E,F,G}

Goal node I reached!

Reconstruct the Final Path: Start from node G, and look up its parent:

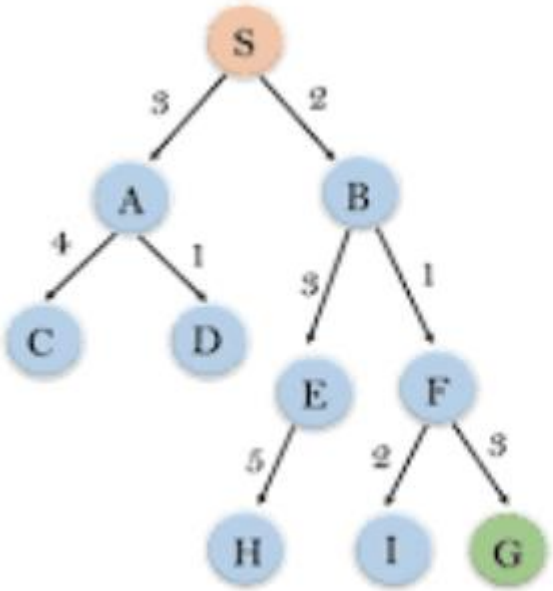
- Parent of G = F, Parent of F = B, Parent of B = S

Path: S → B → F → G, Path Cost = 5

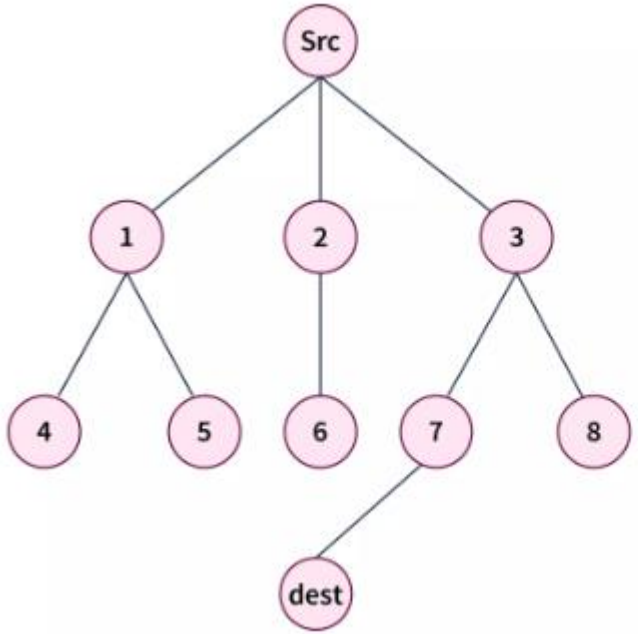


RAMAIAH Institute of Technology

For the following graph find the traversal path and path cost from Src as initial node to dest as Destination node using Best First Search Technique.



Node	H(n)
A	10
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0



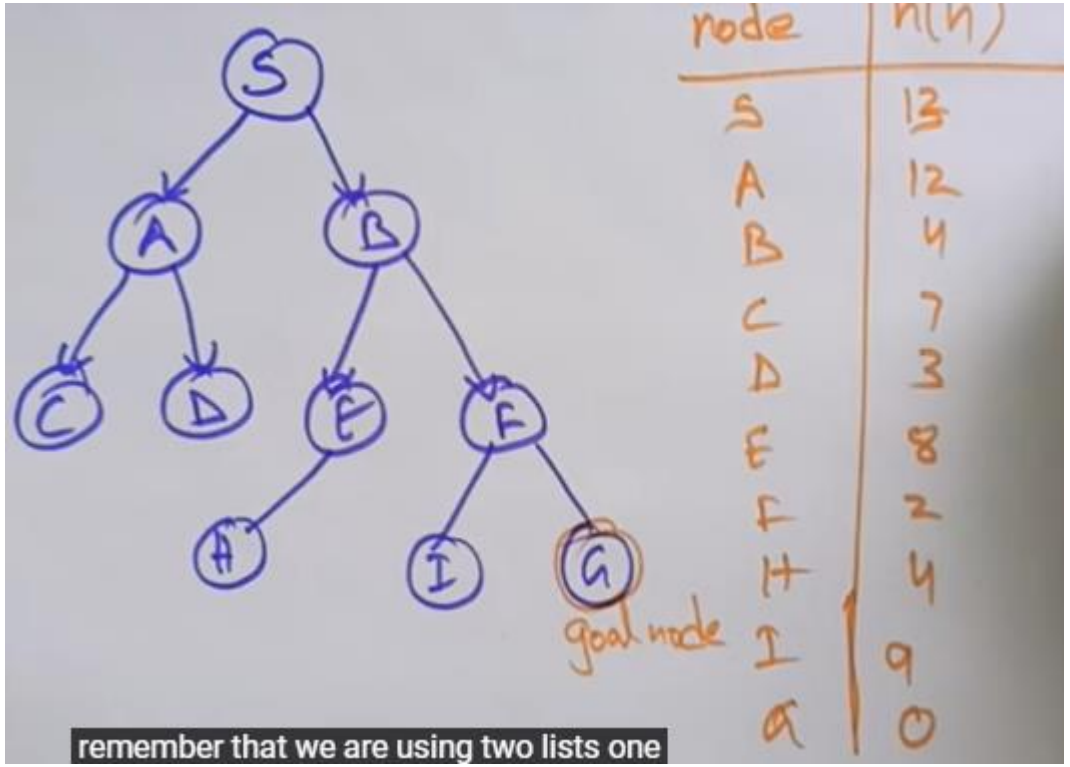
Src	20
1	22
2	21
3	10
4	25
5	24
6	30
7	5
8	12
dest	0

RAMAIAH Institute of Technology

For the following graph find the traversal path and path cost from S as initial node to G as Destination node using Best First Search Technique.

1. Start with:
   - Open List: {S}
   - Closed List: {}
2. Explore node S:
   - Open List: {A (12), B (4)}
   - Closed List: {S}
3. Explore node B:
   - Open List: {A (12), F (2), E (8)}
   - Closed List: {S, B}
4. Explore node F:
   - Open List: {A (12), I (9), G(0)}
   - Closed List: {S, B, F}
5. Explore node G:
   - Open List: {A (12), I (9), G(0)}
   - Closed List: {S, B, F,G}

Goal node I reached!



RAMAIAH
Institute of Technology

Closed List: {S, B, F,G}

Reconstruct the Final Path:

Reconstruct the path using the Parent mapping:

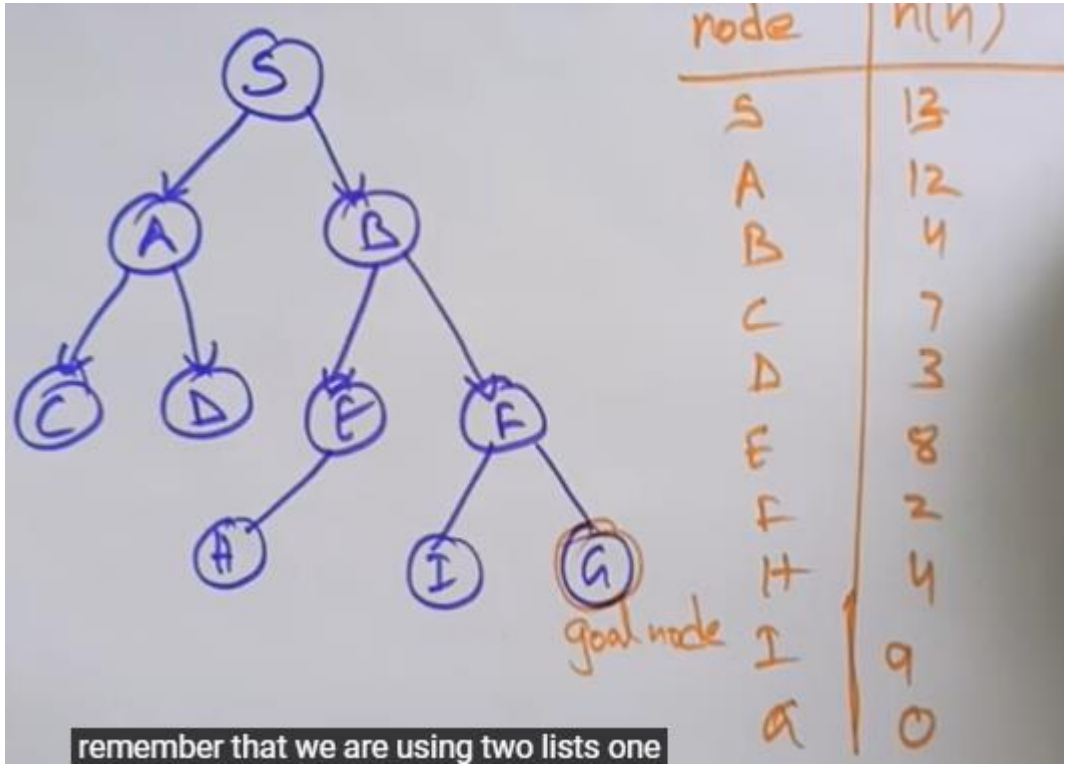
Start from node G, and look up its parent:

- Parent of G = F
- Parent of F = B
- Parent of B = S

Reverse the sequence to get the final path**:**

Path: S → B → F → G

Path Cost = 4+2+0=6



RAMAIAH
Institute of Technology

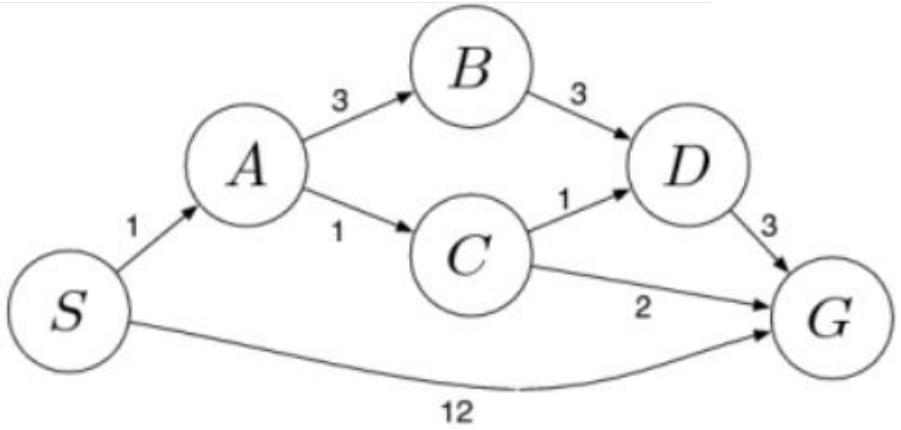
Uniform cost search

- Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph.
- The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost.
- Uniform-cost search expands nodes according to their path costs form the root node.
- A uniform-cost search algorithm is implemented by the priority queue.

RAMAIAH Institute of Technology

For the following graph find the traversal path and path cost from S as initial node to G as Destination node using Uniform cost Search Technique.

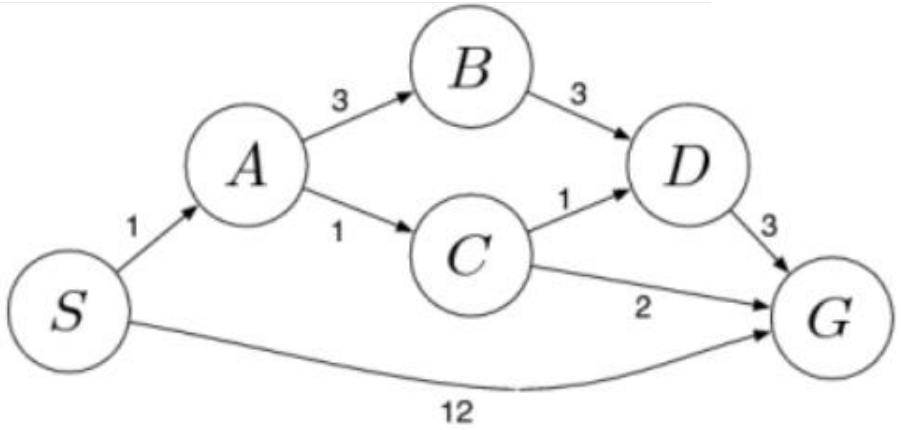
1. **Start at S:**
   - **Open list:** $[(S, 0)]$
   - **Closed list:** $[]$
   - Current path: S
   - Current cost: 0
2. **Expand node S:**
   - From S:
     - $S \to A$ with cost 1.
     - $S \to C$ with cost 12.
   - **Open list:** $[(A, 1), G', 12)]$
   - **Closed list:** $[S]$
3. **Expand node A** (node with minimum cost in the open list):
   - From A:
     - $A \to B$ with cumulative cost $1 + 3 = 4$.
     - $A \to C$ with cumulative cost $1 + 1 = 2$.
   - **Open list:** $[(C, 2), (B, 4), (G, 12)]$
   - **Closed list:** $[S, A]$
4. **Expand node C** (node with minimum cost in the open list):
   - From C:
     - $C \to D$ with cumulative cost $2 + 1 = 3$.
     - $C \to G$ with cumulative cost $2 + 2 = 4$.
   - **Open list:** $[(D, 3), (B, 4), (G, 4), (G, 12)]$
   - **Closed list:** $[S, A, C]$



RAMAIAH
Institute of Technology

Uniform cost search

5. **Expand node** D (node with minimum cost in the open list):
   - From D:
     - $D \rightarrow G$ with cumulative cost $3 + 3 = 6$.
   - **Open list:** $[(B, 4), (G, 4), (G, 6), (G, 12)]$
   - **Closed list:** $[S, A, C, D]$
6. **Expand node** G (node with minimum cost in the open list):
   - **Open list:** $[(B, 4), (G, 6), (G, 12)]$
   - **Closed list:** $[S, A, C, D, G]$

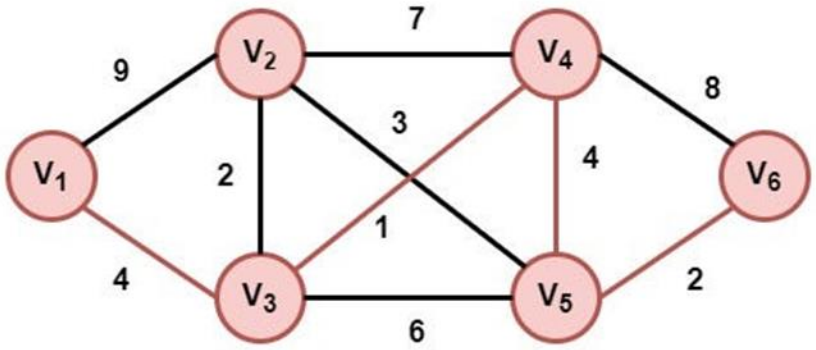


Path:

The optimal path found using UCS with the open and closed lists is $S \rightarrow A \rightarrow C \rightarrow G$ with a total cost of 4.

RAMAIAH
Institute of Te

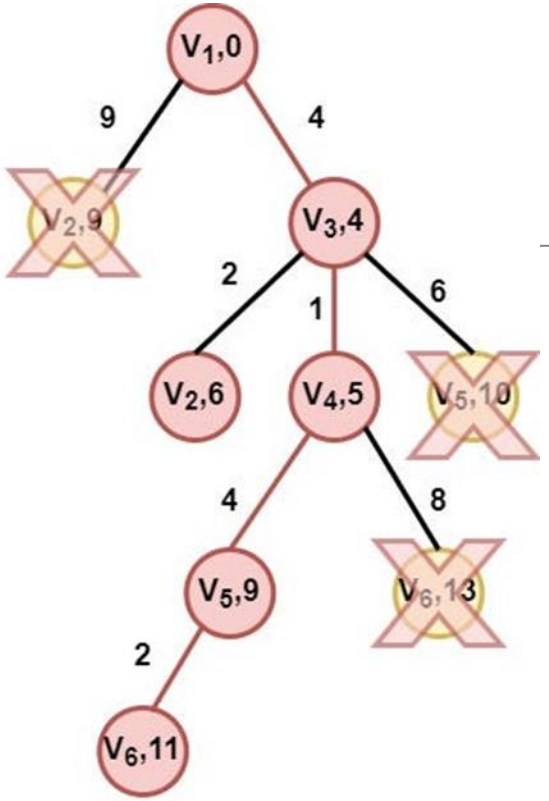
Un



Pace	Opened	Closed
0	$\{(V_1,0)\}$	{-}
1	$\{(V_2,9),(V_3,4)\}$	$\{(V_1,0)\}$
2	$\{(V_2,6),(V_4,5),(V_5,10)\}$	$\{(V_1,0),(V_3,4)\}$
3	$\{(V_2,6),(V_6,13),(V_5,9)\}$	$\{(V_1,0),(V_3,4),(V_4,5)\}$
4	$\{(V_6,13),(V_5,9)\}$	$\{(V_1,0),(V_3,4),(V_4,5),(V_2,6)\}$
5	$\{(V_6,11)\}$	$\{(V_1,0),(V_3,4),(V_4,5),(V_2,6),(V_5,9)\}$
6	{-}	$\{(V_1,0),(V_3,4),(V_4,5),(V_2,6),(V_5,9)\}$

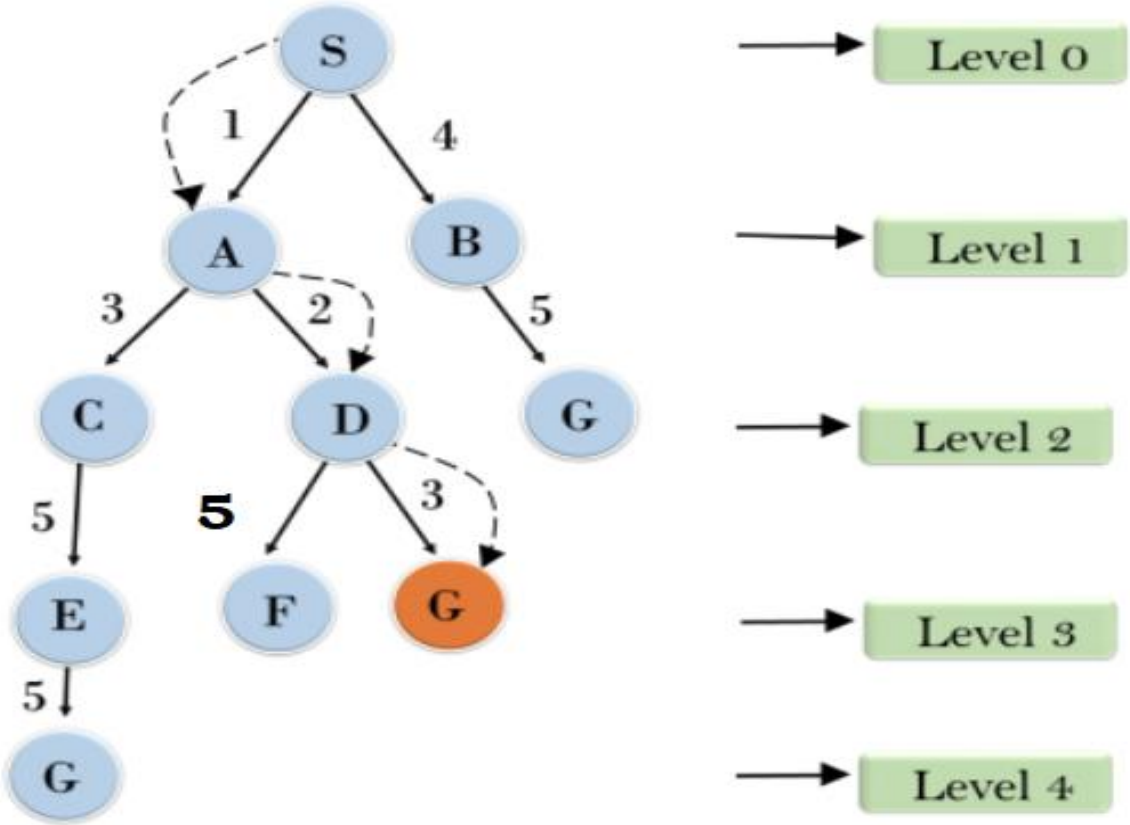
Path: V_1 - V_3 - V_4 - V_5 - V_6

Total Cost: 11



RAMAIAH Institute of Technology

For the following graph find the traversal path and path cost from S as initial node to G as Destination node using Uniform cost Search Technique.



RAMAIAH
Institute of Technology

Uniform cost search

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure

    node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
    frontier ← a priority queue ordered by PATH-COST, with node as the only element
    explored ← an empty set
    loop do
        if EMPTY?(frontier) then return failure
        node ← POP(frontier)   /* chooses the lowest-cost node in frontier */
        if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
        add node.STATE to explored
        for each action in problem.ACTIONS(node.STATE) do
            child ← CHILD-NODE(problem, node, action)
            if child.STATE is not in explored or frontier then
                frontier ← INSERT(child, frontier)
            else if child.STATE is in frontier with higher PATH-COST then
                replace that frontier node with child
```

RAMAIAH Institute of Technology

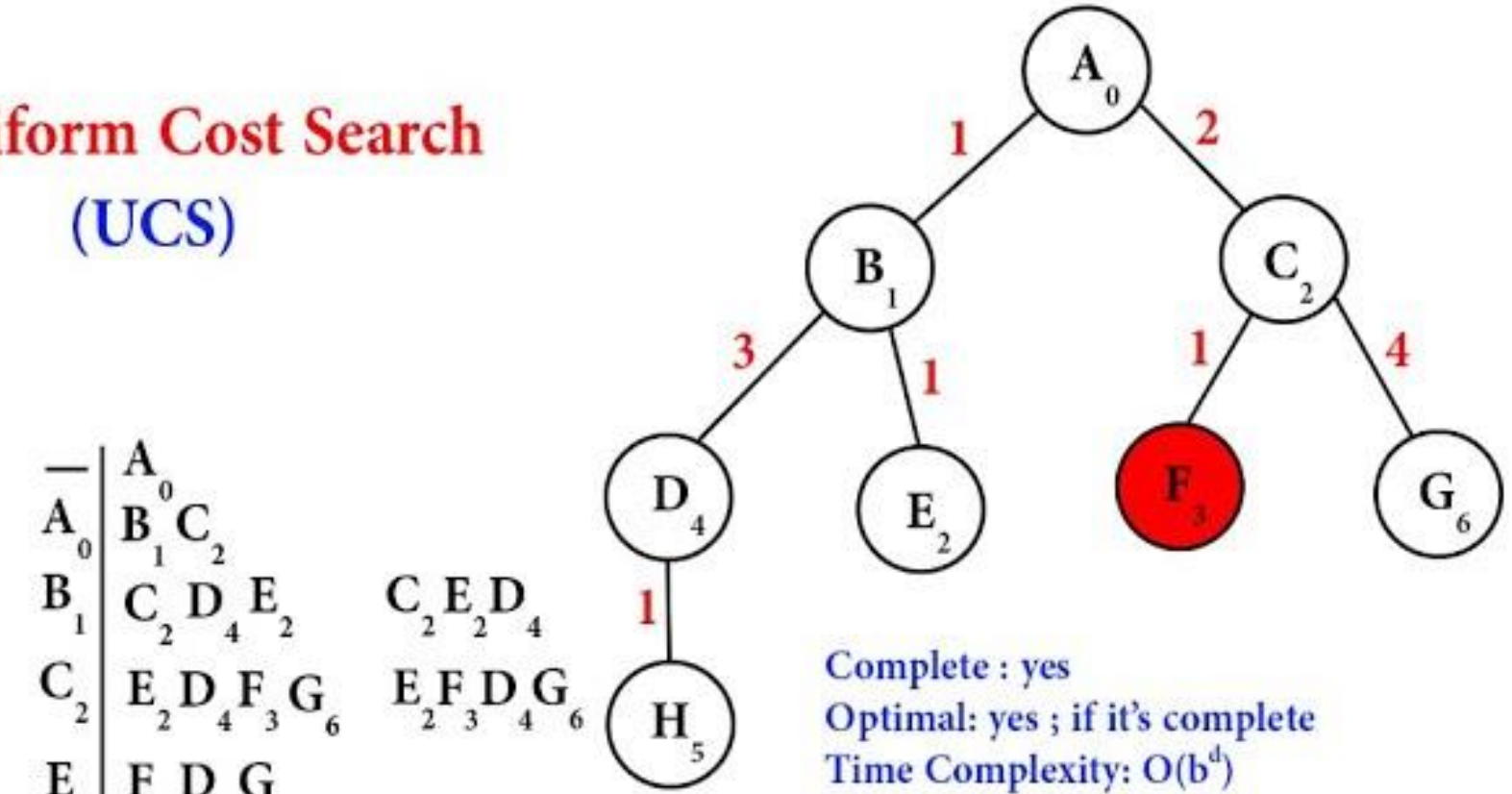
Uniform cost search

```
1. Uniform-Cost-Search(Graph, Start_Node, Goal_Node)
2.    Create an empty Priority Queue called Open (priority based on path cost)
3.    Create an empty Set called Close
4.    Insert Start_Node into Open with priority (cost) = 0
5.
6.    While Open is not empty:
7.        Remove the node N with the lowest path cost from Open
8.
9.        If N is the Goal_Node:
10.           Return the path or success
11.
12.       Add N to Close
13.
14.       For each neighbor of N:
15.           Calculate the path cost to reach the neighbor via N
16.
17.           If the neighbor is not in Close and not in Open:
18.               Insert the neighbor into Open with the calculated path cost
19.           Else if the neighbor is already in Open with a higher cost:
20.               Update the neighbor's cost in Open with the lower path cost
21.
22.   If Goal_Node was not found:
23.       Return failure
```

RAMAIAH Institute of Technology

For the following graph find the traversal path and path cost from A as initial node to F as Destination node using Uniform cost Search Technique.

Uniform Cost Search (UCS)



—	A_0
A_0	B_1 C_2
B_1	C_2 D_4 E_2
C_2	E_2 D_4 F_3 G_6
E_2	F_3 D_4 G_6
F_3	Goal

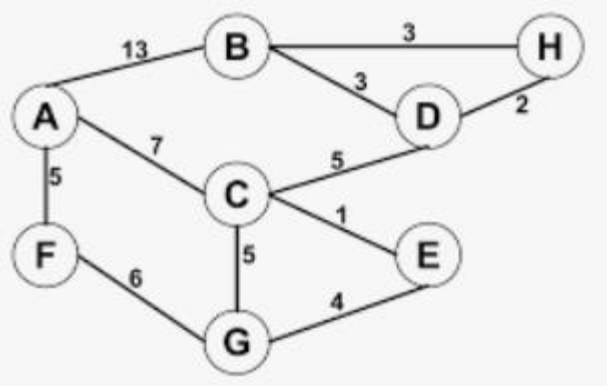
C_2 E_2 D_4

E_2 F_3 D_4 G_6

Complete : yes
Optimal: yes ; if it's complete
Time Complexity: $O(b^d)$
Space Complexity: $O(b^d)$

RAMAIAH
Institute of Technology

For the following graph find the traversal path and path cost from A as initial node to G as Destination node using Uniform cost Search Technique.



RAMAIAH
Institute of Technology

Measuring problem-solving performance

- **There are 4 criteria's used to choose among search algorithms.**
- **Algorithm's performance can be evaluated in four ways:**
- The evaluation of a search strategy in 4 ways:
  - **Completeness:**
    - is the strategy guaranteed to find a solution when there is one?
  - **Optimality**:
    - does the strategy find the highest-quality solution when there are several different solutions?
  - **Time complexity**:
    - how long does it take to find a solution?
  - **Space complexity**:
    - how much memory is needed to perform the search?

Greedy best-first search

- The evaluation function is f(n) = h(n)
- Were, h(n)= estimated cost from node n to the goal.

- Greedy search ignores the cost of the path that has already been traversed to reach n
- Therefore, the solution given is not necessarily optimal

RAMAIAH Institute of Technology

Gre

1. **Heuristic**:
   - The heuristic in Greedy Best First Search is a function, denoted as $h(n)$, that estimates the cost or distance from the current node n to the goal node. It is not concerned with the cost taken to reach the current node but purely focuses on the estimated remaining distance or effort to reach the goal.
   - For example, in a route-finding problem, $h(n)$ could be the straight-line distance from node n to the goal node.

   Purpose: The heuristic is used to prioritize nodes that appear to be closer to the goal, according to this estimate.

2. **Path Cost**:
   - In **Greedy Best First Search**, the algorithm does not directly consider the actual *path cost* (denoted as $g(n)$) from the start node to the current node. It only considers the heuristic value $h(n)$ to make decisions. This is a significant difference from **Uniform Cost Search** or *A Search**.
   - Therefore, the actual path cost is **not a factor** in the node selection process in Greedy Best First Search.

Summary of Differences:

- **Heuristic** $h(n)$: Represents an estimate of the remaining distance or effort from a given node to the goal.
- **Path Cost** $g(n)$: Represents the actual cost taken to reach the current node from the start node, but Greedy Best First Search **does not co** **er** this value during its decision-making.

RAMAIAH Institute of Technology

Greedy best-first search

1. Start with:
   Open List: {A}
   Closed List: {}
2. Explore node A:
   Open List: {B (374), C (329), E(253)}
   Closed List: {A}
3. Explore node E:
   Open List: {B (374), C (329), G(193),F(178)}
   Closed List: {A, E}
4. Explore node F:
   Open List: {B (374), C (329), G(193),I(0)}
   Closed List: {A,E, F}
5. Explore node G:
   Open List: {{B (374), C (329), G(193)}
   Closed List: {A,E, F,I} : **Goal node I reached!**

Reconstruct the Final Path:

Reconstruct the path using the Parent mapping:

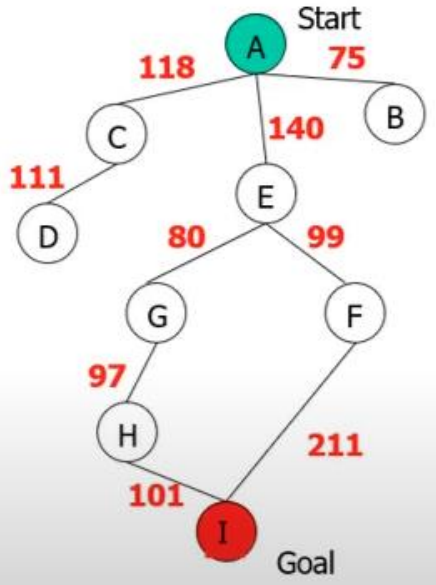
Start from node G, and look up its parent:

- Parent of I = F, Parent of F = E, Parent of F = A

Reverse the sequence to get the final path**:**

Path: A → E → F → I

Path Cost = 140+99+211=450



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

f(n) = h(n) **= straight-line distance heuristic**

Greedy best-first search: Drawbacks

- Greedy best-first search can start down an infinite path and never return to try other possibilities, it is incomplete
- Because of its greediness the search makes choices that can lead to a dead end; then one backs up in the search tree to the deepest unexpanded node
- Greedy best-first search resembles depth-first search in the way it prefers to follow a single path all the way to the goal, but will back up when it hits a dead end
- The quality of the heuristic function determines the practical usability of greedy search

RAMAIAH
Institute of Technology

Greedy best-first

Initialize:

Open list: P(10)

Closed list:{}

Expand node P

Open List: {R(6),C(4),A(11)}

Closed list:{

Expand node C

Open List: {M(9),U(4),R(8)}

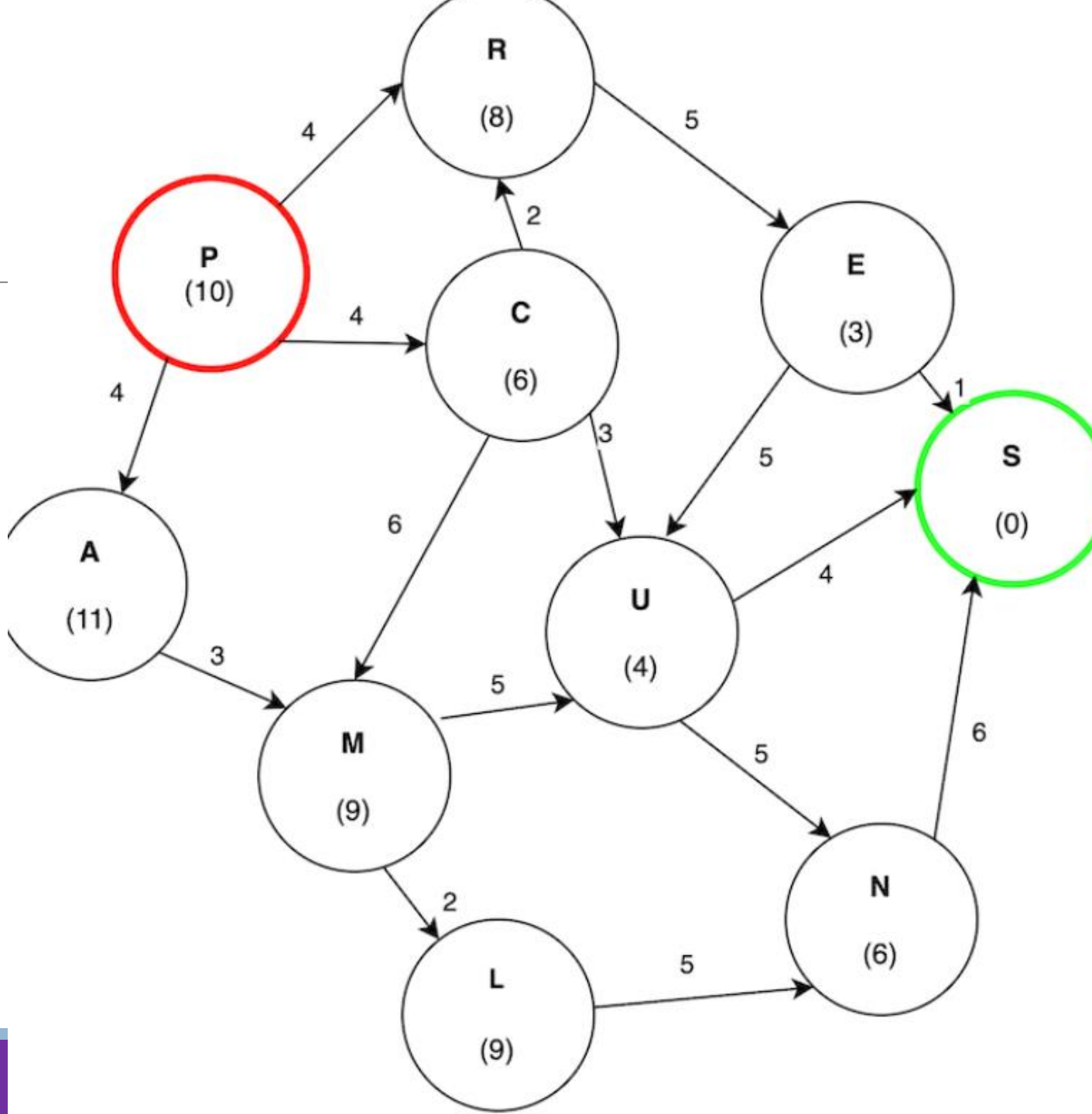
Closed list:{P,C}

Expand node U

Open List: {S(0),N(6)}

Closed list:{P,C,U}

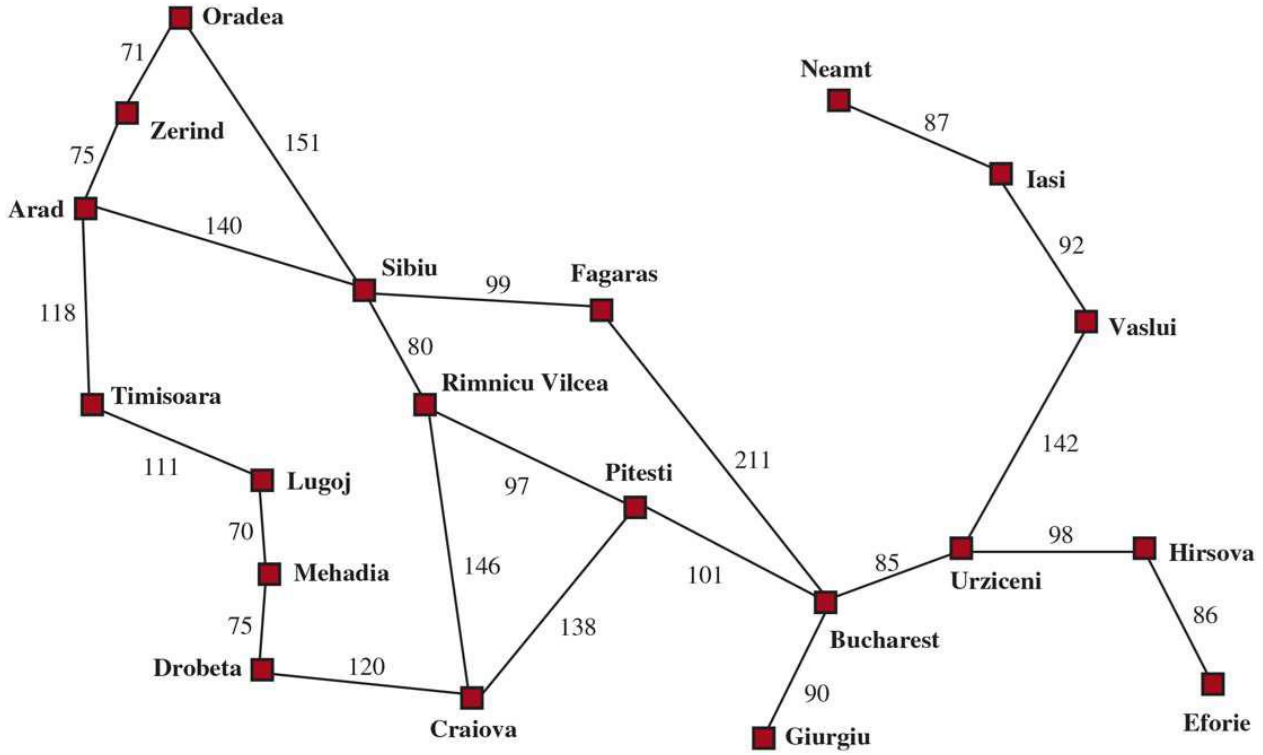
Path: P->C->U->S Path Cost: 4+3+4=11



RAMAIAH
Institute of Technology

Greedy best-first search

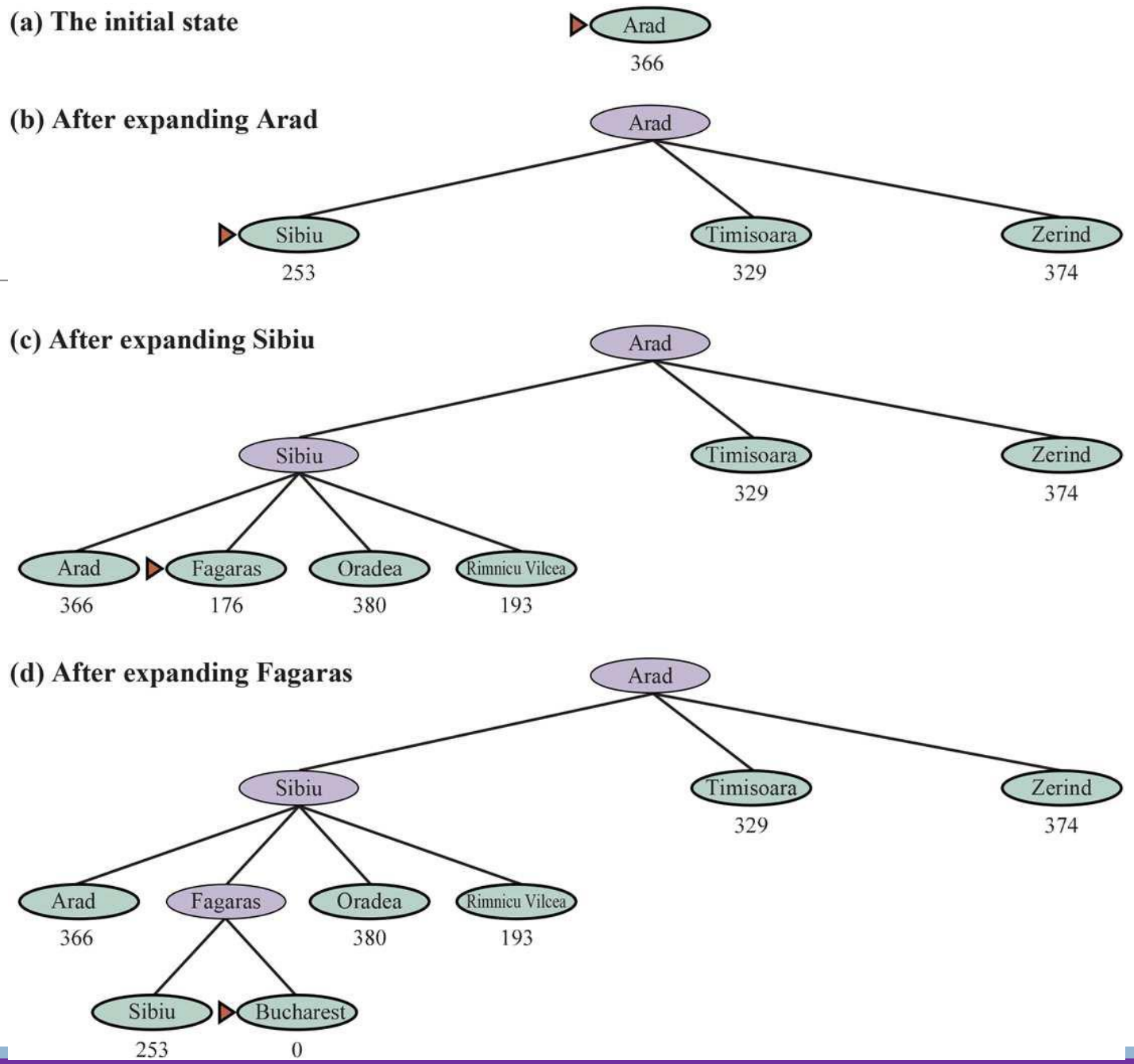
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



Values of h_{SLD} straight-line distances to Bucharest.

RAMAIAH Institute of Technology

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

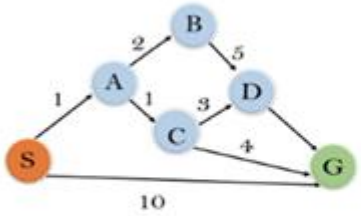


Path: Arad->Sibiu->Fagaras->Bucharest

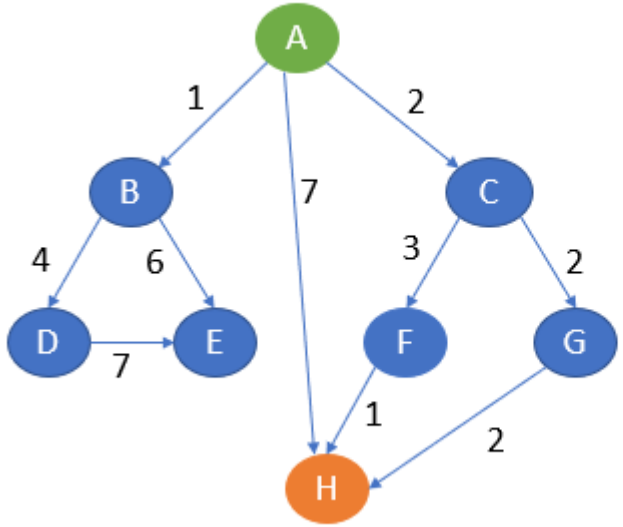
Path: Arad->Sibiu->Fagaras->Bucharest
Path Cost:
140+99+211= 450

RAMAIAH
Institute of Technology

Greedy best-first search Algorithm



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0



NODES	HEURISTICS
A	5
B	3
C	4
D	2
E	6
F	3
G	1
H	0

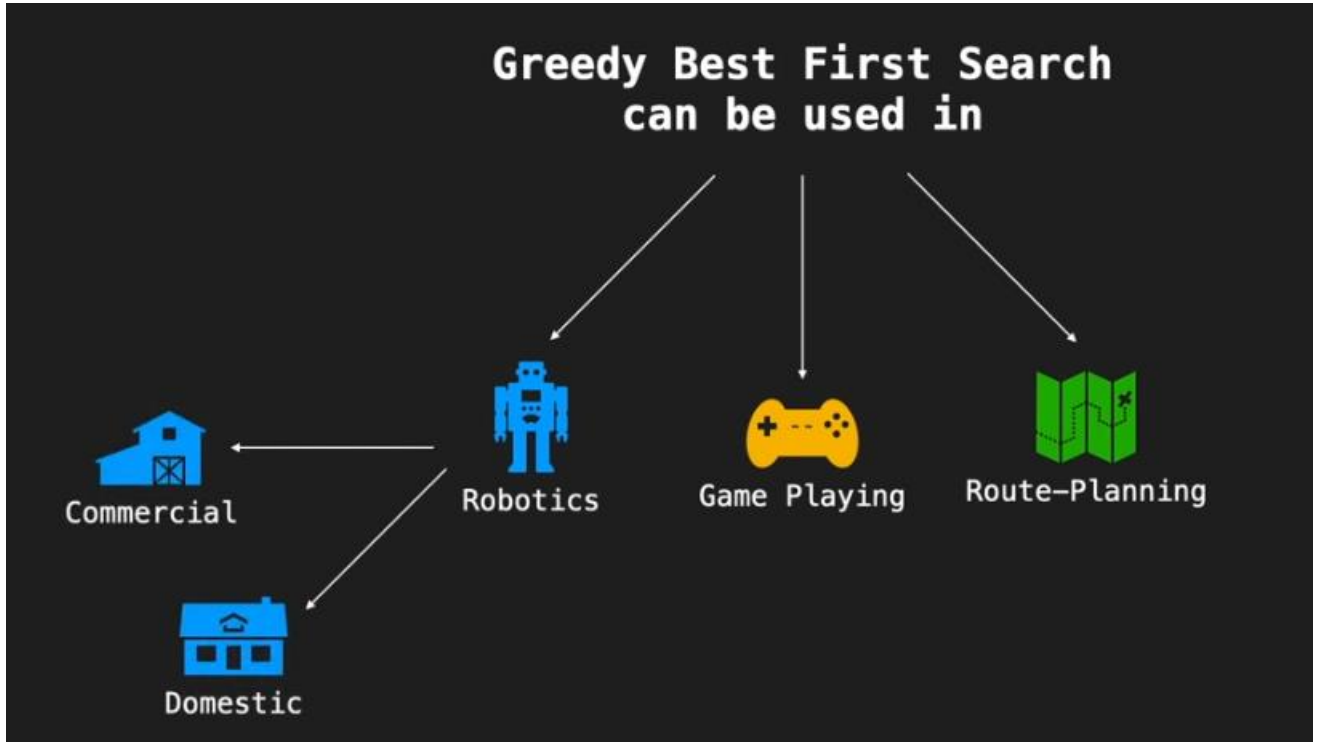
RAMAIAH Institute of Technology

```
function GreedyBestFirstSearch(start, goal):
    openList ← PriorityQueue()          // Nodes to explore, sorted by h(n)
    closedList ← Set()                  // Nodes already explored

    openList.push(start, h(start))      // Add start node to open list with priority h(start)

    while openList is not empty:
        current ← openList.pop()        // Get node with lowest h(n) from open list

        if current == goal:
            return reconstructPath(current) // Path found, reconstruct and return

        closedList.add(current)         // Mark current node as explored

        for each neighbor in neighbors(current):
            if neighbor in closedList:
                continue                // Skip neighbors already explored

            if neighbor not in openList:
                openList.push(neighbor, h(neighbor))  // Add new neighbor to open list with
h(neighbor)

    return "Failure"                    // Goal not reached
```

```
function reconstructPath(node):
    path ← []                          // Initialize empty path list
    while node is not null:
        path.prepend(node)             // Add current node to path
        node ← node.parent             // Move to parent node
    return path
```

RAMAIAH
Institute of Technology

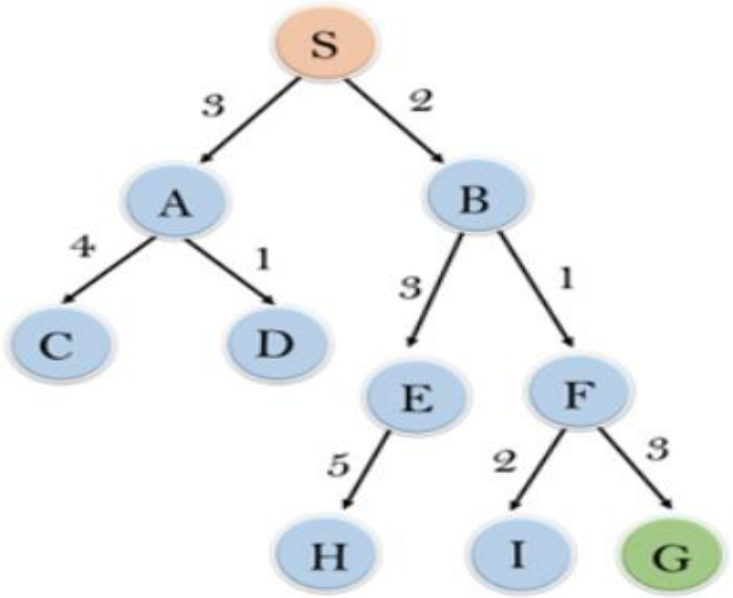
Greedy best-first search Applications



RAMAIAH
Institute of Technology

Greedy best-first search

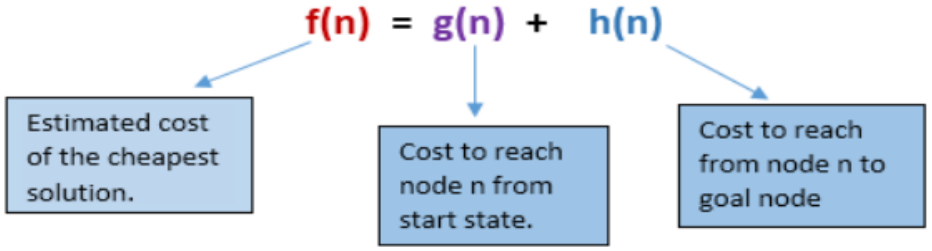
Final solution path will be ***S----> B----->F----> G = 2+1+3 =8***



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

A* search

- A* search is the most commonly known form of best-first search.
- It uses heuristic function h(n), and path cost to reach the node n from the start state g(n).
- A* search algorithm finds the shortest path through the search space using the heuristic function.
- A* search algorithm uses search heuristic as well as the cost to reach the node.

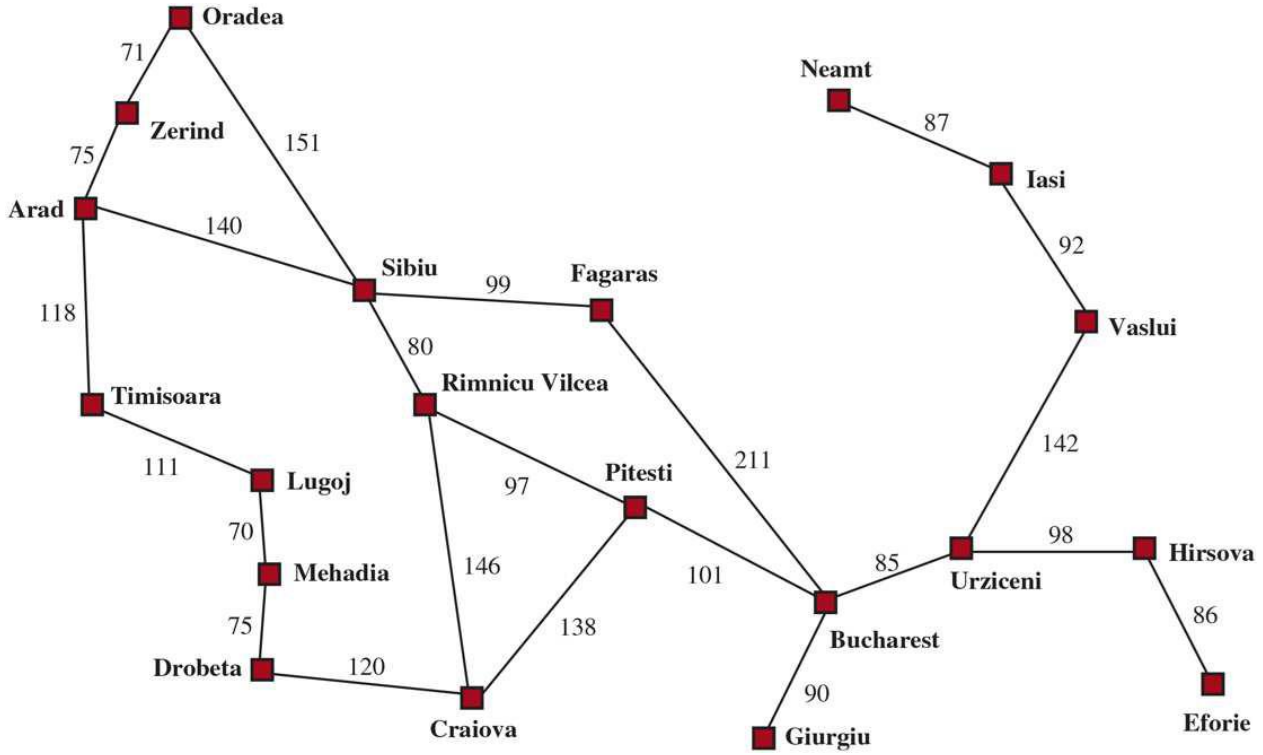


f(n) = estimated cost of the best path that continues from n to a goal.

RAMAIAH
Institute of Technology

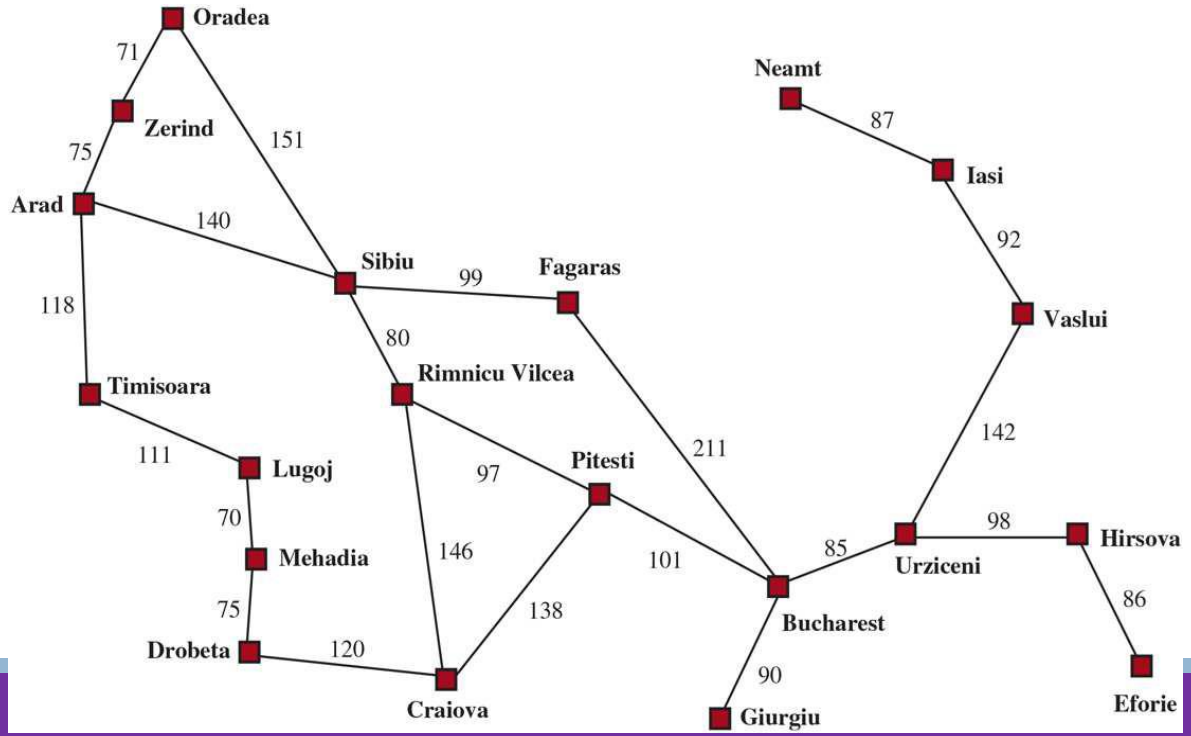
Greedy best-first search

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

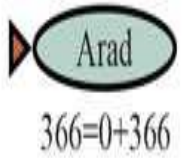


Values of hSLDstraight-line distances to Bucharest.

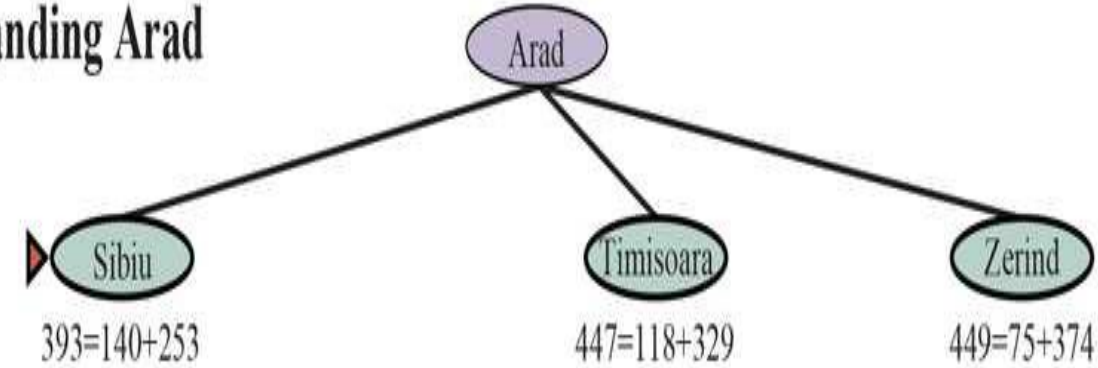
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



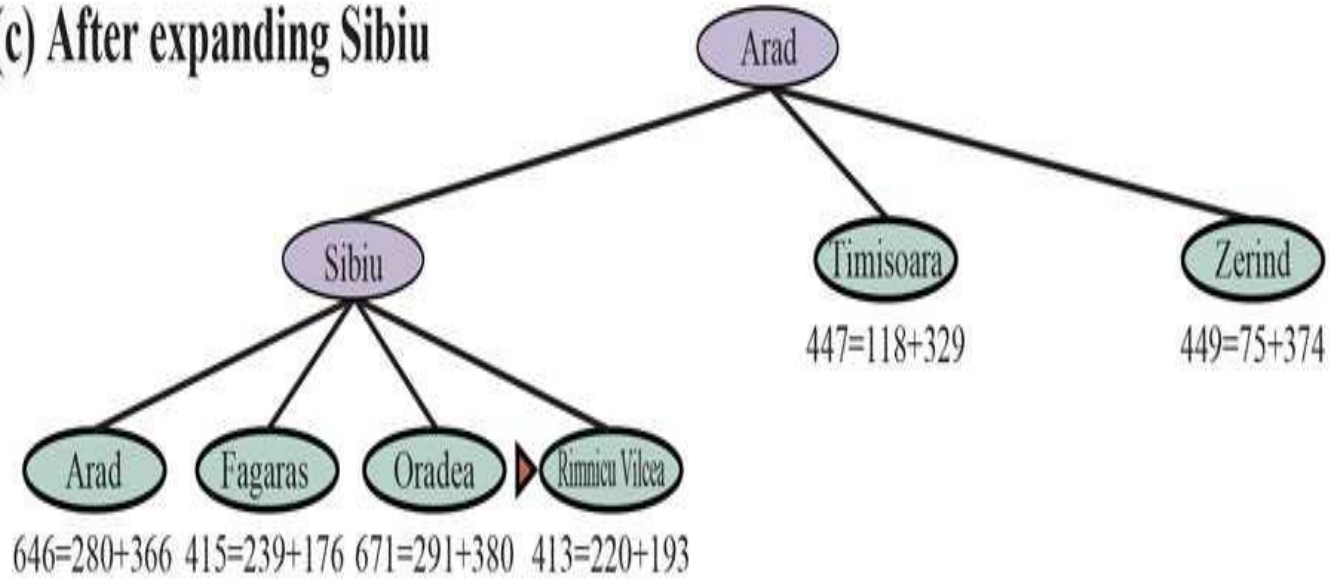
(a) The initial state



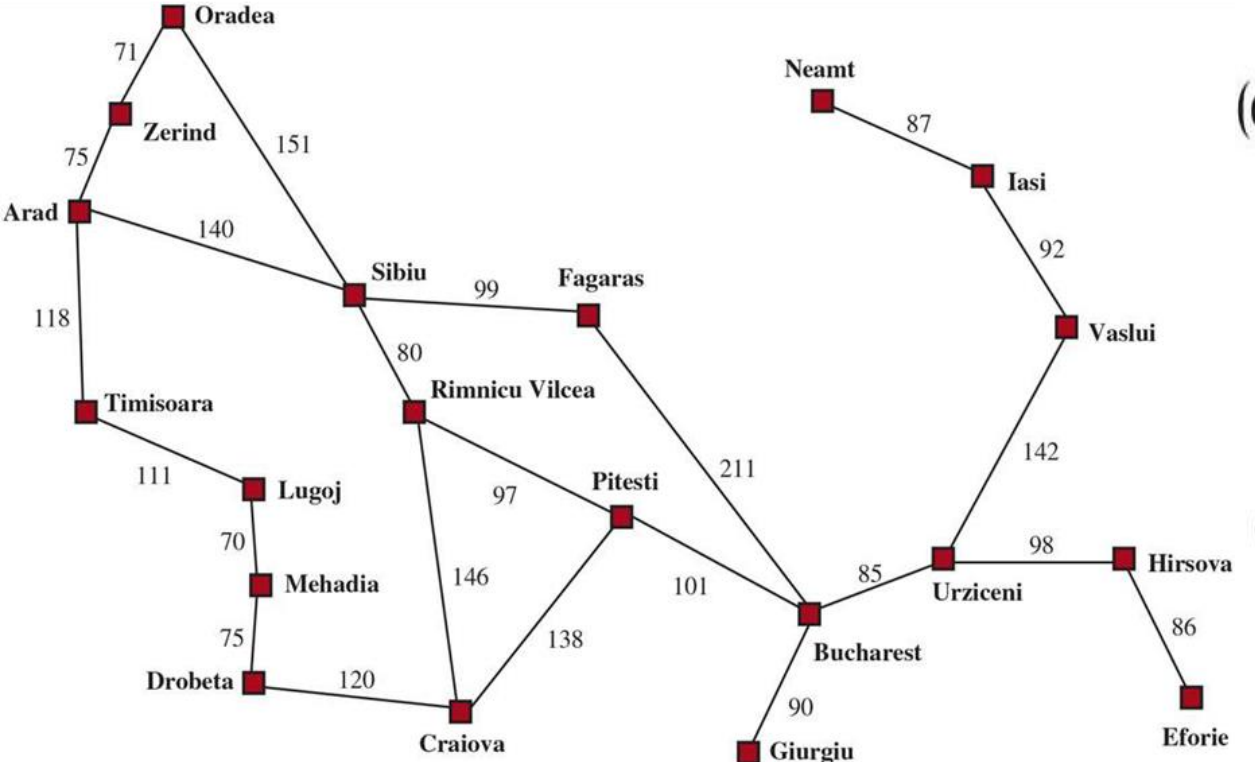
(b) After expanding Arad



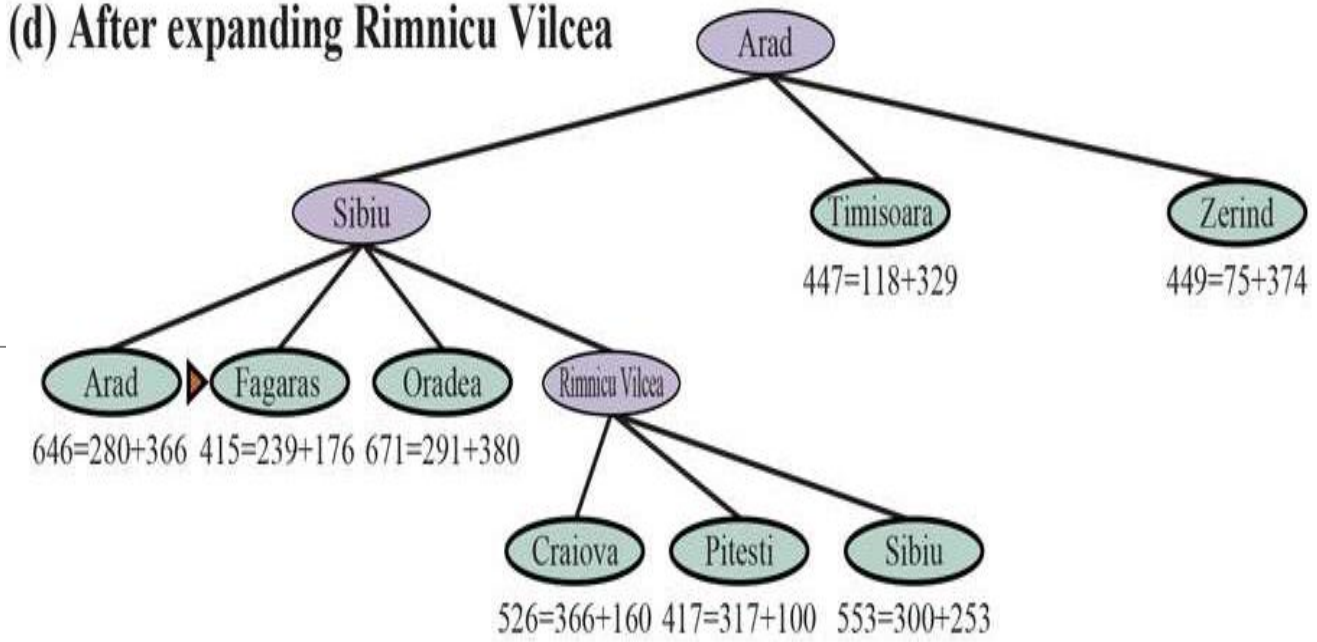
(c) After expanding Sibiu



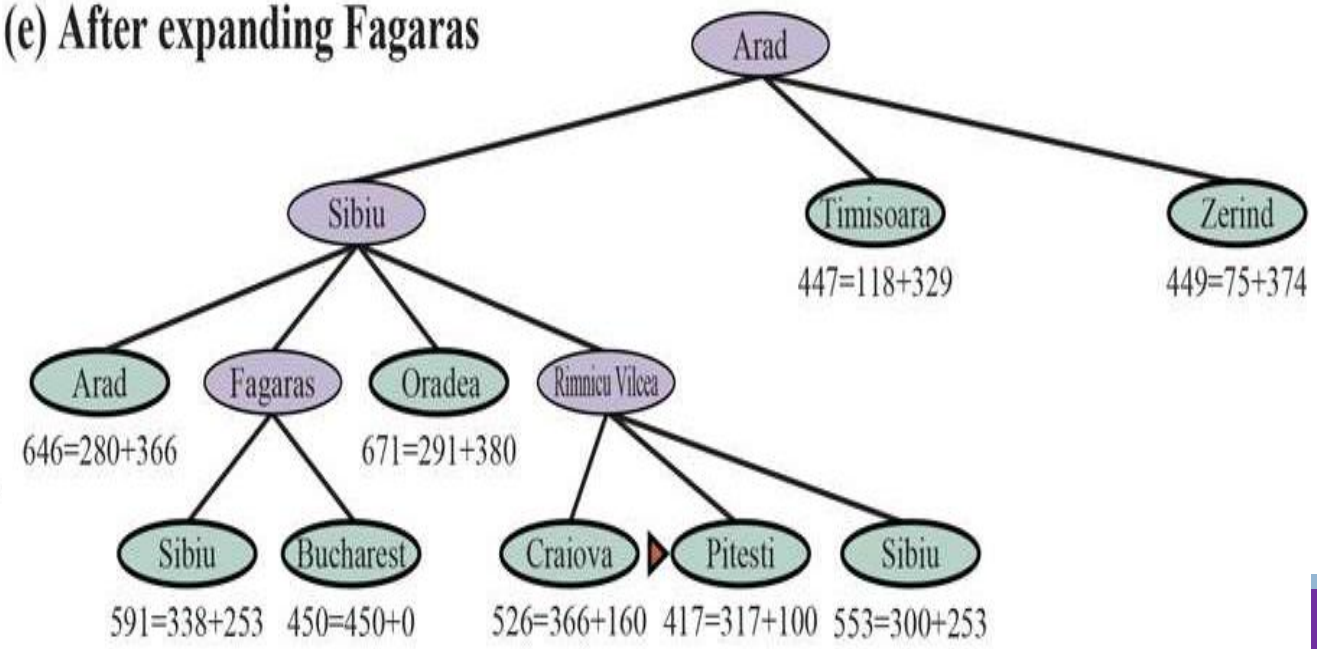
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



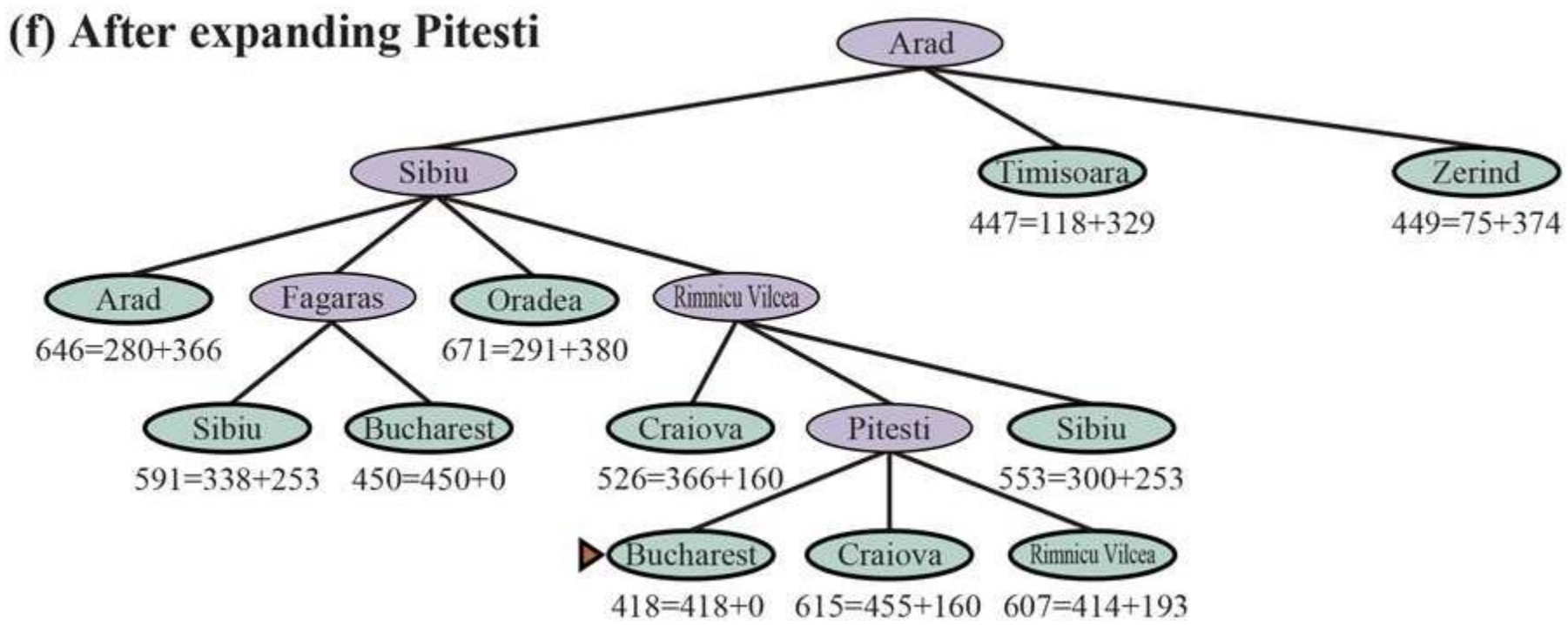
(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



A* search

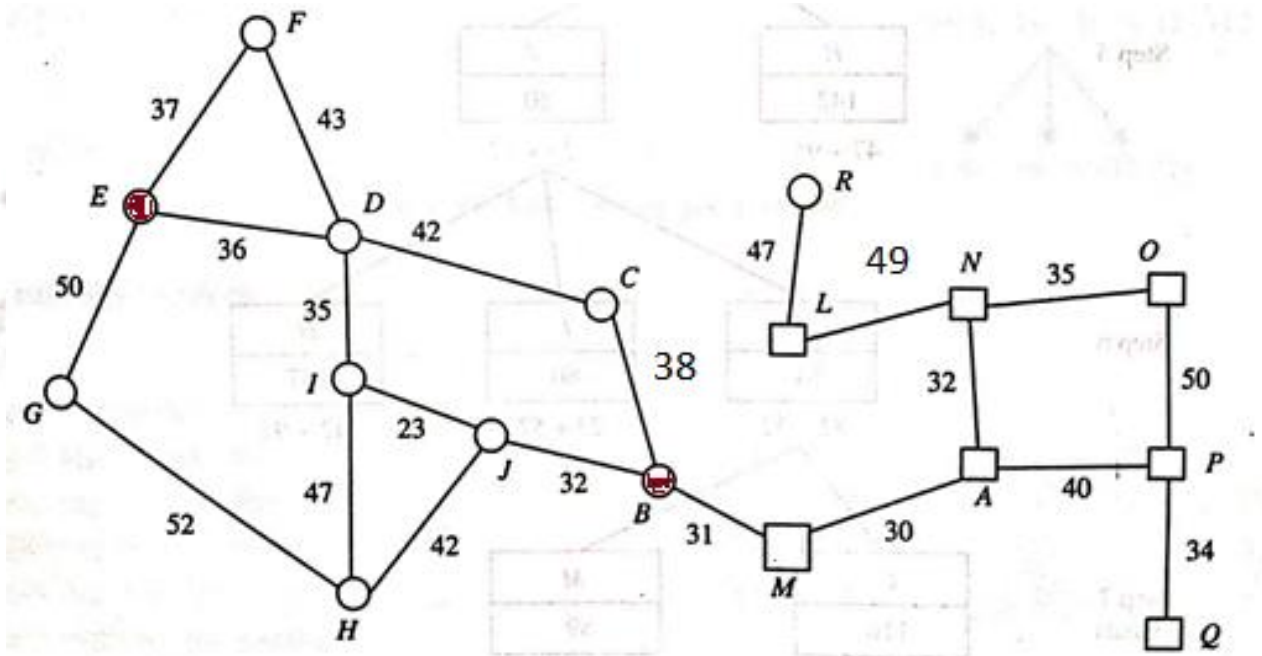

(f) After expanding Pitesti
Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
Sibiu
553=300+253
Bucharest
418=418+0
Craiova
615=455+160
Rimnicu Vilcea
607=414+193

RAMAIAH
Institute of Technology

Example: A* search

Heuristic Table

A 24	F 145	K 108	P 65
B 28	G 123	L 51	Q 11
C 70	H 95	N 58	
D 95	I 77	O 10	
E 118	J 57	M 0	

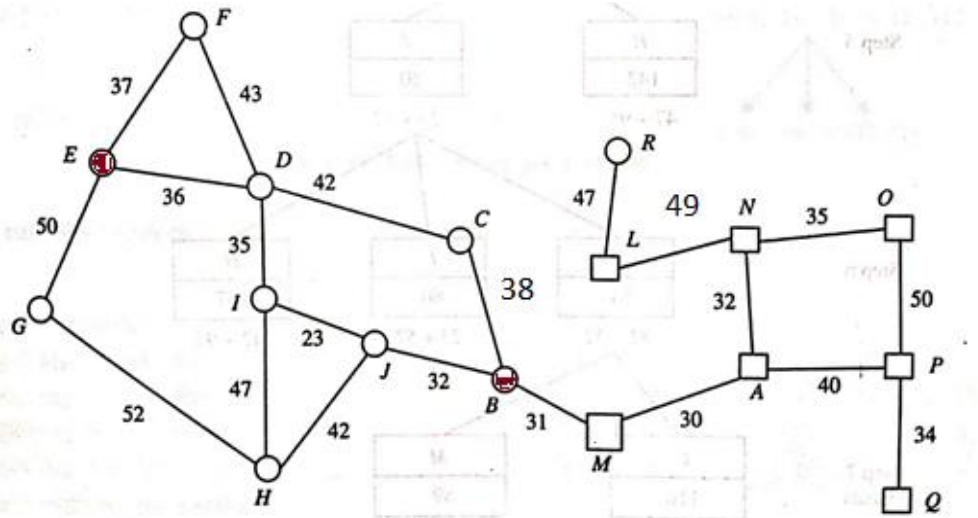


RAMAIAH
Institute of Technology

Example: A* search

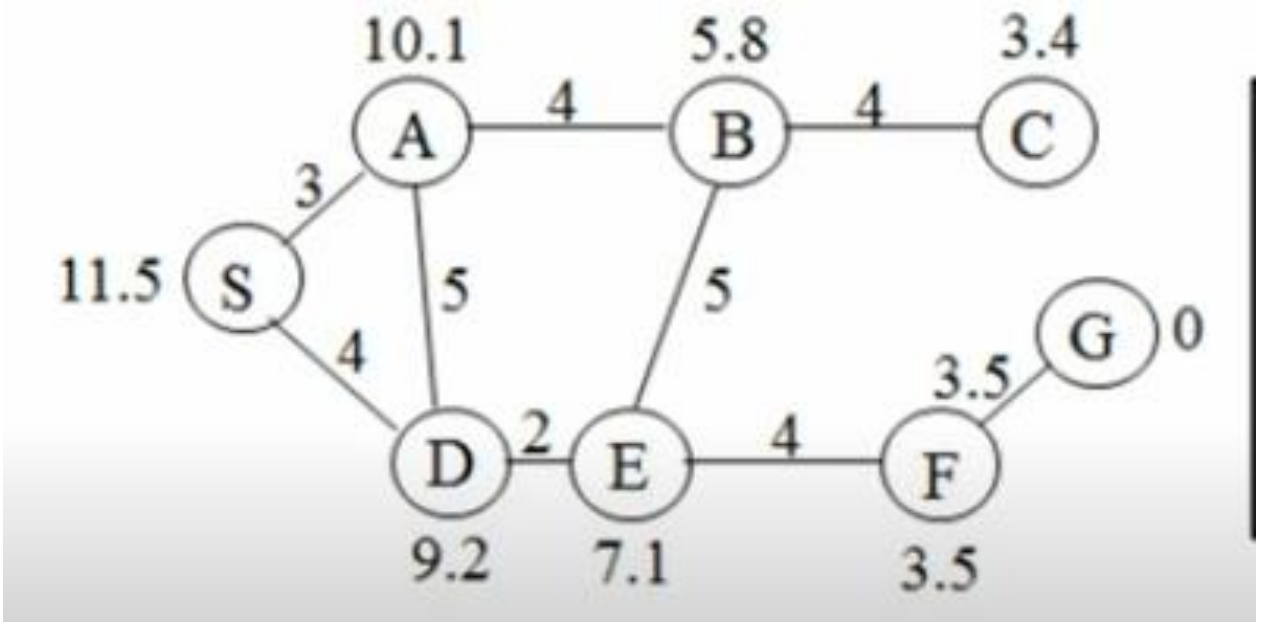
Initialize:
Open list: E(0+118)}
Closed list:{}
Expand node E
Open List: {F(37+145=182),D(36+95=131),G(50+123=173)}
Closed list:{E}
Expand node D
Open List: : {F(27+145=172),G(50+123=173),C(36+42+70=148),
I(36+35+77=148}
Closed list:{E,D}
Expand node C
Open List: : {F(27+145=172),G(50+123=173),
I(36+35+77=148),B(36+42+38+28=144)}
Closed list:{E,D,C}
Path: E->D->C->B
Path Cost: 36+42+38+28=144

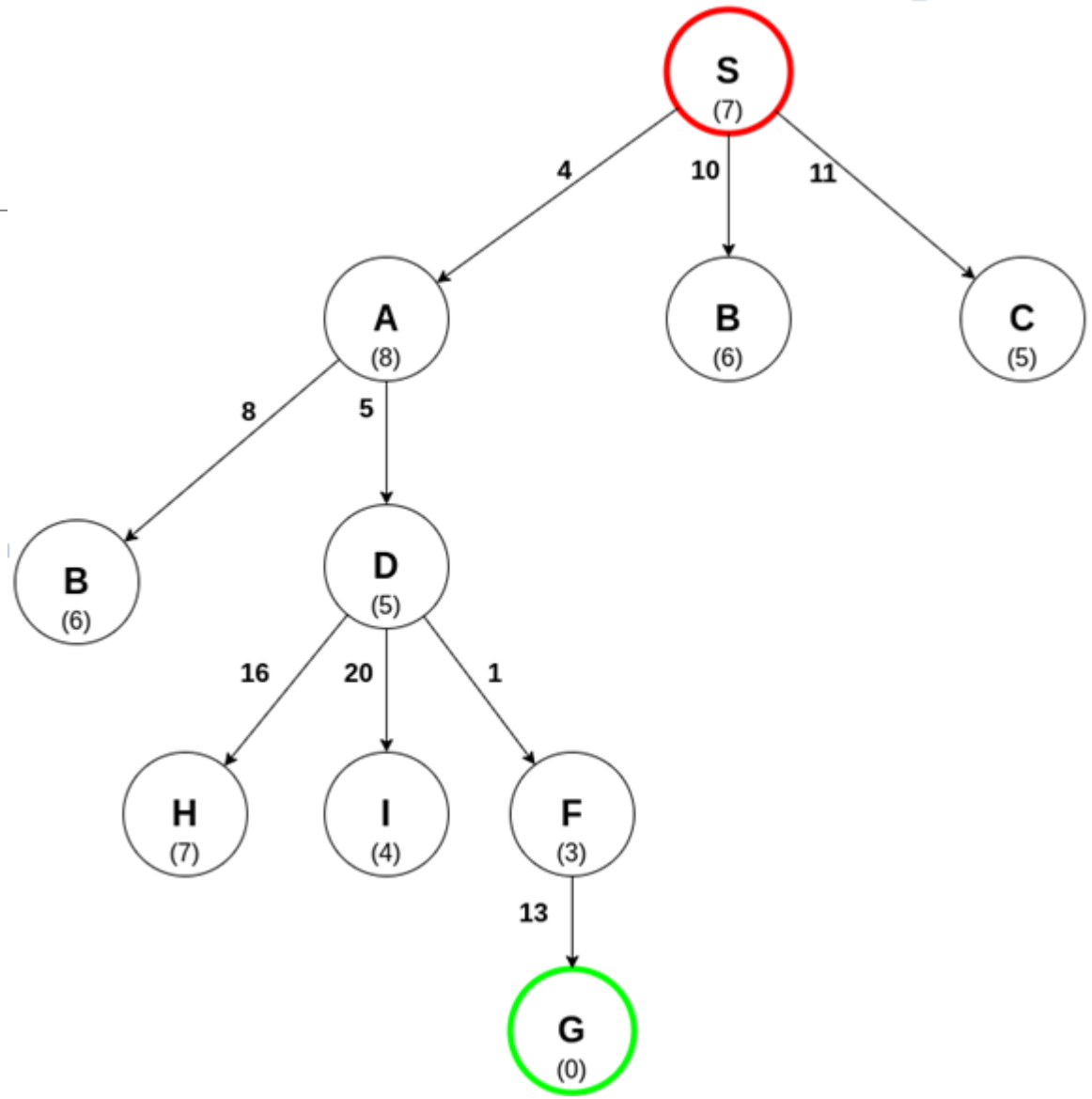
A 24	F 145	K 108	P 65
B 28	G 123	L 51	Q 11
C 70	H 95	N 58	
D 95	I 77	O 10	
E 118	J 57	M 0	



RAMAIAH
Institute of Technology

A* search from S to G





A* search

Advantages

- *A* search algorithm is the best algorithm than other search algorithms.*
- *A* search algorithm is optimal and complete.*
- *This algorithm can solve very complex problems.*

Disadvantages

- *It does not always produce the shortest path as it mostly based on heuristics and approximation.*
- *A* search algorithm has some complexity issues.*
- *The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.*

RAMAIAH
Institute of Technology

Comparison

Summary Table

Algorithm	Evaluation Function	Optimal?	Complete?	Heuristic Used?	Use Case
Best-First Search	Varies	Not necessarily	Not necessarily	Optional	General search problems
Uniform-Cost Search	$f(n) = g(n)$	Yes	Yes	No	Finding least-cost paths
Greedy Best-First	$f(n) = h(n)$	No	No	Yes	Fast pathfinding with good heuristic
*A Search**	$f(n) = g(n) + h(n)$	Yes (if h admissible)	Yes	Yes	Optimal pathfinding with good heuristic

RAMAIAH
Institute of Technology

Comparison

Choosing the Right Algorithm

- **Uniform-Cost Search**: Best when there's no reliable heuristic and you need the shortest path in terms of cost.
- **Greedy Best-First Search**: Suitable when speed is prioritized over path optimality, and you have a strong heuristic to guide the search.
- *A Search**: Preferred when an optimal solution is needed, especially if a good heuristic is available that approximates the cost to the goal well.

Measuring problem-solving performance

- **There are 4 criteria's used to choose among search algorithms.**
- **Algorithm's performance can be evaluated in four ways:**
- The evaluation of a search strategy in 4 ways:
  - **Completeness:**
    - is the strategy guaranteed to find a solution when there is one?
  - **Optimality**:
    - does the strategy find the highest-quality solution when there are several different solutions?
  - **Time complexity**:
    - how long does it take to find a solution?
  - **Space complexity**:
    - how much memory is needed to perform the search?

RAMAIAH
Institute of Technology

b = branching factor.

C^* = cost of the optimal solution.

ϵ = minimum edge cost.

Comparison

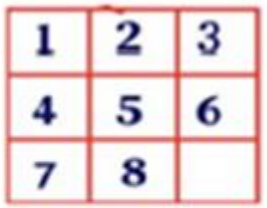
Parameter	Uniform Cost Search (UCS)	Greedy Best-First Search (GBFS)	A* (Optimal BFS)
Completeness	**Complete**	Not guaranteed	**Complete**
Optimality	**Optimal**	Not optimal	**Optimal** with admissible and consistent heuristic
Time Complexity	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^d)$	$O(b^d)$ (better with good heuristic)
Space Complexity	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^d)$	$O(b^d)$

RAMAIAH
Institute of Technology

8 Puzzle Problem

To solve the 8-puzzle problem using a Heuristic Search Algorithm, we will apply the A (A-Star)* search algorithm. A* uses a heuristic function to guide its search. The common heuristic function is: Misplaced Tile Heuristic (h1): Counts the number of tiles out of place compared to the goal state.

Initial State



Goal State

Initial State

```
1 2 3
4   6
7 5 8
```

Goal State:

```
1 2 3
4 5 6
7 8 .
```

Step 1: Calculate h1 for the Initial State

Misplaced tiles are tiles that are not in their correct position:

- **Initial State:** `5` and `8` are misplaced.
- h1 = 2.

Step 2: Possible Moves

From the initial state, the blank tile can move **Up**, **Down**, or **Left**. Let's evaluate each move:

1. Move Blank ~~Up~~ Right (swap blank with `6`):

```
1 2 3
4 6
7 5 8
```

Misplaced tiles: `5`, `6`, and `8`.

h1 = 3.

Initial State

```
1 2 3
4   6
7 5 8
```

Goal State:

```
1 2 3
4 5 6
7 8 .
```

2. Move Blank Down (swap blank with `5`):

```
1 2 3
4 5 6
7   8
```

Misplaced tiles: `8`.

h1 = 1.

3. Move Blank Left (swap blank with `4`):

```
1 2 3
  4 6
7 5 8
```

Misplaced tiles: `4`, `5`, and `8`.

h1 = 3.

Initial State

```
1 2 3
4   6
7 5 8
```

Goal State:

```
1 2 3
4 5 6
7 8 .
```

Step 3: Choose the Best Move

The move with the lowest $f(n) = g(n) + h1$ is chosen, where $g(n)$ is the cost to reach the current state (number of moves).

State	$g(n)$	$h1$	$f(n) = g(n) + h1$
Move Blank U**Right**	1	3	4
Move Blank Down	1	1	2
Move Blank Left	1	3	4

The **Move Blank Down** state is selected, as it has the lowest $f(n) = 2$.

Initial State

```
1 2 3
4   6
7 5 8
```

Goal State:

```
1 2 3
4 5 6
7 8 .
```

Step 4: New State

After moving the blank tile down:

```
1 2 3
4 5 6
7   8
```

Calculate $h1$ for this state:

- Misplaced tiles: `8`.
- h1 = 1.

RAMAIAH
Institute of Technology

8 Puzzle Problem

Step 5: Possible Moves

1. Move Blank Right (swap blank with `8`):

```
1 2 3
4 5 6
7 8
```

Misplaced tiles: None.

h1 = 0.

Step 6: Goal State Reached

When $h1 = 0$, the goal state is reached:

```
1 2 3
4 5 6
7 8
```

RAMAIAH
Institute of Technology

8 Puzzle Problem

Initial State

Goal State

Performance Comments

1. **Heuristic Accuracy:**
   The misplaced tile heuristic is simple but not very accurate because it doesn't account for the distance of tiles to their goal position. For example, a tile far from its correct position contributes the same value as one nearby.

2. **Search Depth:**
   A better heuristic like Manhattan Distance reduces the search depth and number of expanded nodes compared to Misplaced Tile.

RAMAIAH
Institute of Technology

8 Puzzle Problem

Apply the Heuristic Search Algorithm for 8 puzzle problem to reach the goal state from the given initial state. Draw the state space diagram. Also comment on the parameters which contribute to the performance of Heuristic search algorithms.

1	2	3
	4	6
7	5	8

Initial State

1	2	3
4	5	6
7	8	

Goal State

1	2	3
	4	6
7	5	8

Initial State

1	2	3
4	5	6
7	8	

Goal State

Step 1: Initial State

For the initial state, misplaced tiles are:

- `5` and `8` are misplaced.

h1 = 2.

The blank tile can move in three directions:

1. **Up** (swap blank with `6`)
2. **Down** (swap blank with `5`)
3. **Left** (swap blank with `4`).

Step 2: Possible Moves and Heuristic Values

1. Move Blank Up (swap blank with `6`):

```
1 2 3
4 6
7 5 8
```

Misplaced tiles: `6`, `5`, and `8`.

h1 = 3.

Initial State

Goal State

2. Move Blank Down (swap blank with `5`):

```
1 2 3
4 5 6
7   8
```

Misplaced tiles: `8`.

h1 = 1.

3. Move Blank Left (swap blank with `4`):

```
1 2 3
  4 6
7 5 8
```

Misplaced tiles: `4`, `5`, and `8`.

h1 = 3.

Initial State

Goal State

Step 3: Choose the Best Move

The move with the lowest $f(n) = g(n) + h1$ is chosen, where $g(n)$ is the cost to reach the current state (number of moves so far).

Move	$g(n)$	$h1$	$f(n) = g(n) + h1$
Move Blank Up	1	3	4
Move Blank Down	1	1	2
Move Blank Left	1	3	4

The **Move Blank Down** state is selected, as it has the lowest $f(n) = 2$.

Step 4: New State

After moving the blank tile down:

```
1 2 3
4 5 6
7   8
```

Misplaced tiles: `8`.

h1 = 1.

Step 5: Possible Moves

1. Move Blank Right (swap blank with `8`):

```
1 2 3
4 5 6
7 8
```

Misplaced tiles: None.

h1 = 0.

Initial State

Goal State

Step 6: Goal State Reached

When $h1 = 0$, the goal state is achieved:

```
1 2 3
4 5 6
7 8
```

State Space Diagram

Below is the state space diagram that demonstrates the traversal:

```sql
Start: 1 2 3      Move Down (Best Move)    Goal State
       4   6  ------------------------->  1 2 3
       7 5 8                               4 5 6
                                           7 8
```